ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐỒ ÁN TỐT NGHIỆP

Xây dựng Sandbox phát hiện mã độc từ file

NGUYỄN XUÂN BÁCH
bach.nx225259@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật Máy tính

Giảng viên hướng dẫn: TS. Lê Xuân Thành

Khoa: Kỹ thuật Máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐỒ ÁN TỐT NGHIỆP

Xây dựng Sandbox phát hiện mã độc từ file

NGUYỄN XUÂN BÁCH
bach.nx225259@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật Máy tính

Giảng viên hướng dẫn: TS. Lê Xuân Thành _____

Chữ kí GVHD

Khoa: Kỹ thuật Máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp này, em đã nhận được sự hỗ trợ và giúp đỡ từ nhiều phía. Trước tiên, em xin gửi lời cảm ơn chân thành đến TS. Lê Xuân Thành, người đã định hướng, góp ý và theo sát tiến độ thực hiện đồ án từ những bước đầu tiên đến khi hoàn thành. Em cũng xin cảm ơn các thầy cô trong Khoa Kỹ thuật Máy tính, Trường Công nghệ Thông tin và Truyền Thông, Đại học Bách khoa Hà Nội đã trang bị cho em nền tảng kiến thức cần thiết trong suốt những năm học vừa qua. Cuối cùng, em xin cảm ơn gia đình và bạn bè đã tạo điều kiện và động viên em trong suốt quá trình thực hiện đồ án. Do kiến thức và kinh nghiệm còn hạn chế, đồ án không tránh khỏi thiếu sót. Em rất mong nhận được ý kiến đóng góp từ thầy cô và hội đồng phản biện.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Phần mềm độc hại trên nền tảng Windows gia tăng nhanh về số lượng và mức độ tinh vi, trong khi phương pháp phát hiện dựa trên chữ ký không nhận diện được các biến thể mới chưa từng ghi nhận. Phân tích tĩnh phân loại nhanh dựa trên đặc trưng cấu trúc tệp nhưng dễ bị vô hiệu hóa bởi kỹ thuật đóng gói và làm rối mã; phân tích động quan sát được hành vi thực tế nhưng tiêu tốn nhiều tài nguyên. Mỗi hướng tồn tại hạn chế khi đứng độc lập, do đó đồ án kết hợp cả hai trong một hệ thống thống nhất.

Giai đoạn tĩnh trích xuất bộ đặc trưng PE 2.381 chiều theo chuẩn EMBER và huấn luyện mô hình LightGBM trên tập dữ liệu tự thu thập từ nhiều nguồn công khai. Khảo sát số chiều cho thấy hiệu năng phân loại bão hòa từ khoảng 64 đặc trưng, nên mô hình rút gọn còn 384 chiều vẫn đạt F1 bằng 0,9615 trên tập dữ liệu EMBER 2024. Ngưỡng phân loại được đặt ở 0,33 nhằm ưu tiên giảm tỉ lệ bỏ sót mã độc.

Giai đoạn động thực thi mẫu trong môi trường sandbox xây dựng trên VirtualBox và thu thập sự kiện đồng thời bằng Sysmon và SilkETW; trong đó SilkETW bổ sung khả năng quan sát hoạt động nạp assembly trong .NET Runtime mà Sysmon không ghi nhận. Các sự kiện được chuẩn hóa, dựng thành cây tiến trình và chuỗi hành vi, trích xuất chỉ số xâm phạm và ánh xạ tự động sang khung MITRE ATT&CK. Toàn bộ quy trình vận hành tự động từ khâu nhận tệp đến khâu xuất báo cáo và được trình bày qua giao diện web.

Đóng góp chính của đồ án là một pipeline phân tích phần mềm độc hại tự động, kết hợp kết quả của hai giai đoạn thành một báo cáo thống nhất gồm kết luận, chỉ số xâm phạm, cây tiến trình và kỹ thuật MITRE ATT&CK. Khác với các sandbox phổ biến như Cuckoo hay Any.Run vốn can thiệp lời gọi API ở user-mode, đồ án khai thác trực tiếp ETW provider của .NET Runtime, nhờ đó quan sát được hoạt động nạp và sinh mã .NET trong bộ nhớ mà cả phân tích tĩnh lẫn Sysmon đều bỏ sót. Trên 205 mẫu mã độc thuộc bảy nhóm hành vi, hệ thống nhận diện đúng 204 mẫu và thu được bằng chứng hành vi cụ thể ở 72% số mẫu. Kết quả thực nghiệm đầy đủ được trình bày trong Chương 5.

Sinh viên thực hiện
Nguyễn Xuân Bách

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Đặt vấn đề 1

1.2 Mục tiêu và phạm vi đề tài 1

1.3 Định hướng giải pháp 2

1.4 Bố cục đồ án 2

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3

2.1 Phần mềm độc hại 3

2.2 Định dạng tệp PE và đặc trưng tĩnh 4

2.2.1 Cấu trúc tệp PE 4

2.2.2 Đặc trưng tĩnh dùng trong phát hiện mã độc 5

2.3 Phân tích tĩnh dựa trên học máy 6

2.3.1 Hướng tiếp cận học máy trong phát hiện mã độc 6

2.3.2 Các thuật toán phân loại 7

2.3.3 Lựa chọn đặc trưng dựa trên Importance Gain 7

2.4 Phân tích động và giám sát hành vi 8

2.4.1 Khái niệm sandbox và phân tích động 8

2.4.2 Sysmon 8

2.4.3 ETW và SilkETW 10

2.4.4 Chuỗi hành vi và chỉ số xâm phạm 11

2.4.5 Khung MITRE ATT&CK 12

2.5 Các công trình liên quan 12

CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG 14

3.1 Phân tích yêu cầu 14

3.1.1 Yêu cầu chức năng 14

3.1.2 Yêu cầu phi chức năng 14
3.2 Kiến trúc tổng thể 15
3.3 Thiết kế giai đoạn phân tích tĩnh 16
3.3.1 Thu thập và xây dựng tập dữ liệu 16
3.3.2 Trích xuất đặc trưng 16
3.3.3 Huấn luyện mô hình 17
3.4 Thiết kế giai đoạn phân tích động 17
3.4.1 Thiết lập môi trường sandbox 18
3.4.2 Thực thi mẫu 19
3.4.3 Thu thập sự kiện 19
3.4.4 Chuẩn hóa và lọc nhiễu 20
3.4.5 Xây dựng biểu diễn hành vi và ánh xạ MITRE ATT&CK 21
3.5 Thiết kế pipeline tích hợp 22
3.5.1 Luồng xử lý end-to-end 23
3.5.2 Cấu trúc báo cáo đầu ra 23
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 24
4.1 Môi trường triển khai 24
4.1.1 Máy ảo Windows 24
4.1.2 Cấu hình mạng và INetSim 25
4.2 Triển khai giai đoạn phân tích tĩnh 26
4.2.1 Thu thập và xây dựng tập dữ liệu 26
4.2.2 Trích xuất đặc trưng 27
4.2.3 Huấn luyện và lưu mô hình 27
4.3 Triển khai giai đoạn phân tích động 29
4.3.1 Cấu hình Sysmon 29
4.3.2 Cấu hình SilkETW 30

4.3.3 Agent giám sát trong VM và giao tiếp với host 31
4.3.4 Chuẩn hóa và lọc nhiễu 32
4.3.5 Xây dựng chuỗi hành vi, IOC và ánh xạ MITRE ATT&CK 33
4.4 Triển khai pipeline tích hợp 35
4.4.1 Luồng xử lý song song 35
4.4.2 Kết hợp kết quả và xác định kết luận 36
4.4.3 Xuất báo cáo đầu ra 37
4.5 Giao diện web demo 38
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ 42
5.1 Thiết lập thực nghiệm 42
5.2 Đánh giá giai đoạn phân tích tĩnh 42
5.2.1 So sánh ba thuật toán 42
5.2.2 Lựa chọn tập con đặc trưng 43
5.2.3 Tối ưu ngưỡng phân loại 45
5.2.4 Phân tích đặc trưng 46
5.2.5 Phân tích lỗi 47
5.2.6 Hiệu chuẩn độ tin cậy 48
5.3 Đánh giá giai đoạn phân tích động 49
5.3.1 Độ phủ bằng chứng hành vi 49
5.3.2 Bằng chứng hành vi qua các mẫu đại diện 51
5.3.3 Đánh giá pipeline tích hợp 51
5.3.4 Phân tích sâu kết quả tích hợp 52
KẾT LUẬN 55
TÀI LIỆU THAM KHẢO 59
PHỤ LỤC 61

A. NGUỒN DỮ LIỆU 61

A.1 Thống kê mẫu lành tính theo nguồn 61

A.2 Thống kê mẫu mã độc theo nguồn 61

B. QUY TẮC ÁNH XẠ MITRE ATT&CK VÀ THAM CHIẾU KỸ THUẬT 62

B.1 Sysmon Event ID được theo dõi 62

B.2 Các luật ánh xạ MITRE ATT&CK đã triển khai 62

B.3 Endpoint và sự kiện SSE của server web demo 64

B.4 Danh sách lọc nhiễu trong giai đoạn động 64

B.5 Endpoint của Agent HTTP trong máy ảo 66

B.6 Ánh xạ sự kiện chuẩn hóa vào nhóm hành vi 66

B.7 Cấu trúc báo cáo JSON đầu ra của pipeline 67

DANH MỤC HÌNH VẼ

Hình 2.1 Cấu trúc tệp PE 32-bit [17]. 4
Hình 2.2 Quy trình phát hiện mã độc bằng học máy. 6
Hình 2.3 Minh họa cách tính Importance Gain. 8
Hình 2.4 Minh họa kiến trúc Sysmon. 9
Hình 2.5 So sánh phạm vi giám sát của Sysmon và ETW. 11

Hình 3.1 Kiến trúc tổng thể của hệ thống phân tích phần mềm độc hại. . 15
Hình 3.2 Luồng phân tích tĩnh khi tiếp nhận tệp đầu vào 16
Hình 3.3 Quy trình xây dựng mô hình phân loại tĩnh 16
Hình 3.4 Luồng xử lý giai đoạn phân tích động. 18
Hình 3.5 Kiến trúc môi trường sandbox. 18
Hình 3.6 Luồng thực thi mẫu trong môi trường sandbox. 19
Hình 3.7 Luồng thu thập sự kiện trong môi trường sandbox. 20
Hình 3.8 Luồng chuẩn hóa và lọc nhiễu dữ liệu. 21
Hình 3.9 Luồng xử lý end-to-end của pipeline tích hợp. 22

Hình 4.1 Cấu hình phần cứng máy ảo `win10` trong VirtualBox Settings. 25
Hình 4.2 Cấu hình adapter mạng host-only của máy ảo trong VirtualBox Settings. 25
Hình 4.3 Sơ đồ quá trình xây dựng ma trận đặc trưng từ tệp PE thô. . . . 26
Hình 4.4 Kênh sự kiện `Microsoft-Windows-Sysmon/Operational` trong Windows Event Viewer. 29
Hình 4.5 Các sự kiện Sysmon thô trong tệp `sysmon_events.jsonl` sau khi kéo từ guest về host. 30
Hình 4.6 Các sự kiện ETW thô về nạp assembly .NET trong tệp `etw_dotnet.json` sau khi kéo từ guest về host. 31
Hình 4.7 Giao diện tài liệu API tự động (Swagger UI) của Agent tại cổng 8001. 31
Hình 4.8 Sự kiện Sysmon EID 1 trước và sau chuẩn hóa. 32
Hình 4.9 Sự kiện SilkETW `Method/LoadVerbose` trước và sau chuẩn hóa. 33
Hình 4.10 Ví dụ cây tiến trình sau khi lọc nhiễu của một mẫu phân tích. . 34
Hình 4.11 Ví dụ đầu ra chuỗi hành vi của một mẫu phân tích. 34
Hình 4.12 Ví dụ kết quả ánh xạ MITRE ATT&CK của một mẫu phân tích. 35
Hình 4.13 Log thực thi pipeline tích hợp trong một phiên phân tích. . . . 36

Hình 4.14 Ví dụ báo cáo JSON đầu ra của một phiên phân tích mẫu mã độc. 38
Hình 4.15 Trang Analyze: vùng kéo-thả và biểu mẫu cấu hình môi trường phân tích. 39
Hình 4.16 Trang Pipeline: 12 giai đoạn phân tích. 39
Hình 4.17 Trang Reports: bảng tổng hợp kết quả các phiên phân tích đã lưu. 40
Hình 4.18 Trang Report Detail: báo cáo chi tiết 1 mẫu phân tích. 40

Hình 5.1 Đường cong ROC của ba thuật toán trên tập holdout, trục FPR theo thang log. 43
Hình 5.2 Đánh đổi số đặc trưng và hiệu năng trên EMBER 2024 qua mười lăm mốc; vòng đỏ đánh dấu ba ngưỡng 128/256/384, đường chấm dọc là điểm gãy (knee) tại 64 đặc trưng. 44
Hình 5.3 Tỉ lệ phát hiện theo nhóm của bốn cấu hình đặc trưng. 45
Hình 5.4 Biến thiên FPR và FNR theo ngưỡng phân loại. 46
Hình 5.5 Tổng giá trị SHAP tuyệt đối theo nhóm đặc trưng. 47
Hình 5.6 Phân bố các trường hợp phân loại sai theo khoảng điểm số (FN bên trái, FP bên phải). 48
Hình 5.7 Reliability diagram của mô hình Top-384 trước và sau hiệu chuẩn trên tập EMBER 2024. 49
Hình 5.8 Phân mức bằng chứng động trên 205 mẫu mã độc. 50
Hình 5.9 Độ phủ các loại hành vi thu được. 50
Hình 5.10 Tính bù trừ giữa giai đoạn tĩnh và động trên 205 mẫu mã độc. 54

DANH MỤC BẢNG BIỂU

Bảng 2.1 So sánh các nhóm phần mềm độc hại và hành vi quan sát được trong sandbox, tổng hợp từ các nguồn [7], [10], [11], [12], [13], [14], [15], [16]. 3
Bảng 2.2 Chín nhóm đặc trưng tĩnh và số chiều tương ứng [17], [18]. . . 5
Bảng 2.3 Các Event ID của Sysmon được sử dụng trong đồ án. 10
Bảng 2.4 So sánh đặc điểm giữa đồ án và các hệ thống phân tích động hiện có [7]. 13

Bảng 3.1 Yêu cầu chức năng của hệ thống. 14
Bảng 3.2 Yêu cầu phi chức năng của hệ thống. 15

Bảng 4.1 Cấu hình phần cứng và phần mềm môi trường triển khai. . . . 24
Bảng 4.2 Thành phần tập dữ liệu huấn luyện cuối cùng (`X_final.npy`, 2.381 đặc trưng/mẫu). 27
Bảng 4.3 Siêu tham số ba thuật toán trong lần huấn luyện thứ nhất (toàn bộ 2.381 đặc trưng, tập train 161.312 mẫu). 28
Bảng 4.4 Siêu tham số LightGBM trong lần huấn luyện thứ hai (tập con đặc trưng, 5-fold CV trên 201.640 mẫu). 28
Bảng 4.5 Tham số cấu hình SilkETW trong mỗi phiên phân tích. 30
Bảng 4.6 Quy tắc kết hợp kết quả và tính độ tin cậy tổng hợp. 37

Bảng 5.1 Ba tập dữ liệu dùng trong đánh giá giai đoạn phân tích tĩnh. . 42
Bảng 5.2 So sánh ba thuật toán trên tập holdout 40.328 mẫu (toàn bộ 2.381 đặc trưng). 43
Bảng 5.3 So sánh bốn cấu hình đặc trưng trên tập EMBER 2024 (119.992 mẫu). 44
Bảng 5.4 Tỉ lệ phát hiện theo nhóm của mô hình Top-384 (tập kiểm thử riêng). 45
Bảng 5.5 So sánh ba mốc ngưỡng phân loại trên tập EMBER 2024. . . . 46
Bảng 5.6 Đối chiếu điểm dự đoán với tỉ lệ thực tế (trích các khoảng tiêu biểu, thống kê trên nửa tập kiểm EMBER 2024 dùng đánh giá hiệu chuẩn, khoảng 60.000 mẫu). Vùng điểm cực thấp và cực cao khớp tốt, vùng giữa tự tin thái quá. 48
Bảng 5.7 Bằng chứng hành vi của các mẫu mã độc đại diện. 51
Bảng 5.8 Độ nhạy của kết luận theo ngưỡng override xác suất tĩnh khi phát hiện tiêm mã (205 mã độc + 100 lành tính). 52

Bảng 5.9 Tỉ lệ phát hiện hành vi tiêm mã theo nhóm họ; ground-truth lấy xấp xỉ ở mức taxonomy họ. 53
Bảng 5.10 Số lần các quy tắc MITRE cho kết quả dương tính giả trên 100 ứng dụng hợp pháp (mặt precision của ánh xạ hành vi). 53

Bảng A.1 Thống kê mẫu lành tính (benign) theo nguồn. 61
Bảng A.2 Thống kê mẫu mã độc theo nguồn. 61

Bảng B.1 Các Sysmon Event ID được theo dõi và nhóm hành vi tương ứng. 62
Bảng B.2 Các luật ánh xạ MITRE ATT&CK được triển khai trong bộ phân tích hành vi. 63
Bảng B.3 Toàn bộ endpoint của server web demo. 64
Bảng B.4 Chín loại sự kiện SSE trong luồng `/api/jobs/<id>/stream`. 64
Bảng B.5 Tám tiến trình trong `_DOTNET_NOISE_PROCS` bị loại khỏi đầu ra bộ chuẩn hóa SilkETW. 65
Bảng B.6 Bảy từ khóa trong `_SUSPICIOUS_NS` dùng đánh dấu sự kiện SilkETW nghi ngờ. 65
Bảng B.7 27 tiến trình hệ thống bị lọc khỏi cây tiến trình (hằng `_TREE_NOISE`). 65
Bảng B.8 Toàn bộ endpoint của Agent HTTP trong máy ảo. 66
Bảng B.9 Ánh xạ 24 loại sự kiện chuẩn hóa vào 11 nhóm hành vi. 67
Bảng B.10 Cấu trúc báo cáo JSON đầu ra của pipeline. 67

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Ý nghĩa
ADS	Alternate Data Stream	Luồng dữ liệu thay thế của hệ thống tệp NTFS
API	Application Programming Interface	Giao diện lập trình ứng dụng
AUC	Area Under the Curve	Diện tích dưới đường cong
AUC-PR	Area Under the Precision–Recall Curve	Diện tích dưới đường cong Precision–Recall
C2	Command and Control	Máy chủ điều khiển mã độc
COFF	Common Object File Format	Định dạng tệp đối tượng chung
CSS	Cascading Style Sheets	Ngôn ngữ định kiểu trình bày trang web
CV	Cross-Validation	Kiểm chứng chéo
DGA	Domain Generation Algorithm	Thuật toán sinh tên miền tự động
DLL	Dynamic-Link Library	Thư viện liên kết động
DNS	Domain Name System	Hệ thống phân giải tên miền
ECE	Expected Calibration Error	Sai số hiệu chuẩn kỳ vọng (độ lệch trung bình giữa xác suất dự đoán và tần suất thực tế)
EFB	Exclusive Feature Bundling	Gộp đặc trưng loại trừ (kỹ thuật tăng tốc của LightGBM)
EID	Event ID	Mã định danh sự kiện
EMBER	Endgame Malware BEnchmark for Research	Bộ dữ liệu chuẩn về đặc trưng tĩnh của tệp PE
ETW	Event Tracing for Windows	Cơ chế ghi vết sự kiện tích hợp trong Windows
FN	False Negative	Bỏ sót: mã độc bị phân loại nhầm thành lành tính
FNR	False Negative Rate	Tỉ lệ bỏ sót mã độc
FP	False Positive	Báo động giả: lành tính bị phân loại nhầm thành mã độc
FPR	False Positive Rate	Tỉ lệ báo động giả

Từ viết tắt	Tên đầy đủ	Ý nghĩa
GBDT	Gradient Boosting Decision Tree	Cây quyết định tăng cường độ dốc
GOSS	Gradient-based One-Side Sampling	Lấy mẫu một phía theo gradient (kỹ thuật tăng tốc của LightGBM)
HTTP	HyperText Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Giao thức HTTP có mã hóa
IOC	Indicator of Compromise	Chỉ số xâm phạm
IP	Internet Protocol	Địa chỉ giao thức Internet
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi dạng văn bản
LightGBM	Light Gradient Boosting Machine	Mô hình tăng cường độ dốc nhẹ
LSASS	Local Security Authority Subsystem Service	Dịch vụ xác thực bảo mật cục bộ của Windows
MD5	Message-Digest Algorithm 5	Hàm băm 128-bit
ML	Machine Learning	Học máy
NTP	Network Time Protocol	Giao thức đồng bộ thời gian qua mạng
PE	Portable Executable	Định dạng tệp thực thi trên Windows
PID	Process Identifier	Mã định danh tiến trình
RAT	Remote Access Trojan	Trojan điều khiển từ xa
REST	Representational State Transfer	Kiến trúc giao tiếp dịch vụ web
RF	Random Forest	Rừng ngẫu nhiên
ROC	Receiver Operating Characteristic	Đường đặc trưng hoạt động của bộ phân loại
RPC	Remote Procedure Call	Lời gọi thủ tục từ xa
SHAP	SHapley Additive exPlanations	Phương pháp giải thích đóng góp của đặc trưng
SHA-256	Secure Hash Algorithm 256-bit	Hàm băm bảo mật 256-bit
SMB	Server Message Block	Giao thức chia sẻ tệp qua mạng

Từ viết tắt	Tên đầy đủ	Ý nghĩa
SSE	Server-Sent Events	Cơ chế máy chủ đẩy sự kiện theo thời gian thực
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận
TLS	Thread Local Storage	Vùng lưu trữ cục bộ theo luồng trong cấu trúc tệp PE
UDP	User Datagram Protocol	Giao thức truyền dữ liệu không kết nối
URL	Uniform Resource Locator	Địa chỉ định vị tài nguyên trên web
VM	Virtual Machine	Máy ảo
VSS	Volume Shadow Copy Service	Dịch vụ tạo bản sao ảnh đĩa của Windows
WMI	Windows Management Instrumentation	Hạ tầng quản trị của Windows
XGBoost	eXtreme Gradient Boosting	Mô hình tăng cường độ dốc cực hạn
XML	eXtensible Markup Language	Ngôn ngữ đánh dấu mở rộng

DANH MỤC THUẬT NGỮ

Thuật ngữ	Ý nghĩa
Sandbox	Môi trường thực thi cô lập, cho phép quan sát hành vi mã độc mà không ảnh hưởng đến hệ thống thật.
Snapshot	Ảnh chụp trạng thái máy ảo, dùng để khôi phục nhanh về trạng thái sạch trước mỗi phiên phân tích.
Đóng gói (Packing)	Kỹ thuật nén hoặc mã hóa mã thực thi nhằm che giấu nội dung gốc của tệp.
Làm rối mã (Obfuscation)	Kỹ thuật biến đổi mã nhằm gây khó khăn cho quá trình phân tích.
Zero-day	Mẫu mã độc hoặc lỗ hổng chưa từng được ghi nhận và chưa có chữ ký hay bản vá.
Chuỗi hành vi	Tập sự kiện được sắp xếp theo thời gian, phản ánh diễn biến hoạt động của mẫu.
Cây tiến trình	Cấu trúc cha–con giữa các tiến trình sinh ra trong phiên phân tích.
Tiêm mã (Process injection)	Kỹ thuật chèn và thực thi mã trong không gian bộ nhớ của tiến trình khác.
Duy trì hiện diện (Persistence)	Cơ chế giúp mã độc tồn tại sau khi hệ thống khởi động lại.
Beacon	Tín hiệu định kỳ mà mã độc gửi tới máy chủ điều khiển để báo trạng thái và chờ lệnh.
.NET Runtime	Môi trường thực thi cho các chương trình viết bằng C# và VB.NET.
Assembly (.NET)	Đơn vị mã .NET đã biên dịch, được nạp vào bộ nhớ tại thời điểm chạy.
Provider (ETW)	Thành phần phát sự kiện trong cơ chế ETW của Windows.
Feature hashing	Kỹ thuật băm danh sách phần tử có độ dài thay đổi về vector kích thước cố định.
Gain (độ lợi)	Mức cải thiện khả năng tách malware khỏi benign khi một nút trong cây quyết định chia dữ liệu theo một đặc trưng.

Thuật ngữ	Ý nghĩa
Importance Gain	Chỉ số đo mức đóng góp của một đặc trưng dựa trên tổng Gain của đặc trưng đó trên toàn bộ các nút và các cây trong mô hình.
Gradient (độ dốc)	Đạo hàm của hàm mất mát theo dự đoán hiện tại, cho biết mô hình đang sai bao nhiêu và theo hướng nào; trong gradient boosting, mỗi cây mới học để giảm sai số theo gradient của các cây trước.
Mở rộng theo tầng (level-wise)	Chiến lược phát triển cây của XGBoost: mở rộng đồng loạt mọi nút cùng một độ sâu trước khi xuống sâu hơn, cho cây cân đối.
Mở rộng theo lá (leaf-wise)	Chiến lược phát triển cây của LightGBM: ưu tiên mở rộng nút lá có Gain cao nhất bất kể độ sâu, đạt độ chính xác cao hơn với cùng số vòng nhưng cần giới hạn độ sâu để tránh quá khớp.
Dữ liệu lệch phân phối	Dữ liệu khác nguồn thu thập hoặc khác phân phối so với tập huấn luyện, dùng để đánh giá khả năng tổng quát hóa.
Holdout	Tập kiểm thử được tách riêng từ dữ liệu huấn luyện để đánh giá khách quan.
INetSim	Công cụ giả lập các dịch vụ mạng nhằm duy trì luồng thực thi của mẫu trong sandbox.
Quá khớp (Overfitting)	Hiện tượng mô hình học quá sát dữ liệu huấn luyện, làm giảm khả năng tổng quát hóa sang dữ liệu mới.
LOLBin (Living-off-the-Land Binary)	Tệp thực thi hợp pháp có sẵn của hệ điều hành (như `cmd`, `powershell`, `rundll32`) bị mã độc lạm dụng để thực thi hành vi.
Kernel-mode / User-mode	Hai mức đặc quyền thực thi của Windows: kernel-mode là nhân hệ điều hành (đặc quyền cao), user-mode là nơi chạy ứng dụng thông thường (đặc quyền thấp).
Hiệu chuẩn (Calibration)	Quá trình chỉnh xác suất dự đoán của mô hình cho khớp tần suất thực tế, giúp con số độ tin cậy hiển thị đáng tin mà không làm thay đổi nhãn phân loại.
Isotonic Regression	Phép hồi quy đơn điệu (không giảm) dùng để hiệu chuẩn xác suất; vì giữ nguyên thứ tự xếp hạng nên không làm thay đổi nhãn.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Đến năm 2026, số mẫu mã độc trên Windows do AV-TEST ghi nhận đã vượt 1 tỷ và tiếp tục tăng đều qua từng năm [1]. Theo báo cáo IBM [2], chi phí trung bình mỗi vụ vi phạm dữ liệu đạt 4,44 triệu USD năm 2025, bao gồm tổn thất do gián đoạn hoạt động, mất khách hàng và chi phí tuân thủ pháp lý. Ở quy mô toàn cầu, thiệt hại do tội phạm mạng được dự báo đạt 10,5 nghìn tỷ USD mỗi năm [3].

Phần mềm độc hại trên Windows tồn tại dưới nhiều hình thức với cơ chế tấn công khác nhau. Ransomware mã hóa dữ liệu rồi đòi tiền chuộc và hiện diện trong 48% số vụ vi phạm dữ liệu theo báo cáo Verizon DBIR 2026 [4]. Trojan ngụy trang thành phần mềm hợp lệ để cài cửa hậu hoặc đánh cắp thông tin; biến thể phổ biến là RAT, duy trì quyền điều khiển từ xa trung bình 11 ngày trước khi bị phát hiện [5]. Worm tự nhân bản và lan truyền qua mạng mà không cần thao tác của người dùng nên phát tán rất nhanh: mã độc dạng worm như WannaCry từng lây nhiễm hơn 200.000 máy tính tại ít nhất 150 quốc gia chỉ trong vài ngày, gây thiệt hại ước tính tới 4 tỷ USD [6]. Sự đa dạng về cơ chế tấn công đòi hỏi phương pháp phát hiện thích ứng được với nhiều dạng mối đe dọa thay vì chỉ dựa vào một cơ chế duy nhất.

Phương pháp phát hiện phổ biến nhất hiện nay là nhận diện dựa trên chữ ký: hệ thống đối chiếu tệp cần kiểm tra với cơ sở dữ liệu mẫu độc hại đã biết. Cách làm này hiệu quả với các mối đe dọa đã biết nhưng bộc lộ hạn chế trước biến thể mới, bởi kẻ tấn công chỉ cần đóng gói lại hoặc làm rối mã thực thi là đủ khiến chữ ký hiện có không còn nhận diện được [7]. Riêng năm 2024, SonicWall ghi nhận hơn 210.000 biến thể mã độc mới [8]; con số này phản ánh khoảng trễ thường trực giữa thời điểm mẫu xuất hiện và thời điểm chữ ký được cập nhật.

Hạn chế này đặt ra nhu cầu về phương pháp phát hiện không phụ thuộc hoàn toàn vào chữ ký, có khả năng nhận diện cả biến thể zero-day. Kết hợp phân tích tĩnh bằng học máy và quan sát hành vi động trong môi trường sandbox là hướng tiếp cận cho phép hệ thống nhận diện cả mẫu đã biết lẫn biến thể mới.

1.2 Mục tiêu và phạm vi đề tài

Từ hạn chế của phương pháp phát hiện dựa trên chữ ký đã phân tích tại Mục 1.1, đồ án xây dựng hệ thống hỗ trợ phát hiện và phân tích phần mềm độc hại trên nền tảng Windows, thực hiện phân loại tệp PE thành lành tính hoặc độc hại đồng thời thu thập bằng chứng hành vi phục vụ điều tra. Cụ thể, bên cạnh kết quả phân loại,

hệ thống thu thập thông tin phục vụ phân tích hành vi gồm IOC, cây tiến trình và các kỹ thuật tấn công theo khung MITRE ATT&CK [9]. Ba dạng đầu ra này bổ sung cho nhau: IOC phục vụ truy vết và đối chiếu nhanh, cây tiến trình giúp hiểu luồng thực thi, ánh xạ ATT&CK chuẩn hóa kết quả cho đội ngũ phân tích.

Đồ án giới hạn phạm vi đối với các tệp thực thi định dạng PE 32/64-bit (`.exe`, `.dll`) trên hệ điều hành Windows. Các định dạng khác như tài liệu Office chứa macro, script PowerShell hay JavaScript, cũng như các nền tảng ngoài Windows, không thuộc phạm vi đề tài. Đồ án không đặt mục tiêu xử lý các kỹ thuật né tránh sandbox nâng cao hay cơ chế chống phân tích chuyên biệt.

1.3 Định hướng giải pháp

Để đạt được các mục tiêu đề ra, đồ án đề xuất kiến trúc hai giai đoạn kết hợp phân tích tĩnh và phân tích động. Giai đoạn tĩnh thực hiện phân loại tệp PE thông qua mô hình học máy huấn luyện trên các đặc trưng cấu trúc. Giai đoạn động thực thi mẫu trong môi trường sandbox cô lập nhằm thu thập thông tin hành vi thực tế. Phân tích tĩnh đơn lẻ dễ bị vô hiệu hóa bởi kỹ thuật đóng gói và làm rối mã; phân tích động đơn lẻ tiêu tốn nhiều tài nguyên và không cung cấp phân loại tức thời. Kết hợp hai giai đoạn cho phép hệ thống phân loại nhanh dựa trên cấu trúc tệp, đồng thời thu thập bằng chứng hành vi để làm rõ ngữ cảnh tấn công. So với các hệ thống phân tích động hiện có, đồ án tích hợp thêm SilkETW để giám sát .NET Runtime — lớp hành vi mà Sysmon không ghi nhận trực tiếp — và hợp nhất kết quả hai giai đoạn thành một báo cáo thống nhất gồm kết luận, IOC, cây tiến trình và ánh xạ MITRE ATT&CK.

1.4 Bố cục đồ án

Phần còn lại của đồ án được tổ chức như sau. Chương 2 trình bày cơ sở lý thuyết, bao gồm đặc điểm các nhóm mã độc phổ biến, cấu trúc tệp PE và đặc trưng tĩnh, cơ chế thu thập sự kiện qua Sysmon và ETW, và khung MITRE ATT&CK. Chương 3 trình bày kiến trúc tổng thể, quy trình xây dựng mô hình học máy, thiết kế môi trường sandbox và cơ chế xây dựng chuỗi hành vi. Chương 4 trình bày triển khai từng thành phần và giao diện web demo. Chương 5 đánh giá hiệu năng mô hình phân loại và kết quả phân tích hành vi động. Phần Kết luận tóm tắt kết quả đạt được và đề xuất hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các cơ sở lý thuyết mà đồ án dựa vào: đặc điểm các nhóm mã độc trên Windows, định dạng tệp PE và đặc trưng tĩnh, các thuật toán học máy dùng để phân loại, cơ chế giám sát hành vi động bằng Sysmon và ETW, cùng khung MITRE ATT&CK; phần cuối điểm qua các công trình liên quan.

2.1 Phần mềm độc hại

Phần mềm độc hại là các chương trình được thiết kế nhằm xâm nhập, gây hại hoặc khai thác trái phép hệ thống máy tính mà không có sự đồng ý của người dùng [10]. Tùy theo cơ chế hoạt động và mục tiêu tấn công, phần mềm độc hại được phân loại thành nhiều nhóm [7]. Bảng 2.1 tổng hợp đặc điểm, cách lan truyền và hành vi quan sát được trong sandbox của bảy nhóm được đánh giá trong đồ án.

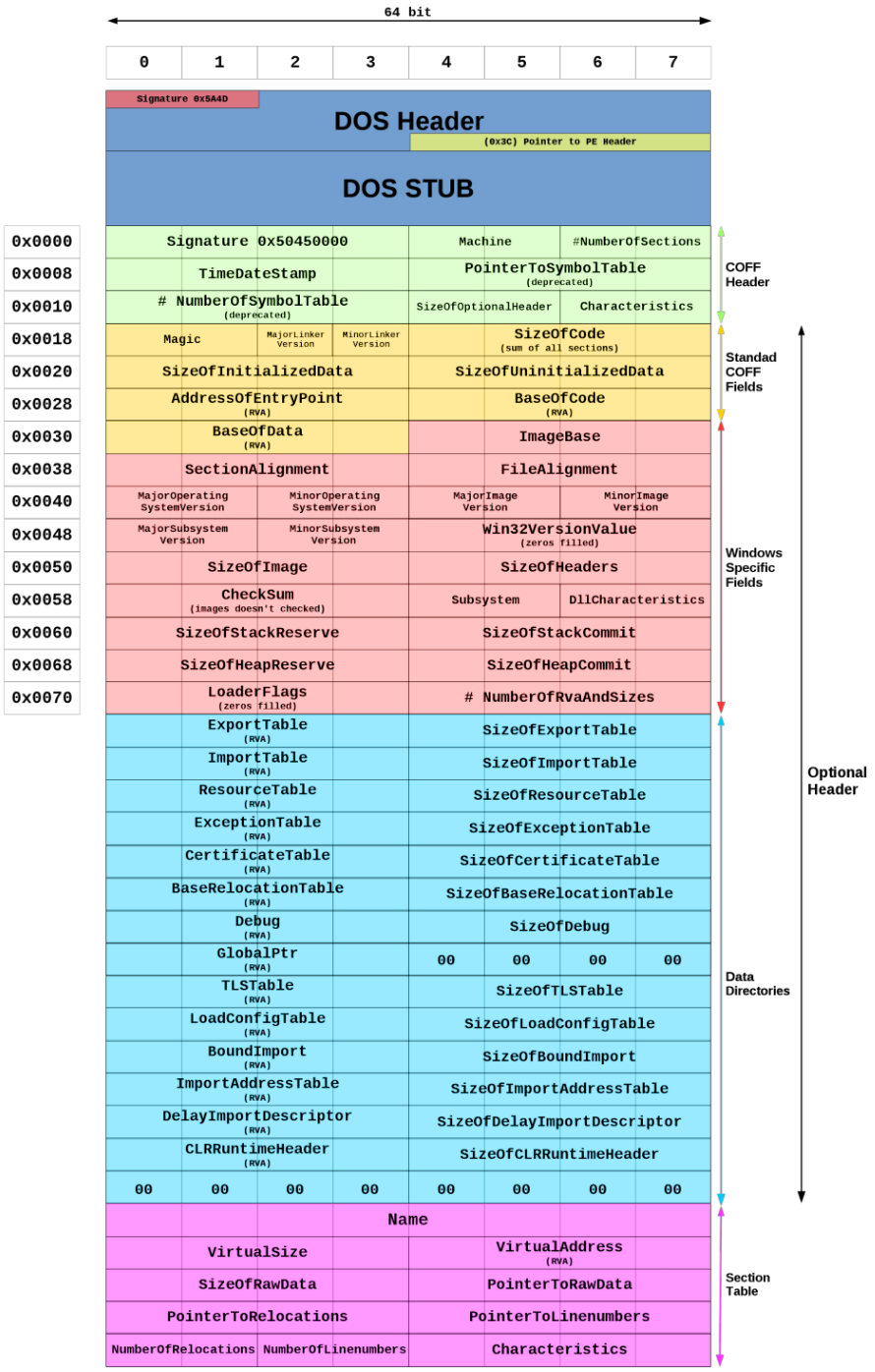
Bảng 2.1: So sánh các nhóm phần mềm độc hại và hành vi quan sát được trong sandbox, tổng hợp từ các nguồn [7], [10], [11], [12], [13], [14], [15], [16].

Nhóm	Đặc điểm chính	Lan truyền	Hành vi đặc trưng trong sandbox
Dropper	Tải và cài payload thứ cấp	Qua người dùng	Giải nén hoặc tải tệp thực thi mới, tạo tiến trình con, thiết lập tự khởi động
Ransomware	Mã hóa dữ liệu, đòi tiền chuộc	Qua người dùng	Đọc/ghi/đổi tên hàng loạt tệp, xóa Shadow Copy, dừng dịch vụ bảo mật, kết nối C2
Trojan	Ngụy trang phần mềm hợp lệ	Qua người dùng	Tạo tiến trình con bất thường, ghi tệp thực thi mới, thiết lập tự khởi động, kết nối C2
RAT	Điều khiển từ xa hai chiều	Qua người dùng	Kết nối TCP từ tiến trình không phải trình duyệt, beacon định kỳ, ghi bàn phím, chụp màn hình, nạp assembly .NET trong bộ nhớ
Worm	Tự nhân bản qua mạng	Tự động	Quét TCP/UDP tốc độ cao, tạo bản sao tiến trình, thêm Registry/Service sau thực thi
Cryptominer	Chiếm tài nguyên đào tiền mã hóa	Qua người dùng	Kết nối mining pool, sinh tiến trình đào tiêu thụ CPU cao, thiết lập tự khởi động
Spyware	Thu thập thông tin bí mật	Qua người dùng	Ghi bàn phím, đọc tệp và dữ liệu trình duyệt, gửi dữ liệu thu thập ra C2

2.2 Định dạng tệp PE và đặc trưng tĩnh

PE là định dạng tệp thực thi chủ đạo trên hệ điều hành Microsoft Windows, bao gồm các tệp thực thi (`.exe`) và thư viện liên kết động (`.dll`). Vì hầu hết phần mềm độc hại trên Windows đều tồn tại dưới dạng tệp PE, phân tích tĩnh cấu trúc PE mà không cần thực thi tệp trở thành một lớp bảo vệ quan trọng trong hệ thống phát hiện mã độc [17].

2.2.1 Cấu trúc tệp PE



Hình 2.1: Cấu trúc tệp PE 32-bit [17].

Tệp PE được tổ chức thành hai phần chính: phần header và các section, như minh họa trong Hình 2.1 [17]. Phần header bao gồm COFF Header (kiến trúc máy đích, số section), Optional Header (entry point, kích thước vùng mã, tham chiếu bảng import/export) và Section Table (vị trí, kích thước và thuộc tính từng section). Phần sections chứa mã lệnh và dữ liệu; công cụ đóng gói có thể thêm section mới để lưu dữ liệu nén hoặc mã được bảo vệ, làm thay đổi cấu trúc PE và gây khó khăn cho phân tích tĩnh [17].

2.2.2 Đặc trưng tĩnh dùng trong phát hiện mã độc

Anderson và Roth đề xuất bộ đặc trưng EMBER gồm tám nhóm đặc trưng tĩnh [17]; phiên bản cập nhật bổ sung thêm nhóm Data Directories, nâng tổng số nhóm lên chín. Cấu hình chín nhóm này cũng được sử dụng trong nghiên cứu của Choudhary et al. [18]. Bảng 2.2 mô tả các nhóm đặc trưng cùng số chiều tương ứng.

Bảng 2.2: Chín nhóm đặc trưng tĩnh và số chiều tương ứng [17], [18].

Nhóm đặc trưng	Số chiều	Mô tả
Thông tin tổng quát	10	Kích thước tệp, số hàm import/export, sự hiện diện của debug, tài nguyên, TLS và chữ ký số
Header	62	Các trường từ COFF Header và Optional Header: kiến trúc máy đích, subsystem, cờ DLL
Import	1.280	Tên thư viện (256 vị trí hash) và tên hàm import (1.024 vị trí hash)
Export	128	Tên các hàm được export
Section	255	Tên, kích thước, entropy và thuộc tính của từng section
Chuỗi ký tự	104	Thống kê chuỗi có thể in được: đường dẫn, URL, khóa Registry, mẫu ký tự
Histogram byte	256	Tần suất của 256 giá trị byte trong toàn bộ tệp
Histogram entropy-byte	256	Phân bố kết hợp giữa giá trị byte và entropy cục bộ (lưới 16×16)
Data Directories	30	Kích thước và địa chỉ ảo của 15 bảng dữ liệu trong PE header
Tổng	**2.381**	

Bốn nhóm có danh sách phần tử dài thay đổi giữa các tệp (tên thư viện, hàm import/export, tên section) được đưa về vector cố định bằng feature hashing: mỗi phần tử được một hàm băm ánh xạ vào một vị trí cố định trong mảng, nhờ đó mọi tệp đều cho ra vector cùng số chiều bất kể số phần tử nhiều hay ít.

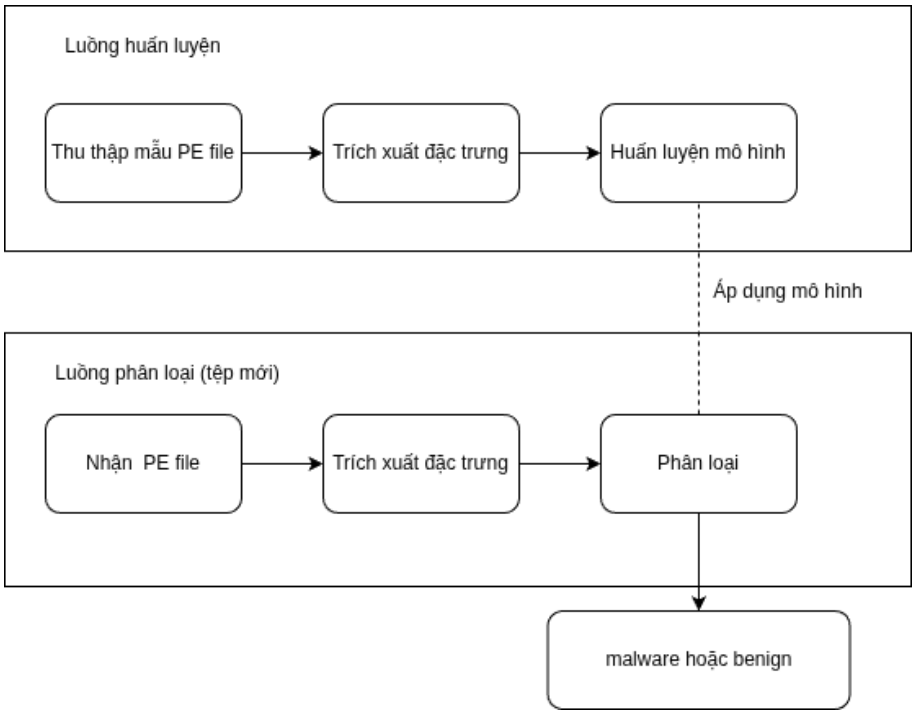
Các đặc trưng tĩnh có ưu điểm là trích xuất nhanh và an toàn mà không cần môi trường thực thi. Tuy nhiên, các đặc trưng tĩnh dễ bị vô hiệu hóa bởi kỹ thuật đóng gói: mã thực thi được nén hoặc mã hóa, chỉ giải nén tại thời điểm chạy, khiến cấu trúc PE quan sát được khác biệt hoàn toàn so với tệp gốc [7]. Hạn chế này là lý do nhiều hệ thống kết hợp phân tích tĩnh với phân tích động để quan sát hành vi thực tế của mẫu.

2.3 Phân tích tĩnh dựa trên học máy

2.3.1 Hướng tiếp cận học máy trong phát hiện mã độc

Phương pháp nhận diện dựa trên chữ ký không thể phát hiện các mẫu mã độc chưa từng được ghi nhận. Học máy khắc phục hạn chế này bằng cách tự động học các mối quan hệ phức tạp giữa đặc trưng của tệp từ dữ liệu huấn luyện, tổng quát hóa sang các mẫu mới có phân phối đặc trưng tương tự [17].

Quy trình phát hiện mã độc bằng học máy gồm ba bước chính [7] như minh họa trong Hình 2.2: trích xuất đặc trưng từ tệp PE thành vector số, huấn luyện mô hình trên tập dữ liệu đã gán nhãn malware hoặc benign, và sử dụng mô hình đã huấn luyện để phân loại các tệp mới. Chất lượng của bước trích xuất đặc trưng ảnh hưởng trực tiếp đến hiệu năng phân loại, do đó việc lựa chọn bộ đặc trưng phù hợp đóng vai trò then chốt trong toàn bộ quy trình [7]. Bộ đặc trưng EMBER 2.381 chiều được trích xuất từ cấu trúc tệp PE; thiết kế cụ thể được trình bày tại Chương 3.



Hình 2.2: Quy trình phát hiện mã độc bằng học máy.

2.3.2 Các thuật toán phân loại

Đồ án đánh giá ba thuật toán phân loại nhị phân trên dữ liệu dạng bảng: Random Forest, XGBoost và LightGBM.

Random Forest xây dựng nhiều cây quyết định độc lập trên các tập con dữ liệu và đặc trưng được lấy mẫu ngẫu nhiên, sau đó tổng hợp kết quả bằng biểu quyết đa số [19]. Vì các cây được xây dựng độc lập với nhau, Random Forest cho phép huấn luyện song song và chống quá khớp tốt nhờ tính ngẫu nhiên, nhưng không có cơ chế học từ sai số của cây trước.

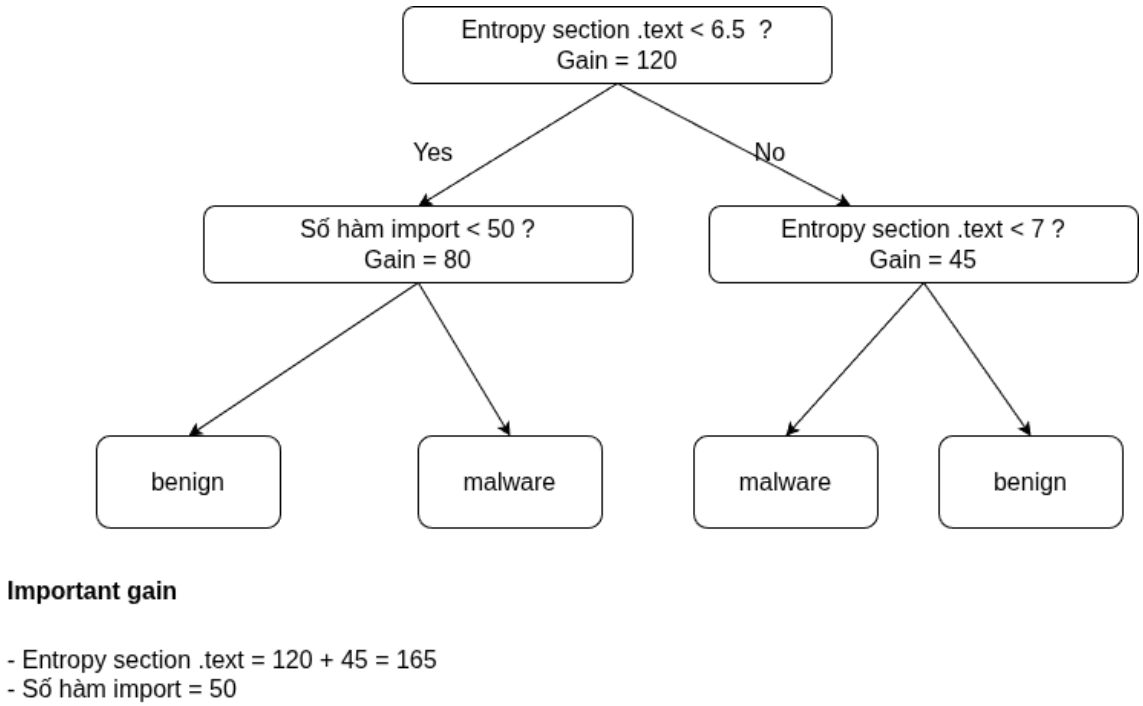
XGBoost thuộc họ Gradient Boosting Decision Tree (GBDT), xây dựng cây tuần tự trong đó mỗi cây học từ sai số của cây trước [20]. So với GBDT truyền thống, XGBoost bổ sung số hạng phạt vào hàm mục tiêu để giới hạn độ phức tạp của mô hình và mở rộng cây theo từng tầng, giúp kiểm soát quá khớp tốt hơn.

LightGBM cũng thuộc họ GBDT nhưng tối ưu hóa tốc độ trên tập dữ liệu lớn thông qua hai kỹ thuật [21]: GOSS ưu tiên giữ lại các mẫu mà mô hình đang học kém (gradient lớn) và lấy ngẫu nhiên một phần mẫu gradient nhỏ, giảm số mẫu cần xử lý; EFB gộp các đặc trưng thưa thớt, giảm số chiều. Khác với XGBoost, LightGBM mở rộng nút lá có Gain cao nhất, đạt độ chính xác cao hơn với cùng số vòng học. Kết quả so sánh thực nghiệm được trình bày tại Chương 5.

2.3.3 Lựa chọn đặc trưng dựa trên Importance Gain

Không phải tất cả đặc trưng đều đóng góp như nhau vào khả năng dự đoán của mô hình; giữ quá nhiều đặc trưng không chỉ tăng thời gian huấn luyện mà còn có thể khiến mô hình học nhiễu, làm giảm khả năng tổng quát hóa [7]. Lựa chọn đặc trưng giải quyết vấn đề này bằng cách chỉ giữ lại những đặc trưng đóng góp cao nhất và loại bỏ phần còn lại, nhờ đó giảm số chiều mà vẫn duy trì hiệu năng [7].

Với các mô hình dựa trên cây quyết định, cách đánh giá trực tiếp nhất là dựa vào chỉ số Importance Gain. Tại mỗi nút trong cây, thuật toán chọn đặc trưng giúp tách malware ra khỏi benign rõ nhất; mức độ tách này được gọi là Gain. Importance Gain của một đặc trưng là tổng Gain mà đặc trưng đó tạo ra trên tất cả các lần được sử dụng để chia, cộng dồn qua toàn bộ các cây trong mô hình, như minh họa trong Hình 2.3 [21]. Kết quả là mỗi đặc trưng nhận một điểm quan trọng duy nhất. Trên thực tế các điểm này rất chênh lệch: một số ít đặc trưng chiếm phần lớn tổng Gain, còn rất nhiều đặc trưng gần như không đóng góp. Đây chính là lý do có thể loại bỏ phần lớn đặc trưng mà mô hình vẫn giữ được hiệu năng.



Hình 2.3: Minh họa cách tính Importance Gain.

Phương pháp này có ưu điểm là được tính sẵn trong quá trình huấn luyện mà không cần bước đánh giá riêng. Sau khi huấn luyện, các đặc trưng được xếp hạng theo Gain giảm dần và chỉ giữ lại top–K đặc trưng có giá trị cao nhất để huấn luyện lại mô hình với số chiều nhỏ hơn.

2.4 Phân tích động và giám sát hành vi

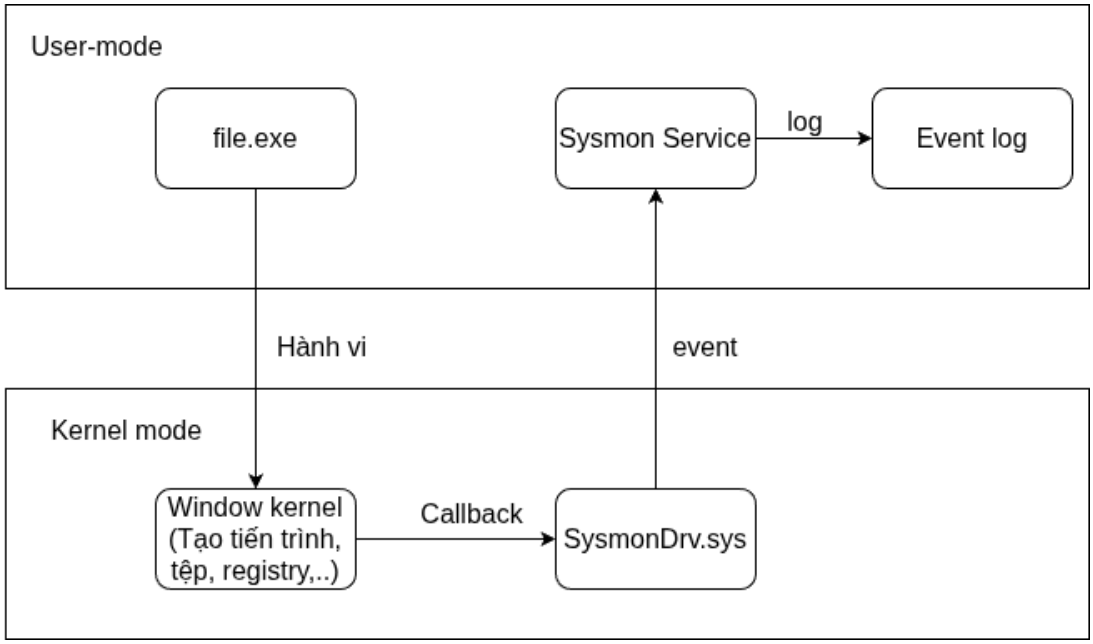
2.4.1 Khái niệm sandbox và phân tích động

Phân tích động thực thi tệp nghi ngờ trong một sandbox nhằm quan sát trực tiếp hành vi của chương trình lúc chạy [7]. Các hành vi thường được theo dõi gồm tạo, sửa đổi hoặc xóa tệp, thay đổi khóa Registry, tạo tiến trình con, nạp thư viện động và thiết lập kết nối mạng; trước mỗi lần phân tích, sandbox được khôi phục về trạng thái ban đầu để bảo đảm tính độc lập giữa các lần chạy.

So với phân tích tĩnh, phân tích động quan sát được hành vi thực tế ngay cả khi mẫu bị đóng gói hoặc mã hóa, nhưng tiêu tốn nhiều tài nguyên và thời gian hơn và có thể bị né tránh bởi các kỹ thuật phát hiện môi trường ảo hóa [7].

2.4.2 Sysmon

Sysmon là công cụ giám sát hệ thống thuộc bộ Sysinternals của Microsoft, ghi lại các hoạt động chi tiết trên Windows [22]. Về kiến trúc, Sysmon gồm một driver kernel-mode đăng ký callback để ghi nhận sự kiện hệ thống ở mức thấp, và một dịch vụ user-mode chuẩn hóa dữ liệu rồi ghi vào Windows Event Log [22]. Hình 2.4 minh họa kiến trúc hai tầng này.



Hình 2.4: Minh họa kiến trúc Sysmon.

Phần lớn mã độc chạy ở user-mode (tầng đặc quyền thấp dành cho ứng dụng thông thường), nhưng mọi thao tác tạo tiến trình, ghi tệp, sửa registry hay kết nối mạng của chúng đều phải đi qua kernel để được thực thi. Nhờ đặt cảm biến tại driver kernel-mode — tầng đặc quyền cao hơn nơi mã độc user-mode hoạt động — Sysmon ghi nhận được đầy đủ các hoạt động này và khó bị mã độc vô hiệu hóa, khác với các công cụ giám sát chạy ở user-mode vốn cùng tầng quyền với mã độc nên có thể bị qua mặt.

Một điểm mạnh của Sysmon là lượng thông tin đi kèm mỗi sự kiện. Ngoài loại hành vi, một sự kiện Sysmon còn ghi nhiều trường ngữ cảnh như đường dẫn và giá trị băm của tệp thực thi, dòng lệnh khởi chạy, cùng tiến trình cha sinh ra nó. Nhờ các trường này, mỗi sự kiện mô tả hành vi đầy đủ hơn so với nhật ký mặc định của Windows: quan hệ cha–con giữa các tiến trình là cơ sở để tái dựng cây tiến trình, còn đường dẫn, dòng lệnh và giá trị băm là nguồn để rút ra các chỉ số xâm phạm.

Mỗi loại sự kiện được Sysmon ghi lại với một mã định danh riêng gọi là Event ID. Sysmon cho phép cấu hình linh hoạt thông qua tệp XML, trong đó người dùng xác định loại sự kiện cần thu thập và điều kiện lọc tương ứng để tập trung vào các hành vi đáng ngờ và giảm lượng dữ liệu nhiễu từ các tiến trình hệ thống thông thường [22]. Bảng 2.3 liệt kê các EID được sử dụng trong đồ án này.

Bảng 2.3: Các Event ID của Sysmon được sử dụng trong đồ án.

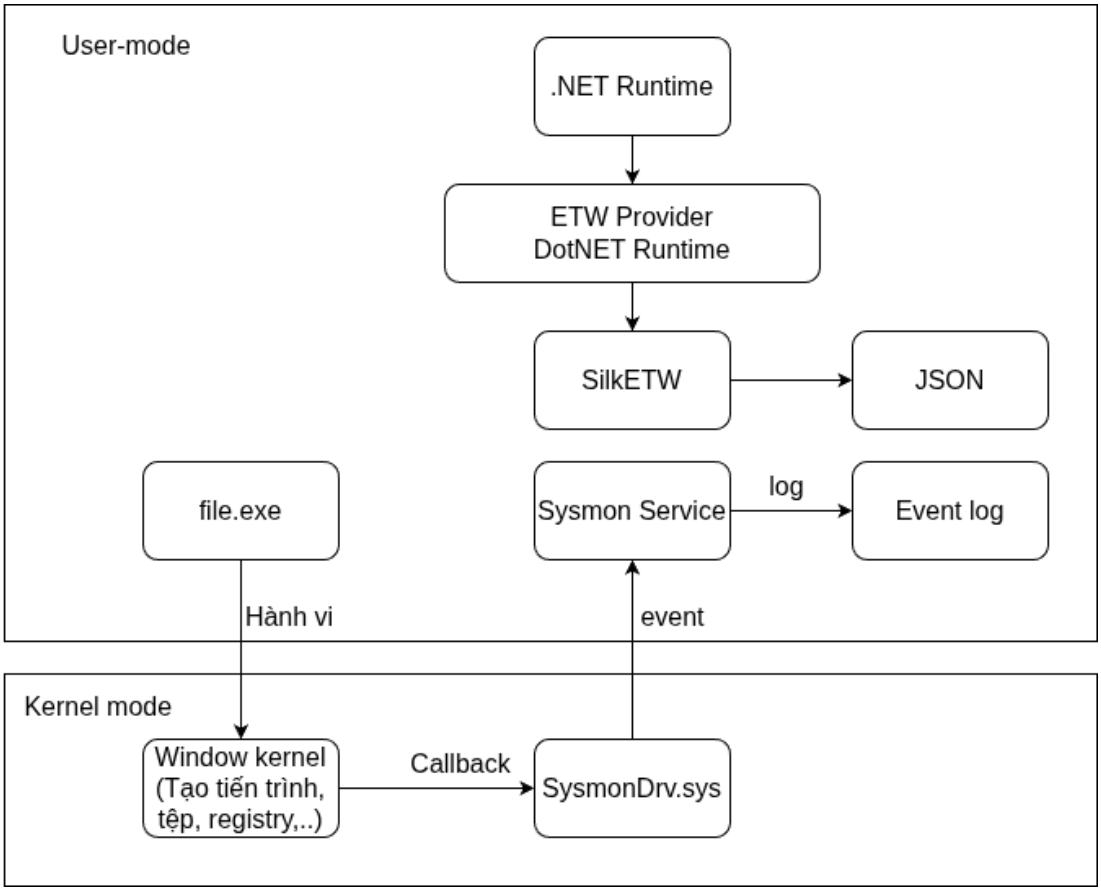
EID	Sự kiện	Ý nghĩa
1	Process Create	Ghi nhận tiến trình mới được tạo
2	File Creation Time	Thay đổi timestamp của tệp (ngụy trang)
3	Network Connection	Kết nối mạng được thiết lập từ tiến trình
5	Process Terminate	Tiến trình kết thúc
6	Driver Load	Trình điều khiển (driver) được nạp
7	Image Load	DLL được nạp vào tiến trình
8	CreateRemoteThread	Tạo luồng trong tiến trình khác (dấu hiệu tiêm mã)
9	Raw Disk Access	Truy cập trực tiếp ổ đĩa
10	Process Access	Truy cập bộ nhớ tiến trình khác
11	File Create	Tệp mới được tạo
12	Registry Event	Tạo hoặc xóa khóa registry
13	Registry Value Set	Giá trị registry được thay đổi
14	Registry Key Rename	Đổi tên khóa/giá trị registry
15	FileCreateStreamHash	Ghi dữ liệu vào Alternate Data Stream (ADS)
17	Pipe Created	Named pipe được tạo
18	Pipe Connected	Kết nối vào named pipe
19	WMI Event Filter	Đăng ký WMI event filter
20	WMI Event Consumer	Tạo WMI consumer
21	WMI Event Binding	Liên kết WMI filter và consumer
22	DNS Query	Truy vấn DNS
23	File Delete	Tệp bị xóa
25	Process Tampering	Phát hiện chỉnh sửa bất thường tiến trình
26	File Delete Logged	Ghi nhận sự kiện xóa tệp

2.4.3 ETW và SilkETW

Sysmon chủ yếu thu thập sự kiện ở mức hệ điều hành; cơ chế này không phản ánh các hoạt động bên trong .NET Runtime. Ngày càng nhiều mã độc Windows được viết bằng .NET và thực thi theo kiểu fileless, nạp payload thẳng vào bộ nhớ để né phân tích tĩnh [23]. Hai kỹ thuật tiêu biểu là nạp assembly từ bộ nhớ bằng `Assembly.Load` (không ghi tệp ra ổ đĩa) và sinh mã động bằng `Reflection.Emit` (tự sinh mã thực thi ngay lúc chạy); cả hai không tạo tệp trên đĩa nên Sysmon không quan sát được.

ETW là cơ chế ghi nhận sự kiện tích hợp sẵn trong Windows theo mô hình provider [24]. .NET Runtime được Microsoft trang bị sẵn provider `DotNETRuntime`: mỗi khi Runtime nạp assembly hoặc gọi phương thức, provider tự động phát sự kiện tương ứng — cơ chế này nằm ngoài tầm kiểm soát của mã độc. SilkETW là công cụ mã nguồn mở do FuzzySecurity phát triển, đăng ký lắng nghe ETW provider và xuất sự kiện ra tệp JSON [25]. Hình 2.5 so sánh hai nguồn sự kiện:

Sysmon nhận callback từ kernel khi chương trình gọi API hệ điều hành, còn ETW nhận sự kiện do .NET Runtime tự phát ra từ bên trong.



Hình 2.5: So sánh phạm vi giám sát của Sysmon và ETW.

Provider `DotNETRuntime` phát sự kiện cho tất cả chương trình .NET đang chạy, không chỉ mẫu đang phân tích; dữ liệu thu được cần lọc theo PID của mẫu trước khi xử lý, như trình bày tại Chương 4.

2.4.4 Chuỗi hành vi và chỉ số xâm phạm

Các sự kiện đơn lẻ thu được từ Sysmon và ETW thường chưa đủ để đánh giá mức độ nguy hiểm của một mẫu, do nhiều hành vi tương tự cũng có thể xuất hiện trong phần mềm hợp pháp. Các sự kiện do đó cần được tổng hợp thành biểu diễn có cấu trúc ở mức cao hơn nhằm mô tả toàn bộ hành vi của mẫu.

Chuỗi hành vi là tập hợp các sự kiện được sắp xếp theo thứ tự thời gian và có thể được nhóm theo loại hành vi. Cách biểu diễn này cho phép quan sát luồng thực thi tổng thể của mẫu thay vì các sự kiện rời rạc. Ví dụ, một mẫu có thể lần lượt thực hiện tạo tiến trình con, ghi tệp thực thi, thiết lập kết nối mạng và cấu hình cơ chế tự khởi động, qua đó phản ánh chuỗi hành vi ở mức chiến thuật.

IOC là các dấu hiệu cụ thể trích xuất từ hành vi của mẫu, dùng để nhận diện hoặc truy vết mã độc trên các hệ thống khác. IOC bao gồm địa chỉ IP, tên miền,

URL, đường dẫn tệp được tạo hoặc chỉnh sửa, giá trị băm, khóa registry bị thay đổi và tên named pipe. IOC hỗ trợ điều tra sự cố bằng cách cho phép đối chiếu nhanh giữa các hệ thống để phát hiện dấu hiệu lây nhiễm tương tự.

2.4.5 Khung MITRE ATT&CK

MITRE ATT&CK là cơ sở tri thức mở do MITRE phát triển, mô tả và phân loại các kỹ thuật tấn công mạng dựa trên quan sát thực tế từ các sự cố an ninh toàn cầu [9].

Khung ATT&CK được tổ chức theo cấu trúc phân cấp gồm ba mức. Cấp cao nhất là chiến thuật (Tactic), mô tả mục tiêu của kẻ tấn công trong từng giai đoạn, ví dụ như duy trì quyền truy cập (Persistence) hoặc thu thập dữ liệu (Collection). Cấp tiếp theo là kỹ thuật (Technique), mô tả cách thức cụ thể để đạt được mục tiêu đó — ví dụ: tạo khóa registry Run để tự khởi động (T1547.001) hoặc tiêm mã vào tiến trình khác (T1055). Cấp cuối cùng là kỹ thuật con (Sub-technique), cung cấp mức độ chi tiết hơn của từng kỹ thuật cụ thể.

Trong phân tích mã độc, việc ánh xạ các hành vi quan sát được sang các kỹ thuật trong MITRE ATT&CK giúp chuẩn hóa mô tả hành vi, đồng thời hỗ trợ việc suy luận ngữ cảnh tấn công ở mức cao hơn thay vì chỉ dựa trên các sự kiện rời rạc [9].

2.5 Các công trình liên quan

Cuckoo Sandbox và Any.Run là các hệ thống phân tích động được sử dụng rộng rãi, thu thập hành vi mẫu qua lời gọi API, lưu lượng mạng và thay đổi hệ thống tệp, tổng hợp thành báo cáo IOC cho nhà phân tích [7]. Hai hệ thống này chủ yếu dựa vào kỹ thuật can thiệp lời gọi API ở user-mode, vốn có thể bị mã độc phát hiện và vô hiệu hóa, đồng thời chưa quan sát sâu hoạt động thực thi trong .NET Runtime. Về hướng học máy trên đặc trưng tĩnh PE, Anderson và Roth đề xuất bộ đặc trưng EMBER và mô hình LightGBM với hiệu quả cao trên dữ liệu quy mô lớn [17]; Choudhary et al. mở rộng hướng này bằng lựa chọn đặc trưng, giảm số chiều từ 2.381 xuống 384 trên các tập EMBER, BODMAS và ERMDS [18]. Tuy nhiên, các phương pháp tĩnh thuần túy mất tính phân biệt khi tệp PE bị đóng gói, do cấu trúc quan sát được khác xa tệp gốc.

Đồ án kế thừa bộ đặc trưng EMBER và đánh giá ba thuật toán phân loại trên tập dữ liệu tự thu thập, đồng thời tích hợp phân tích động với Sysmon và SilkETW để thu thập chuỗi hành vi, IOC và ánh xạ MITRE ATT&CK. Khác với Cuckoo và Any.Run, đồ án khai thác trực tiếp ETW provider của .NET Runtime thay vì can thiệp lời gọi API, nhờ đó quan sát được các kỹ thuật thực thi trong bộ nhớ mà phân tích tĩnh và Sysmon khó ghi nhận, đồng thời giảm khả năng bị mã độc phát hiện môi trường phân tích. Việc bổ sung giai đoạn động cũng giúp xử lý các mẫu bị

đóng gói — tình huống mà đặc trưng tĩnh kém hiệu quả.

Những khác biệt trên được tổng hợp trong Bảng 2.4, đặt đồ án bên cạnh hai hệ thống phân tích động phổ biến để làm rõ vị trí của hướng tiếp cận. Điểm phân biệt rõ nhất là cơ chế thu thập: trong khi Cuckoo và Any.Run can thiệp lời gọi API ở user-mode, đồ án quan sát ở mức kernel qua Sysmon kết hợp ETW của .NET Runtime, nhờ đó vừa khó bị mã độc phát hiện, vừa nhìn được hoạt động thực thi trong bộ nhớ mà hai hệ thống kia khó ghi nhận.

Bảng 2.4: So sánh đặc điểm giữa đồ án và các hệ thống phân tích động hiện có [7].

Đặc điểm	**Cuckoo**	**Any.Run**	**Đồ án**
Cơ chế thu thập động	API hooking	API hooking	Sysmon + ETW
Quan sát .NET trong bộ nhớ	Hạn chế	Hạn chế	Có
Kháng phát hiện hook	Thấp	Trung bình	Cao
Phân loại tĩnh bằng học máy	Không	Không	Có
Hợp nhất tĩnh–động	Không	Không	Có
Ánh xạ MITRE ATT&CK	Một phần	Có	Có
Mã nguồn mở / tự triển khai	Có	Không	Có

Các cơ sở trên cho thấy phân tích tĩnh và phân tích động bù trừ cho nhau: đặc trưng tĩnh PE cho phép phân loại nhanh nhưng mất tác dụng khi mẫu bị đóng gói, còn giám sát động bằng Sysmon kết hợp ETW quan sát được hành vi thực — kể cả hoạt động .NET trong bộ nhớ mà Sysmon bỏ sót. Đây là căn cứ để đồ án kết hợp hai hướng trong thiết kế ở Chương 3.

CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế của hệ thống, bắt đầu từ phân tích yêu cầu và kiến trúc tổng thể hai giai đoạn, rồi đi vào thiết kế giai đoạn phân tích tĩnh, giai đoạn phân tích động và pipeline tích hợp.

3.1 Phân tích yêu cầu

Trước khi đi vào thiết kế chi tiết, cần xác định rõ các yêu cầu mà hệ thống phải đáp ứng, vì đây vừa là định hướng cho các quyết định thiết kế, vừa là tiêu chí để đánh giá hệ thống về sau. Các yêu cầu được phân thành hai nhóm. Yêu cầu chức năng mô tả những chức năng mà hệ thống cung cấp. Yêu cầu phi chức năng quy định các ràng buộc về chất lượng vận hành — như tính cô lập của môi trường thực thi, mức độ tự động hóa và khả năng chịu lỗi khi một giai đoạn gặp sự cố.

3.1.1 Yêu cầu chức năng

Bảng 3.1 liệt kê các yêu cầu chức năng của hệ thống.

Bảng 3.1: Yêu cầu chức năng của hệ thống.

Mã	Yêu cầu
F1	Nhận tệp PE 32/64-bit (`.exe`, `.dll`) làm đầu vào
F2	Trích xuất vector đặc trưng 2.381 chiều từ cấu trúc tệp PE mà không thực thi tệp
F3	Phân loại tệp là malware hoặc benign, kèm xác suất dự đoán từ mô hình học máy đã huấn luyện
F4	Thực thi mẫu trong môi trường sandbox Windows VM cô lập với mạng giả lập
F5	Thu thập sự kiện hệ thống từ Sysmon và SilkETW trong suốt quá trình thực thi
F6	Trích xuất IOC gồm địa chỉ IP, tên miền, URL, tệp được tạo và bị xóa, khóa registry, câu lệnh, named pipe, PID tiến trình bị tiêm mã và giá trị băm SHA-256
F7	Xây dựng cây tiến trình và chuỗi hành vi theo thứ tự thời gian
F8	Ánh xạ hành vi quan sát được sang các kỹ thuật MITRE ATT&CK tương ứng
F9	Xuất báo cáo phân tích dạng JSON

3.1.2 Yêu cầu phi chức năng

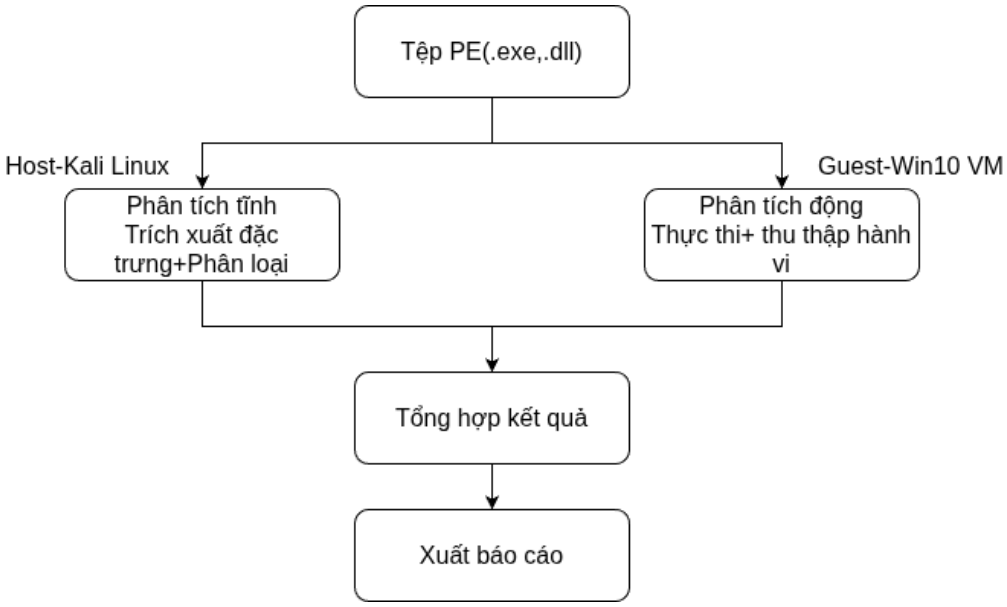
Bảng 3.2 liệt kê các yêu cầu phi chức năng của hệ thống.

Bảng 3.2: Yêu cầu phi chức năng của hệ thống.

Mã	Yêu cầu
NF1	Mẫu độc hại chỉ được thực thi bên trong máy ảo, không bao giờ chạy trực tiếp trên máy chủ phân tích
NF2	VM được khôi phục về snapshot sạch `Clean_Base` trước mỗi lần phân tích, đảm bảo tính độc lập giữa các lần chạy
NF3	Pipeline vận hành hoàn toàn tự động từ khâu nhận tệp đến khâu xuất báo cáo, không yêu cầu can thiệp thủ công
NF4	Nếu giai đoạn phân tích động thất bại, hệ thống vẫn lưu kết quả phân tích tĩnh và xuất báo cáo với thông tin hiện có
NF5	Thời gian thu thập sự kiện trong giai đoạn động có thể cấu hình, mặc định là 120 giây

3.2 Kiến trúc tổng thể

Hình 3.1 minh họa kiến trúc tổng thể của hệ thống, gồm hai giai đoạn phân tích tĩnh và động.



Hình 3.1: Kiến trúc tổng thể của hệ thống phân tích phần mềm độc hại.

Giai đoạn phân tích tĩnh chạy trên máy chủ Kali Linux: trích xuất vector đặc trưng từ cấu trúc tệp PE, sau đó đưa qua mô hình phân loại để trả về nhãn cùng xác suất dự đoán. Thuật toán được sử dụng là LightGBM, được lựa chọn dựa trên kết quả so sánh thực nghiệm với Random Forest và XGBoost trên cùng bộ đặc trưng (trình bày tại Chương 5).

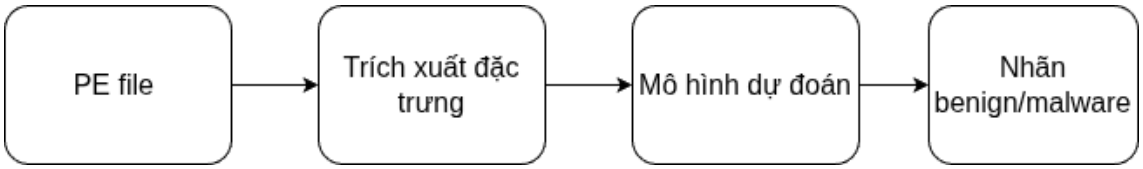
Giai đoạn phân tích động chạy trong máy ảo Windows 10 trên VirtualBox: khôi phục môi trường về trạng thái sạch, thực thi mẫu trong điều kiện mạng giả lập, đồng thời thu thập sự kiện hệ thống qua Sysmon và SilkETW. Sau khi thu thập xong, dữ liệu sự kiện được kéo về máy chủ để xử lý thành chuỗi hành vi, IOC và

ánh xạ MITRE ATT&CK.

Khi cả hai giai đoạn hoàn tất, hệ thống tổng hợp kết quả và xuất báo cáo JSON duy nhất chứa kết luận, bằng chứng hành vi, cây tiến trình và các chỉ số xâm phạm.

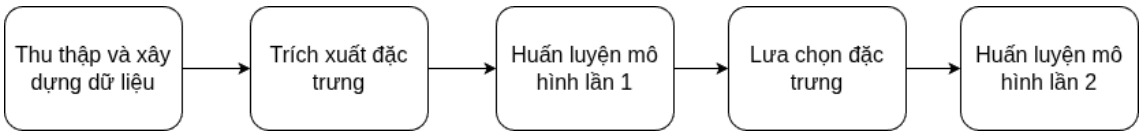
3.3 Thiết kế giai đoạn phân tích tĩnh

Giai đoạn phân tích tĩnh phân loại tệp PE dựa hoàn toàn vào đặc trưng cấu trúc mà không thực thi tệp. Hình 3.2 mô tả luồng xử lý: tệp PE qua bộ trích xuất đặc trưng để tạo vector, sau đó mô hình phân loại trả về nhãn dự đoán cùng xác suất tương ứng.



Hình 3.2: Luồng phân tích tĩnh khi tiếp nhận tệp đầu vào

Để xây dựng mô hình phân loại trên, hệ thống trải qua quy trình huấn luyện gồm năm bước như thể hiện trong Hình 3.3: thu thập và xây dựng tập dữ liệu, trích xuất đặc trưng, huấn luyện sơ bộ để tính độ quan trọng đặc trưng, lựa chọn tập con đặc trưng tối ưu, và huấn luyện lại mô hình cuối dựa trên tập đặc trưng đã chọn.



Hình 3.3: Quy trình xây dựng mô hình phân loại tĩnh

3.3.1 Thu thập và xây dựng tập dữ liệu

Tập dữ liệu gồm hai lớp: khoảng 132.000 tệp mã độc thu thập từ các kho lưu trữ công khai và khoảng 110.000 tệp lành tính từ phần mềm hợp pháp và thành phần hệ điều hành Windows. Sau khi kiểm tra hợp lệ PE, loại trùng lặp theo giá trị băm và tiền xử lý, tập dữ liệu có hơn 201.000 mẫu phân bố gần cân bằng giữa hai lớp. Thống kê chi tiết theo nguồn và bước tiền xử lý được trình bày tại Chương 4.

3.3.2 Trích xuất đặc trưng

Mỗi tệp PE được chuyển đổi thành vector 2.381 chiều theo chín nhóm đặc trưng EMBER (Bảng 2.2 trong Chương 2). Các nhóm có kích thước cố định như entropy và header info được giữ nguyên giá trị số. Các nhóm có độ dài thay đổi như danh sách hàm import được chuẩn hóa về kích thước cố định qua kỹ thuật feature hashing [26]: mỗi tên hàm được băm thành một vị trí trong mảng, nhờ đó mọi tệp PE đều tạo ra vector cùng kích thước.

3.3.3 Huấn luyện mô hình

Như minh họa trong Hình 3.3, quy trình huấn luyện được thiết kế thành hai lần huấn luyện nối tiếp, với bước lựa chọn đặc trưng đặt ở giữa: lần thứ nhất để chọn thuật toán phù hợp, lần thứ hai để chọn tập đặc trưng tối ưu cho thuật toán đó. Việc tách thành hai lần giúp mỗi quyết định được đưa ra độc lập và có cơ sở so sánh rõ ràng.

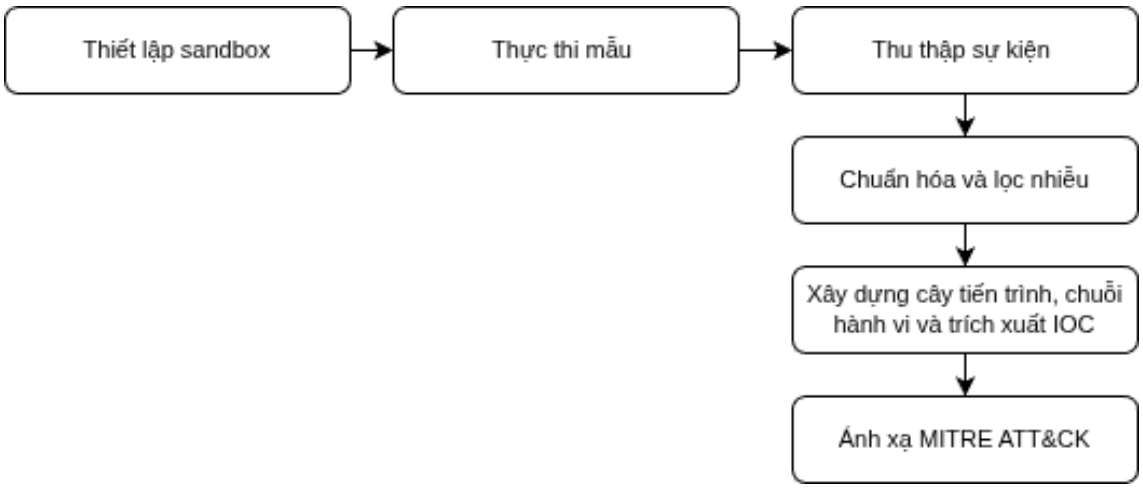
Ở lần huấn luyện thứ nhất, ba thuật toán Random Forest, XGBoost và LightGBM (Mục 2.3.2) được huấn luyện trên cùng tập dữ liệu và toàn bộ đặc trưng, trong cùng điều kiện. Nhờ giữ mọi yếu tố giống nhau, khác biệt về kết quả chỉ phản ánh bản chất của từng thuật toán, giúp việc lựa chọn được khách quan. Thuật toán cho hiệu năng tốt nhất được giữ lại làm mô hình triển khai.

Ở lần huấn luyện thứ hai, số chiều đặc trưng được giảm bớt nhằm thu được mô hình gọn và dễ diễn giải hơn mà vẫn duy trì hiệu năng. Các đặc trưng được xếp hạng theo Importance Gain (Mục 2.3.3) của mô hình đã chọn, sau đó chỉ giữ lại nhóm đặc trưng quan trọng nhất để huấn luyện lại. Thay vì ấn định trước một số chiều, hiệu năng được khảo sát ở nhiều mức khác nhau nhằm tìm điểm bão hòa — nơi việc thêm đặc trưng gần như không còn cải thiện kết quả — rồi chọn số chiều quanh điểm đó.

Để kết quả đánh giá đáng tin cậy, mô hình không chỉ được kiểm tra trên phần dữ liệu tách ra từ tập huấn luyện mà còn trên các tập dữ liệu độc lập hoàn toàn, trong đó có một tập công khai thu thập từ nguồn khác. Cách này cho phép đo khả năng tổng quát hóa của mô hình trên dữ liệu ngoài phân phối huấn luyện, thay vì chỉ trên dữ liệu cùng nguồn. Mô hình cuối còn được phân tích sâu hơn ở các khía cạnh như ngưỡng phân loại, các trường hợp phân loại sai và mức đóng góp của từng nhóm đặc trưng; toàn bộ kết quả được trình bày tại Chương 5.

3.4 Thiết kế giai đoạn phân tích động

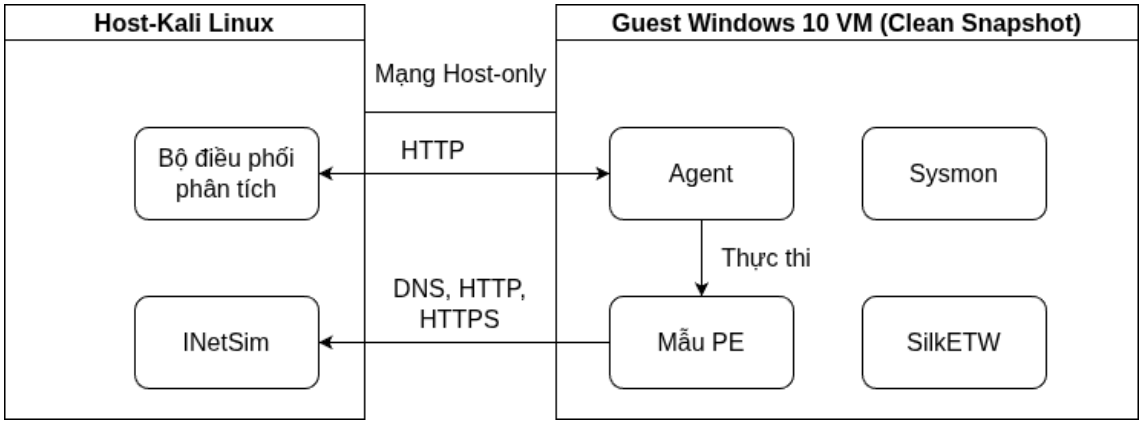
Giai đoạn phân tích động thực thi mẫu nghi ngờ trong môi trường sandbox cô lập nhằm thu thập hành vi thực tế trong quá trình chạy, bổ sung cho kết quả phân loại tĩnh. Luồng xử lý gồm sáu bước như minh họa trong Hình 3.4: (1) thiết lập môi trường sandbox, (2) thực thi mẫu trong máy ảo, (3) thu thập sự kiện từ Sysmon và SilkETW, (4) chuẩn hóa và lọc nhiễu, (5) xây dựng cây tiến trình và chuỗi hành vi đồng thời trích xuất IOC, và (6) ánh xạ hành vi sang MITRE ATT&CK. Tại mỗi bước, nếu xảy ra lỗi, hệ thống ghi nhận và dừng giai đoạn động, kết quả phân tích tĩnh vẫn được giữ lại (yêu cầu NF4).



Hình 3.4: Luồng xử lý giai đoạn phân tích động.

3.4.1 Thiết lập môi trường sandbox

Môi trường phân tích động gồm hai thành phần: máy chủ phân tích (host) chạy Kali Linux và máy ảo (guest) chạy Windows 10 trên nền tảng VirtualBox, kết nối qua mạng host-only đảm bảo máy ảo chỉ giao tiếp được với host mà không truy cập trực tiếp ra Internet (yêu cầu NF1). Hình 3.5 phác họa kiến trúc môi trường sandbox.



Hình 3.5: Kiến trúc môi trường sandbox.

Phía host triển khai INetSim — công cụ giả lập các dịch vụ mạng phổ biến như DNS, HTTP, HTTPS. Khi mã độc cố gắng kết nối ra ngoài, INetSim cung cấp phản hồi giả, nhờ đó hệ thống vừa quan sát được hành vi kết nối mạng của mẫu, vừa ngăn chặn rủi ro lây lan.

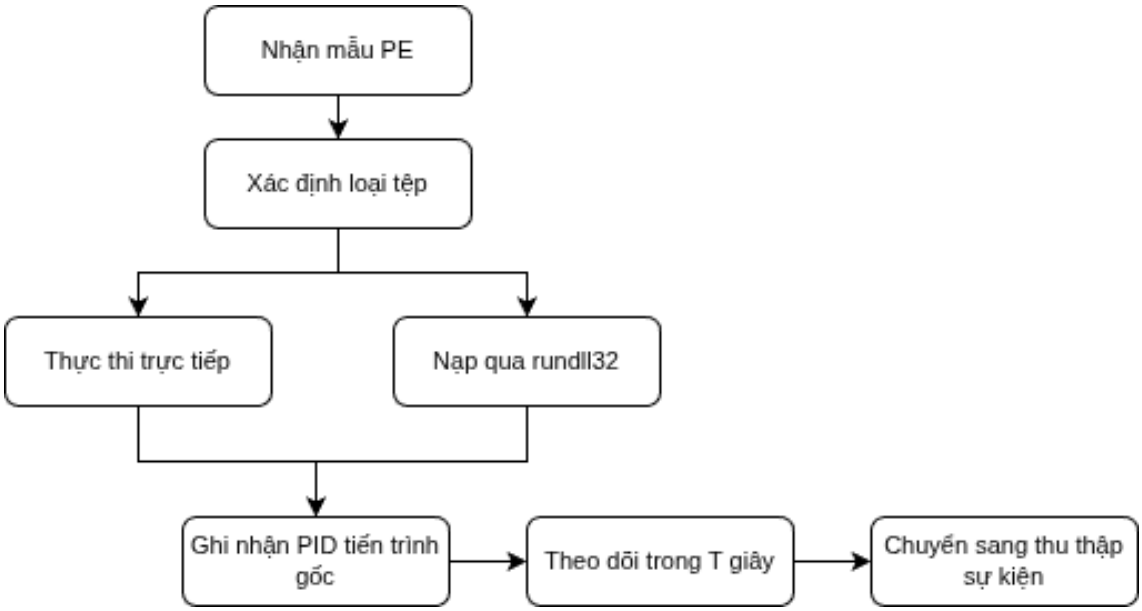
Phía guest được cài đặt sẵn ba thành phần: Sysmon giám sát sự kiện hệ điều hành, SilkETW thu thập sự kiện từ .NET Runtime và một agent chạy dưới dạng dịch vụ HTTP bên trong VM. Mẫu PE được tải lên VM thông qua cơ chế Guest Additions của VirtualBox, cho phép truyền tệp trực tiếp giữa host và guest. Agent đóng vai trò đầu mối điều khiển các tác vụ phân tích bên trong VM, bao gồm thực

thi mẫu, điều phối quá trình thu thập sự kiện và chuyển kết quả về host. Toàn bộ quá trình phân tích động do đó được điều khiển từ xa mà không cần tương tác trực tiếp với giao diện Windows (yêu cầu NF3).

Trước mỗi lần phân tích, VM được khôi phục về snapshot sạch nhằm đảm bảo mỗi mẫu được thực thi trong môi trường độc lập và không chịu ảnh hưởng từ các lần chạy trước (yêu cầu NF2).

3.4.2 Thực thi mẫu

Sau khi môi trường sandbox được khởi tạo, mẫu PE được thực thi bên trong máy ảo để tạo ra các hành vi có thể quan sát được. Quá trình thực thi được điều phối hoàn toàn bởi Agent bên trong VM, không yêu cầu tương tác thủ công với hệ điều hành (yêu cầu NF3). Hình 3.6 tóm tắt luồng xử lý của bước này.



Hình 3.6: Luồng thực thi mẫu trong môi trường sandbox.

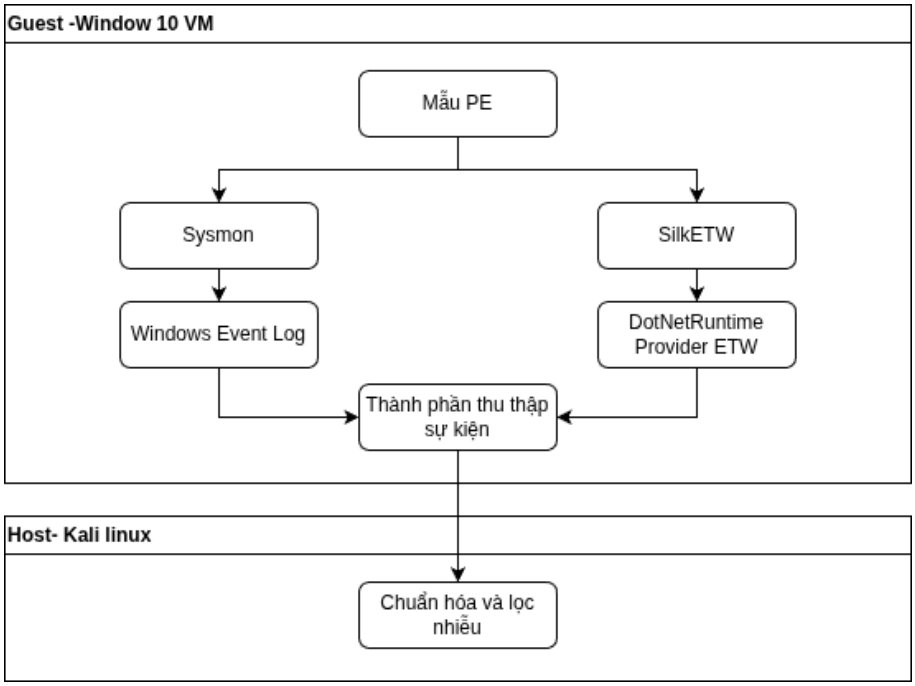
Agent thực thi mẫu theo cách phù hợp với định dạng: `.exe` được khởi chạy trực tiếp; `.dll` nạp qua `rundll32.exe`. Sau khi khởi chạy, Agent ghi nhận PID của tiến trình gốc và chuyển cho bước thu thập sự kiện ở Mục 3.4.3.

Thời gian quan sát có thể cấu hình, mặc định 120 giây (yêu cầu NF5). Khi hết thời gian hoặc gặp lỗi không thể phục hồi, hệ thống kết thúc thực thi và giữ lại kết quả phân tích tĩnh theo yêu cầu NF4.

3.4.3 Thu thập sự kiện

Trong suốt khoảng thời gian thực thi mẫu, hệ thống thu thập sự kiện song song từ Sysmon và SilkETW như đã trình bày tại Mục 2.4.2 và Mục 2.4.3. Sysmon cung cấp các sự kiện ở mức hệ điều hành như tạo tiến trình, thao tác tệp, thay đổi registry và kết nối mạng; trong khi SilkETW ghi nhận các sự kiện từ .NET Runtime nhằm

quan sát các hoạt động thực thi bên trong môi trường .NET.



Hình 3.7: Luồng thu thập sự kiện trong môi trường sandbox.

Hình 3.7 mô tả kiến trúc thu thập sự kiện. Trước khi mẫu được thực thi, bộ thu thập ghi nhận số thứ tự (`RecordId`) mới nhất của kênh Sysmon làm mốc bắt đầu. Trong quá trình thực thi, Sysmon được truy vấn định kỳ để lấy riêng các sự kiện có `RecordId` lớn hơn mốc đó, không lẫn với lịch sử hệ thống có trước phiên phân tích.

Sau khi Agent khởi chạy mẫu, PID của tiến trình gốc được chuyển cho bộ thu thập; mỗi tiến trình con mới sinh ra tự động được thêm vào danh sách theo dõi. Phạm vi giám sát do đó bao phủ toàn bộ cây tiến trình của mẫu, loại bỏ các sự kiện từ tiến trình không liên quan.

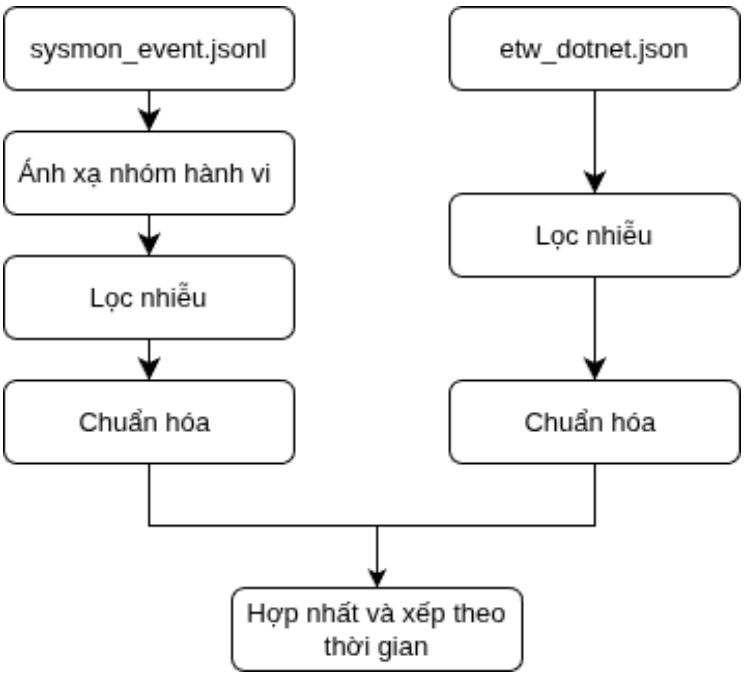
Đối với SilkETW, quá trình ghi nhận sự kiện được khởi động đồng thời với việc thực thi mẫu và kết thúc khi hết thời gian quan sát. SilkETW lắng nghe provider `Microsoft-Windows-DotNETRuntime` và ghi các sự kiện thu được ra tệp log.

Kết thúc giai đoạn thu thập, dữ liệu Sysmon được lưu dưới dạng `sysmon_events.jsonl`, còn dữ liệu ETW được lưu dưới dạng `etw_dotnet.json`. Hai tệp này được sử dụng làm đầu vào cho bước chuẩn hóa và lọc nhiễu ở Mục 3.4.4.

3.4.4 Chuẩn hóa và lọc nhiễu

Dữ liệu thu được từ Sysmon và SilkETW có định dạng và cấu trúc khác nhau do được sinh ra từ hai cơ chế giám sát độc lập; trước khi phân tích hành vi, hệ thống thực hiện bước chuẩn hóa nhằm chuyển toàn bộ sự kiện về một mô hình dữ liệu

thống nhất. Hình 3.8 minh họa luồng xử lý của bước này.



Hình 3.8: Luồng chuẩn hóa và lọc nhiễu dữ liệu.

Đối với Sysmon, mỗi sự kiện được ánh xạ từ Event ID sang nhóm hành vi tương ứng như tạo tiến trình, thao tác tệp, thay đổi registry hoặc kết nối mạng. Các trường dữ liệu quan trọng được trích xuất và chuẩn hóa về cùng tên gọi. Đồng thời, hệ thống loại bỏ các sự kiện phát sinh từ một số tiến trình hạ tầng đã biết của môi trường phân tích như Sysmon, SilkETW, VirtualBox và các dịch vụ hệ thống không liên quan nhằm giảm nhiễu trong dữ liệu đầu vào.

Đối với SilkETW, hệ thống chỉ giữ lại các sự kiện phản ánh hoạt động nạp assembly (tải đơn vị mã .NET đã biên dịch vào bộ nhớ tại thời điểm chạy) của .NET Runtime và loại bỏ các sự kiện phát sinh từ tiến trình hạ tầng của môi trường phân tích. Các sự kiện còn lại được chuẩn hóa về cùng mô hình dữ liệu thống nhất với Sysmon, đảm bảo hai nguồn có thể được xử lý chung ở các bước sau.

Sau khi chuẩn hóa, mọi sự kiện đều được biểu diễn dưới dạng bản ghi gồm thời gian xảy ra, nguồn sự kiện, loại hành vi, PID tiến trình và dữ liệu chi tiết đi kèm. Toàn bộ sự kiện từ hai nguồn được hợp nhất, sắp xếp theo thứ tự thời gian tăng dần và sử dụng làm đầu vào cho bước xây dựng cây tiến trình, chuỗi hành vi và trích xuất IOC ở Mục 3.4.5.

3.4.5 Xây dựng biểu diễn hành vi và ánh xạ MITRE ATT&CK

Tập sự kiện đã chuẩn hóa được tổ chức thành ba thành phần biểu diễn hành vi: cây tiến trình, chuỗi hành vi và IOC.

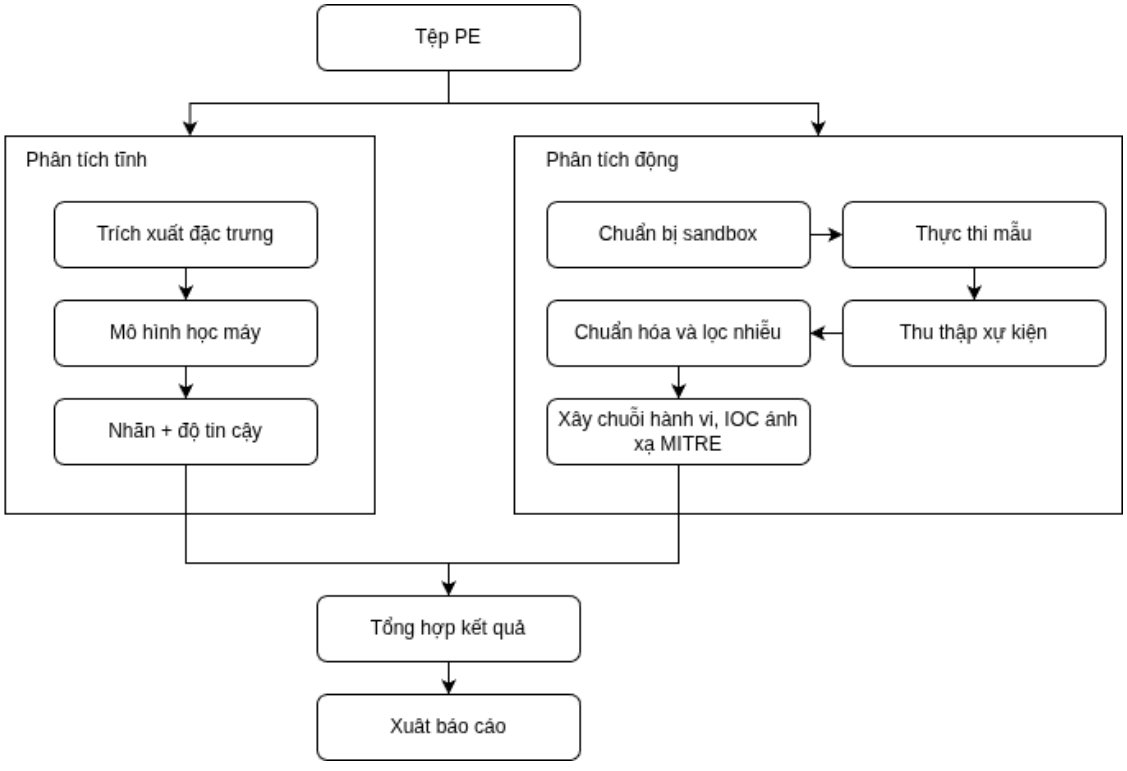
Cây tiến trình được xây dựng từ quan hệ cha–con giữa các tiến trình, lưu PID,

đường dẫn thực thi, dòng lệnh và thời điểm khởi chạy. Chuỗi hành vi gom các sự kiện thành mười một nhóm hành vi (tiến trình, tiêm mã, nạp thư viện động, tệp, registry, mạng, DNS, named pipe, WMI, .NET Runtime và nhóm khác) sắp xếp theo thứ tự thời gian. IOC được trích xuất từ nội dung sự kiện, gồm mười danh mục: địa chỉ IP, tên miền, URL, tệp được tạo, tệp bị xóa, khóa registry, câu lệnh, named pipe, PID tiến trình bị tiêm mã và giá trị băm SHA-256.

Dựa trên ba thành phần trên, hệ thống áp dụng tập luật để ánh xạ sang các kỹ thuật MITRE ATT&CK. Mỗi luật kiểm tra điều kiện trên cây tiến trình, chuỗi hành vi hoặc IOC; khi thỏa mãn, kỹ thuật ATT&CK tương ứng được ghi nhận. Các luật được tổ chức theo nhóm hành vi: thực thi tiến trình, duy trì truy cập, tiêm mã, giao tiếp mạng và thực thi .NET. Cách tiếp cận dựa trên luật cho phép bổ sung kỹ thuật mới hoặc điều chỉnh tiêu chí nhận diện mà không cần huấn luyện lại mô hình.

3.5 Thiết kế pipeline tích hợp

Pipeline tích hợp phối hợp hai giai đoạn đã thiết kế tại Mục 3.3 và Mục 3.4 để đưa ra kết quả phân loại cuối cùng và xuất báo cáo; luồng xử lý tổng thể được minh họa trong Hình 3.9.



Hình 3.9: Luồng xử lý end-to-end của pipeline tích hợp.

Thiết kế hướng đến hai mục tiêu: rút ngắn thời gian tổng thể bằng cách chạy song song hai giai đoạn, và đảm bảo kết quả luôn được xuất ra ngay cả khi một giai đoạn gặp lỗi (yêu cầu NF4).

3.5.1 Luồng xử lý end-to-end

Ngay sau khi tiếp nhận tệp PE đầu vào, pipeline khởi động đồng thời hai luồng độc lập. Luồng phân tích tĩnh trích xuất đặc trưng, đưa qua mô hình học máy và trả về nhãn phân loại cùng xác suất tương ứng. Luồng phân tích động lần lượt thực hiện các bước chuẩn bị sandbox, thực thi mẫu, thu thập sự kiện, chuẩn hóa và lọc nhiễu, sau đó xây dựng chuỗi hành vi, trích xuất IOC và ánh xạ MITRE ATT&CK. Hai luồng không phụ thuộc vào nhau; thời gian hoàn thành của toàn pipeline xấp xỉ thời gian của luồng chậm hơn.

Sau khi cả hai luồng hoàn tất, hệ thống kết hợp kết quả ML và tín hiệu phân tích động theo năm quy tắc để đưa ra kết luận cuối: `malware`, `suspicious`, `benign` hoặc `unknown`. Tín hiệu tiêm mã tiến trình được ưu tiên cao nhất và có thể thay đổi kết luận của mô hình ML; nếu giai đoạn động không thu được tín hiệu hành vi nào, hệ thống ghi cảnh báo nghi ngờ mẫu dùng kỹ thuật lẩn tránh sandbox. Bảng quy tắc đầy đủ được trình bày tại Chương 4 (Bảng 4.6).

3.5.2 Cấu trúc báo cáo đầu ra

Kết quả phân tích được tổng hợp thành một báo cáo JSON duy nhất gồm năm nhóm. Ba nhóm đầu là kết quả chính: (1) kết quả phân loại — nhãn kết luận (`verdict`), độ tin cậy tổng hợp (`confidence`), nhãn và xác suất thô của mô hình ML; (2) thông tin hành vi — chuỗi hành vi, cây tiến trình, mô tả tóm tắt và danh sách kỹ thuật MITRE ATT&CK; (3) chỉ số xâm phạm — địa chỉ IP, tên miền, URL, tệp được tạo, tệp bị xóa, khóa registry, câu lệnh, named pipe, danh sách PID tiêm mã và giá trị băm tệp liên quan. Hai nhóm còn lại phục vụ truy vết: (4) metadata phiên — SHA-256, kích thước mẫu, thời gian phân tích và danh sách các giai đoạn đã hoàn thành; (5) cảnh báo và lỗi từ từng giai đoạn. Cấu trúc đầy đủ được trình bày tại Chương 4.

Độ tin cậy tổng hợp khác với xác suất thô của mô hình ML: xác suất thô phản ánh riêng kết quả giai đoạn tĩnh, còn độ tin cậy tổng hợp có thể được điều chỉnh dựa trên kết quả phân tích động.

Điểm cốt lõi của thiết kế là kiến trúc hai giai đoạn chạy song song, hợp nhất kết quả tĩnh và động theo một tập luật cố định, và luôn xuất được báo cáo ngay cả khi một giai đoạn gặp lỗi. Cách hiện thực hóa kiến trúc này được trình bày ở Chương 4.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

Chương này trình bày cách hiện thực các thành phần đã thiết kế ở Chương 3: môi trường triển khai, giai đoạn phân tích tĩnh, giai đoạn phân tích động, pipeline tích hợp và giao diện web demo.

4.1 Môi trường triển khai

Hệ thống chạy trên hai nút phân tách: máy chủ vật lý cài Kali Linux đóng vai host điều phối, và máy ảo Windows 10 22H2 x64 đóng vai guest thực thi mẫu. Bảng 4.1 liệt kê các thông số cấu hình của từng nút.

Bảng 4.1: Cấu hình phần cứng và phần mềm môi trường triển khai.

Thành phần	Thông số	Giá trị
Máy chủ (host)	CPU	AMD Ryzen 7 8845HS
	RAM	32 GB
	Hệ điều hành	Kali Linux
	Nền tảng ảo hóa	VirtualBox
	Mạng giả lập	INetSim
	Thời gian chờ khởi động	120 giây
	Cửa sổ quan sát (mặc định)	120 giây
Máy ảo (guest)	vCPU	4
	RAM	4 GB
	Hệ điều hành	Windows 10 22H2 x64
	Tên VM	`win10`
	Snapshot sạch	`Clean_Base`
	Địa chỉ IP	`192.168.56.106`
	Sysmon	v15.x (schema 4.90)
	SilkETW	v8
	Agent HTTP	FastAPI, cổng `8001`

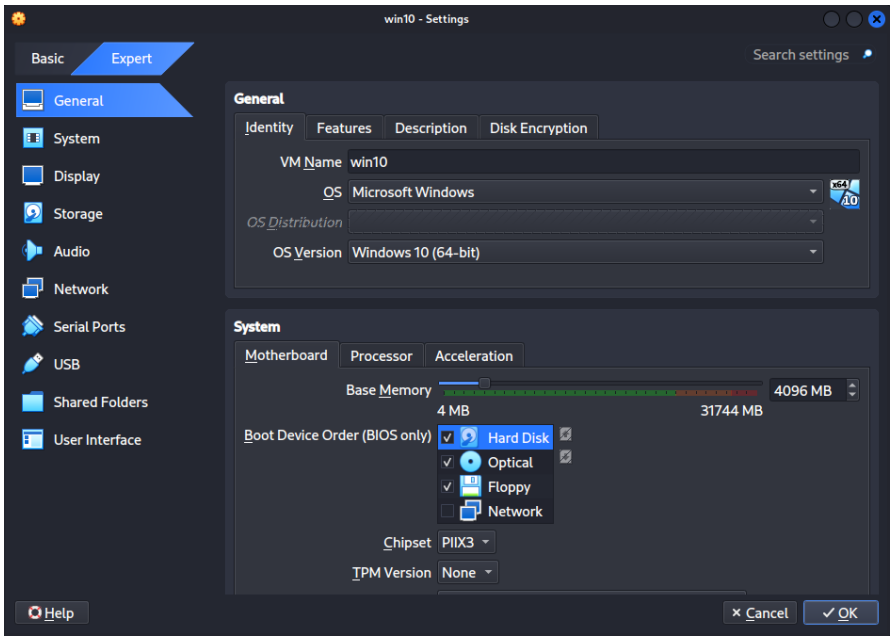
Máy chủ vật lý cài Kali Linux điều phối toàn bộ vòng đời guest qua `VBoxManage`: khôi phục snapshot trước mỗi phiên, truyền tệp mẫu, truy vấn trạng thái Agent, rồi tắt máy khi phiên kết thúc. Pipeline viết bằng Python; giai đoạn phân tích tĩnh thực hiện trực tiếp trên host, song song với quan sát động ở guest.

4.1.1 Máy ảo Windows

Máy ảo chạy Windows 10 22H2 x64, đặt tên `win10` trong VirtualBox. Trước mỗi phiên, pipeline phục hồi snapshot `Clean_Base` để đưa guest về trạng thái nguyên vẹn, nhanh hơn tái khởi động và loại bỏ dữ liệu tồn dư từ mẫu trước.

Ba công cụ được nhúng sẵn trong snapshot: Sysmon v15 ghi các sự kiện tiến trình, mạng, registry và tệp; SilkETW v8 thu sự kiện ETW từ `Microsoft-`

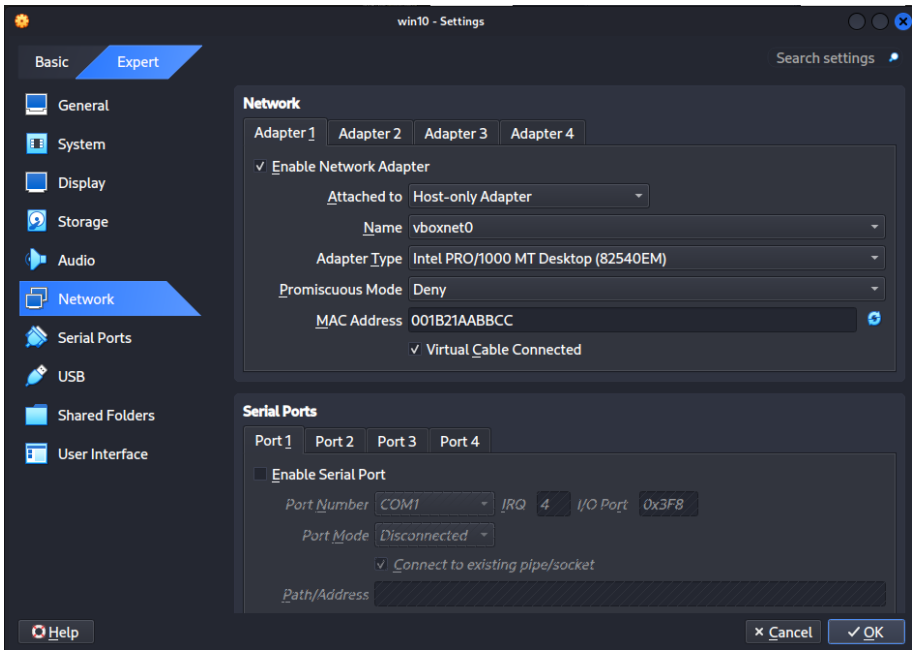
`Windows-DotNETRuntime` để bắt hoạt động .NET; và Agent (FastAPI, cổng `8001`) làm điểm nhận lệnh từ host. Sau khi guest khởi động, host thăm dò `/health` tối đa 120 giây để xác nhận Agent đã sẵn sàng nhận lệnh. Hình 4.1 minh họa cấu hình phần cứng máy ảo.



Hình 4.1: Cấu hình phần cứng máy ảo `win10` trong VirtualBox Settings.

4.1.2 Cấu hình mạng và INetSim

Adapter mạng guest ở chế độ host-only (IP tĩnh `192.168.56.106`), không có kết nối trực tiếp ra Internet. INetSim chạy trên host giả lập DNS, HTTP/HTTPS và TCP để duy trì luồng thực thi mẫu mà không phát sinh lưu lượng ra mạng thực tế. Hình 4.2 trình bày cấu hình adapter mạng host-only trong VirtualBox Settings.



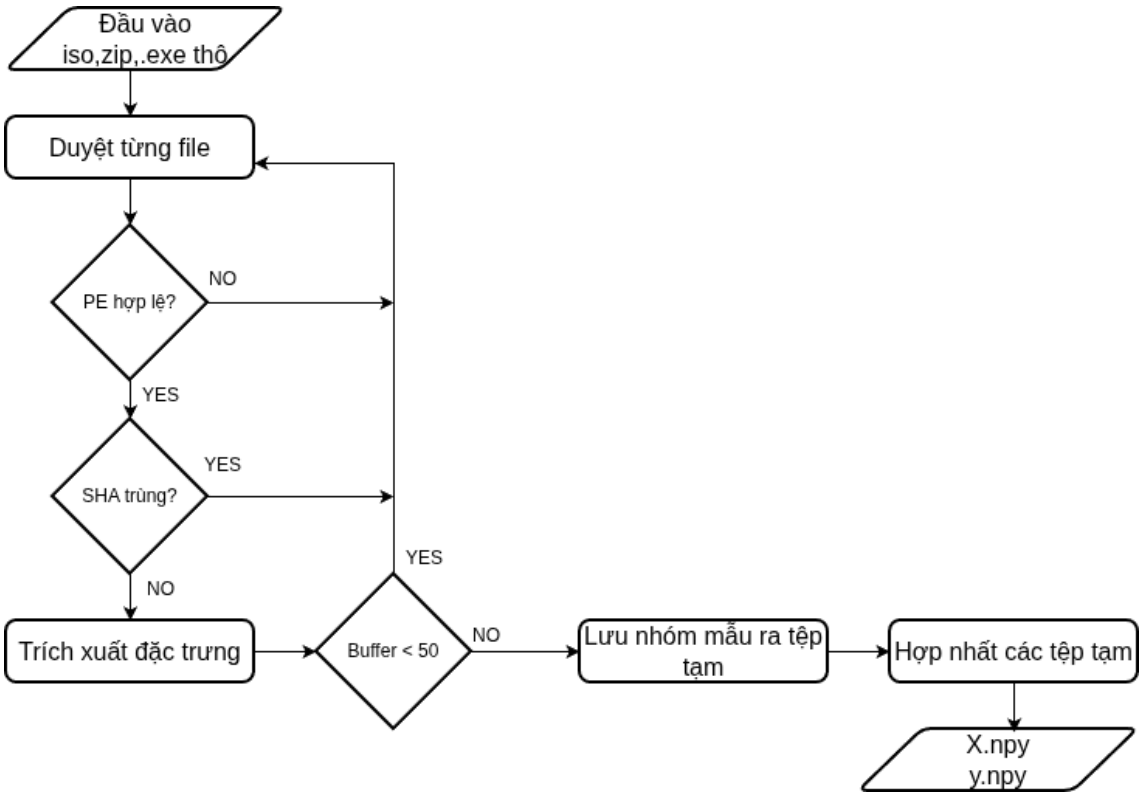
Hình 4.2: Cấu hình adapter mạng host-only của máy ảo trong VirtualBox Settings.

4.2 Triển khai giai đoạn phân tích tĩnh

4.2.1 Thu thập và xây dựng tập dữ liệu

Phần lành tính gồm 110.002 tệp PE thu thập từ tám phiên bản Windows và các nguồn phần mềm hợp pháp (Chocolatey, Sysinternals, Nirsoft và các bản phát hành mã nguồn mở trên GitHub); phần mã độc gồm 132.476 tệp từ VirusShare [27] và MalwareBazaar [28] thuộc nhiều họ phổ biến như AgentTesla, AsyncRAT, Emotet, STOP và nhiều họ khác. Thống kê chi tiết theo từng nguồn được trình bày tại Phụ lục A.1 và A.2. Do khối lượng hàng trăm nghìn mẫu, quá trình xây dựng tập huấn luyện được chia làm hai bước, trong đó bước đầu xử lý theo lô để kiểm soát mức sử dụng bộ nhớ.

Bước 1 — Trích xuất theo lô. Tệp PE được xử lý riêng theo từng nhãn bằng cơ chế lô 50 mẫu: kiểm tra chữ ký MZ, loại trùng lặp theo SHA-256, trích xuất vector 2.381 chiều và ghi tệp tạm sau mỗi lô để có thể tiếp tục nếu gián đoạn; sau khi duyệt hết, các lô được hợp nhất thành ma trận lành tính và mã độc riêng. Hình 4.3 minh họa quy trình xây dựng từng tập theo nhãn ở bước này.



Hình 4.3: Sơ đồ quá trình xây dựng ma trận đặc trưng từ tệp PE thô.

Bước 2 — Ghép, xáo trộn và tiền xử lý. Hai ma trận được nối theo hàng rồi xáo trộn với seed cố định ($seed = 42$). Tập kết hợp sau đó được làm sạch: lọc bỏ nhãn ngoài $\{0, 1\}$; điền giá trị NaN/Inf bằng trung vị theo đặc trưng; loại trùng lặp theo MD5 của từng dòng vector; xáo trộn lại lần cuối. Kết quả ghi đè `X_final.npy`

và `y_final.npy` để dùng cho huấn luyện.

Bảng 4.2 trình bày thành phần tập dữ liệu huấn luyện sau ba bước xử lý trên.

Bảng 4.2: Thành phần tập dữ liệu huấn luyện cuối cùng (`X_final.npy`, 2.381 đặc trưng/mẫu).

Nhãn	Số mẫu	Tỉ lệ
Lành tính (benign)	100.668	49,9%
Mã độc (malware)	100.972	50,1%
Tổng	**201.640**	**100%**

4.2.2 Trích xuất đặc trưng

Bộ trích xuất được hiện thực theo thiết kế tại Mục 3.3.2, dùng thư viện `pefile` để phân tích cấu trúc PE. Mỗi tệp được biểu diễn bằng một vector 2.381 chiều — hiểu đơn giản là một dãy 2.381 con số, mỗi số là giá trị của một đặc trưng — ghép từ chín nhóm đặc trưng EMBER (Bảng 2.2 ở Chương 2).

Phần lớn đặc trưng là các giá trị số có sẵn trong cấu trúc PE (kích thước tệp, entropy từng section, các trường header, tần suất byte...) nên được điền thẳng vào vector. Khó khăn nằm ở các nhóm chứa danh sách tên có độ dài thay đổi giữa các tệp — tên thư viện và hàm import, tên hàm export, tên section: số lượng tên mỗi tệp một khác nên không thể xếp trực tiếp vào số ô cố định. Feature hashing giải quyết việc này: một hàm băm ánh xạ mỗi tên tới một vị trí cố định trong mảng rồi cộng dồn vào vị trí đó, nhờ vậy dù một tệp import 5 hay 500 hàm, đầu ra vẫn là vector cùng số chiều. Cụ thể, tên thư viện import được băm vào 256 vị trí, cặp (thư viện, hàm) import vào 1.024 vị trí, tên hàm export vào 128 vị trí và tên section vào 50 vị trí. Tệp thiếu bảng import hoặc export cho vector toàn 0 ở nhóm tương ứng.

Bộ trích xuất chạy trên 242.478 tệp PE thô và tạo ra 201.640 vector hợp lệ. Tệp hợp lệ là tệp PE đúng định dạng, được `pefile` đọc thành công để tính đủ chín nhóm đặc trưng; phần còn lại bị loại vì không phải PE thật (thiếu chữ ký PE) hoặc có cấu trúc bị hỏng, cắt cụt nên không đọc được.

4.2.3 Huấn luyện và lưu mô hình

Lần huấn luyện thứ nhất — lựa chọn thuật toán. Ba thuật toán Random Forest, XGBoost và LightGBM được huấn luyện độc lập trên 161.312 mẫu (80%, seed $=$ 42) và đánh giá trên 40.328 mẫu kiểm thử chung, đảm bảo kết quả khác biệt đến từ bản chất thuật toán. Siêu tham số của từng thuật toán được trình bày trong Bảng 4.3.

Bảng 4.3: Siêu tham số ba thuật toán trong lần huấn luyện thứ nhất (toàn bộ 2.381 đặc trưng, tập train 161.312 mẫu).

Siêu tham số	Random Forest	XGBoost	LightGBM
Số cây / vòng tối đa	500	1.000	1.000
Chiều sâu tối đa	15	15	15
Tỉ lệ đặc trưng sử dụng	0,8	0,8	0,8
Mẫu tối thiểu mỗi lá	20	20	—
Tốc độ học	—	0,05	0,05
Số lá tối đa	—	—	1.024
Early stopping	—	50 vòng	50 vòng

Kết quả so sánh trên tập kiểm thử nội bộ (Chương 5) cho thấy LightGBM đạt hiệu năng tốt nhất và được chọn cho lần huấn luyện thứ hai. Chỉ số Importance Gain của mô hình này được dùng để xếp hạng 2.381 đặc trưng theo đóng góp vào khả năng phân loại; ba tập chỉ số ứng với $N \in \{128, 256, 384\}$ đặc trưng có đóng góp cao nhất được lưu lại theo ba ngưỡng tham chiếu từ Choudhary et al. [18].

Lần huấn luyện thứ hai — lựa chọn tập con đặc trưng. Để định vị số chiều cần thiết, số đặc trưng được khảo sát trên lưới mười lăm mốc từ 8 đến 2.381: mỗi mốc rút gọn dữ liệu theo bộ chỉ số tương ứng và huấn luyện LightGBM một lần với cùng siêu tham số, cho ra đường hiệu năng theo số chiều dùng tại Mục 5.2.2. Riêng ba ngưỡng tham chiếu 128, 256 và 384 được huấn luyện đầy đủ hơn bằng 5-fold Stratified Cross-Validation (seed $= 42$), giữ lại fold đạt AUC-ROC cao nhất làm mô hình đại diện; Bảng 4.4 trình bày siêu tham số sử dụng.

Bảng 4.4: Siêu tham số LightGBM trong lần huấn luyện thứ hai (tập con đặc trưng, 5-fold CV trên 201.640 mẫu).

Siêu tham số	Giá trị
Tốc độ học	0,05
Số lá tối đa	512
Chiều sâu tối đa	12
Tỉ lệ đặc trưng sử dụng	0,8
Số vòng tối đa	600
Early stopping	30 vòng

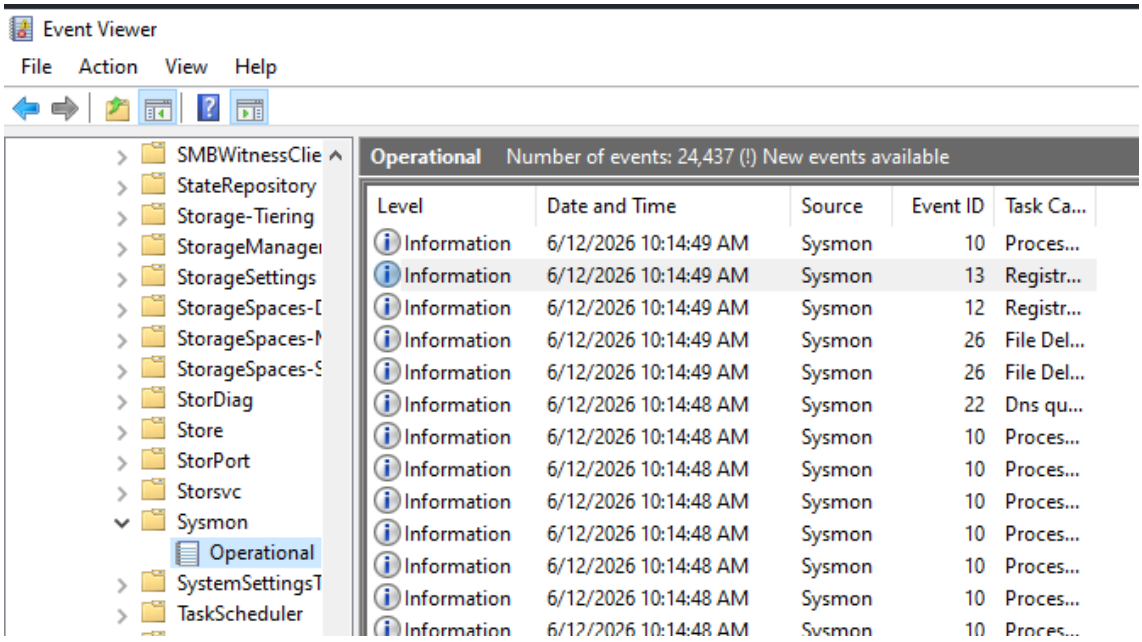
Kết quả so sánh bốn mô hình trên tập EMBER 2024 được trình bày tại Chương 5. Mô hình Top-384 được chọn tích hợp vào pipeline; khi suy luận, mô hình nhận vector 2.381 chiều, rút gọn xuống 384 chiều theo bộ chỉ số đã lưu rồi trả về nhãn và xác suất malware. Ngưỡng phân loại τ được xác định qua phân tích đánh đổi FPR/FNR, trình bày tại Chương 5.

4.3 Triển khai giai đoạn phân tích động

4.3.1 Cấu hình Sysmon

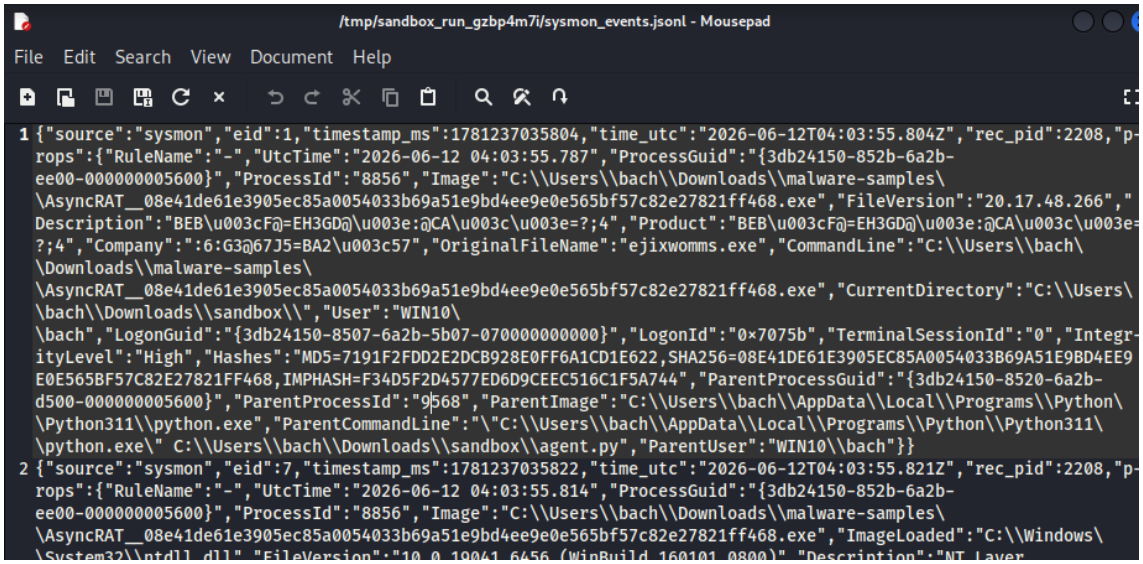
Sysmon v15 (schema 4.90) được nhúng sẵn vào snapshot `Clean_Base` nên không cần cài lại sau mỗi phiên. Hệ thống đọc trực tiếp kênh `Microsoft-Windows-Sysmon/Operational` qua PowerShell, ghi lại số thứ tự sự kiện (`RecordId`) mới nhất làm mốc trước khi mẫu chạy; mỗi vòng lặp 100 ms chỉ lấy sự kiện có `RecordId` lớn hơn mốc và chuyển đổi sang JSON, đảm bảo đầu ra chỉ chứa sự kiện phát sinh trong phiên hiện tại. Phạm vi giám sát được thu hẹp về cây tiến trình của mẫu: bộ thu thập theo dõi PID gốc và tự động bổ sung PID con; sự kiện từ PID ngoài danh sách bị bỏ qua. Danh sách 23 Event ID được liệt kê tại Phụ lục B.1 (Bảng B.1).

Hình 4.4 cho thấy các sự kiện Sysmon thô trên kênh `Microsoft-Windows-Sysmon/Operational` trong Windows Event Viewer.



Hình 4.4: Kênh sự kiện `Microsoft-Windows-Sysmon/Operational` trong Windows Event Viewer.

Trên guest, tập lệnh `start_collector.ps1` đọc kênh Sysmon và ghi từng sự kiện thô ra tệp `sysmon_events.jsonl` trong thư mục log của phiên; lớp Bridge trên host kéo tệp này về qua endpoint `/read` của Agent (Mục 4.3.3) để đưa vào bước chuẩn hóa. Hình 4.5 trình bày nội dung thô của tệp `sysmon_events.jsonl` sau khi kéo về host.



Hình 4.5: Các sự kiện Sysmon thô trong tệp `sysmon_events.jsonl` sau khi kéo từ guest về host.

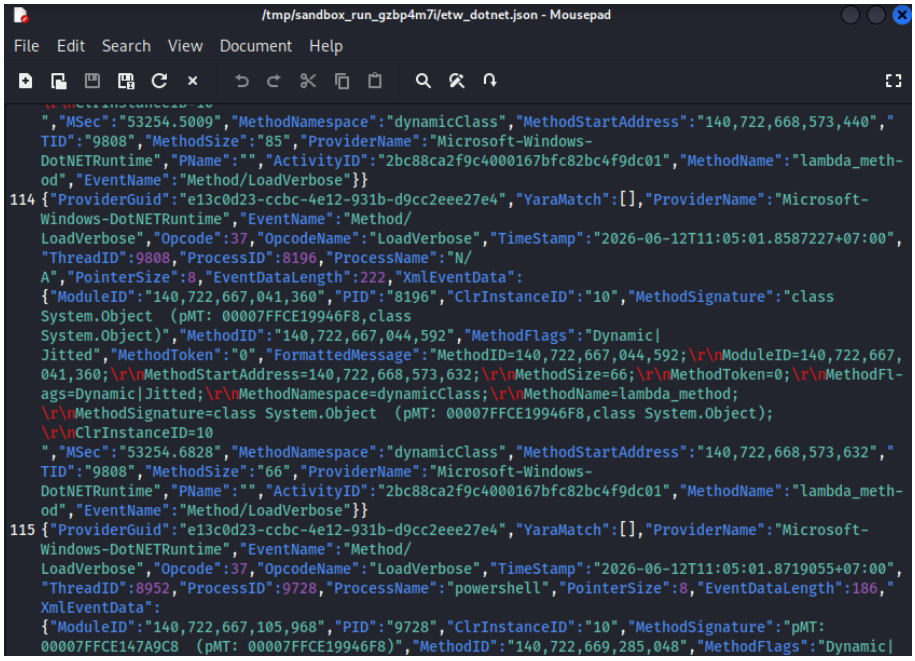
4.3.2 Cấu hình SilkETW

SilkETW v8 cũng được nhúng sẵn trong snapshot. Provider `Microsoft-Windows-DotNETRuntime` với bộ lọc `Method/LoadVerbose` bổ sung khả năng quan sát hoạt động nạp assembly .NET mà Sysmon không ghi nhận ở mức hệ điều hành; thông tin namespace và phương thức thu được phục vụ nhận diện assembly đáng ngờ ở bước chuẩn hóa. Bảng 4.5 liệt kê các tham số cấu hình.

Bảng 4.5: Tham số cấu hình SilkETW trong mỗi phiên phân tích.

Tham số	Giá trị
Chế độ thu thập	User-mode (`-t user`)
Provider	`Microsoft-Windows-DotNETRuntime`
Bộ lọc sự kiện	Tên sự kiện = `Method/LoadVerbose`
Kiểu đầu ra	Tệp JSON (`-ot file`)
Tệp đầu ra	`etw_dotnet.json`
Thời gian chạy	Bằng cửa sổ quan sát (mặc định 120 giây)

Cả hai tác vụ được khởi động đồng thời bởi cùng một script `start_collector.ps1` trên guest; SilkETW dừng tự động khi hết cửa sổ quan sát và ghi các sự kiện thô ra tệp `etw_dotnet.json`. Tương tự tệp Sysmon, tệp này được lớp Bridge kéo từ guest về host qua endpoint `/read` của Agent (Mục 4.3.3) để đưa vào bước chuẩn hóa. Hình 4.6 cho thấy nội dung thô của tệp `etw_dotnet.json` sau một phiên quan sát.

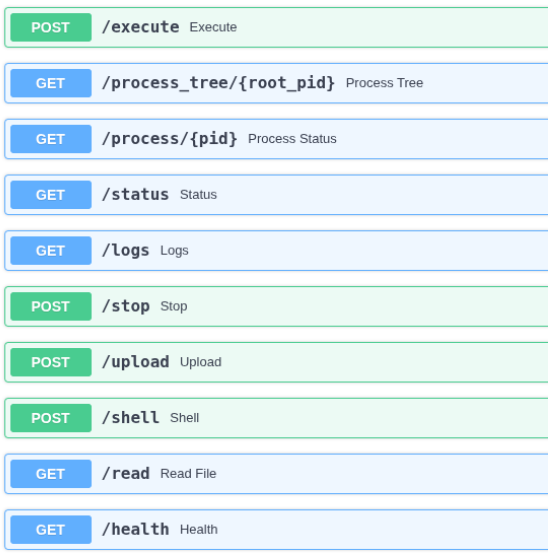


Hình 4.6: Các sự kiện ETW thô về nạp assembly .NET trong tệp `etw_dotnet.json` sau khi kéo từ guest về host.

4.3.3 Agent giám sát trong VM và giao tiếp với host

Agent là ứng dụng HTTP nhẹ xây dựng trên FastAPI, lắng nghe tại cổng `8001`, được khởi động tự động qua Task Scheduler khi hệ điều hành khởi động mà không yêu cầu đăng nhập. Agent đóng vai trò lớp trung gian giữa host và các thành phần thu thập bên trong VM, cho phép host điều khiển phiên phân tích qua API HTTP mà không tương tác trực tiếp với giao diện Windows. HTTP REST được chọn thay cho WinRM hay PsExec vì không yêu cầu cấu hình dịch vụ hệ thống phức tạp và dễ mở rộng thêm endpoint.

Hình 4.7 cho thấy giao diện tài liệu API tự động (Swagger UI) của Agent.



Hình 4.7: Giao diện tài liệu API tự động (Swagger UI) của Agent tại cổng 8001.

Toàn bộ 11 endpoint được liệt kê tại Phụ lục B.5 (Bảng B.8). Phía host, lớp Bridge điều phối phiên: xác nhận Agent sẵn sàng, lấy PID mẫu, tải tập lệnh thu thập lên guest và khởi chạy; khi phiên kết thúc, các tệp nhật ký được kéo về host phục vụ chuẩn hóa.

4.3.4 Chuẩn hóa và lọc nhiễu

Dữ liệu thô từ Sysmon và SilkETW được xử lý qua hai bộ chuẩn hóa độc lập trước khi hợp nhất.

Bộ chuẩn hóa Sysmon ánh xạ mỗi Event ID sang tên hành vi tương ứng, trích xuất các trường quan trọng và lọc nhiễu hạ tầng (Sysmon, Windows Defender, VirtualBox). Việc lọc được áp dụng theo từng loại sự kiện thay vì cắt bỏ đồng loạt, nhằm giảm nhiễu mà không làm mất hành vi thật: sự kiện tạo tiến trình (EID 1) luôn được giữ để bảo toàn cây tiến trình; kết nối DNS/NTP nền của `svchost.exe` (EID 3) bị loại nhưng tiến trình này vẫn được giữ trong các trường hợp khác vì là đích tiêm mã phổ biến; thư viện hệ thống đã ký (EID 7) bị lọc thêm vì chiếm phần lớn lượng sự kiện nạp DLL. Hình 4.8 minh họa kết quả chuẩn hóa một sự kiện Sysmon EID 1 từ phiên phân tích mẫu AsyncRAT.

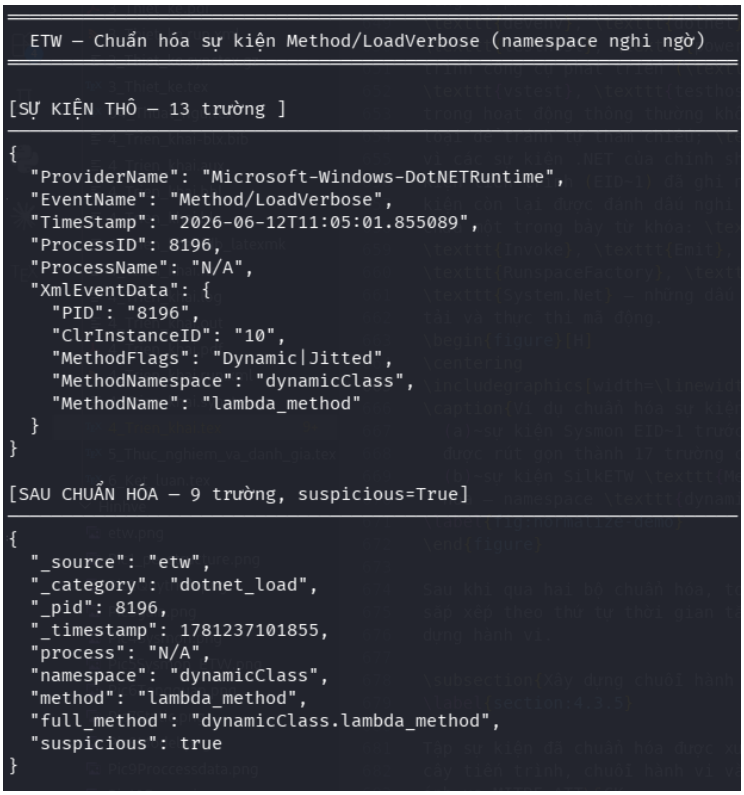
```
═══════════════════════════════════════════════════════════
  SYSMON — Chuẩn hóa sự kiện EID 1 (Process Create)
═══════════════════════════════════════════════════════════

[SỰ KIỆN THÔ — 23 trường ]
─────────────────────────────────────────────────────────
{
  "eid": 1,
  "time_utc": "2026-06-12T04:03:55.804Z",
  "props": {
    "ProcessId": "8856",
    "Image": "C:\\Users\\bach\\Downloads\\malware-samples\\AsyncRAT__08e41de61e3905ec8…",
    "CommandLine": "C:\\Users\\bach\\Downloads\\malware-samples\\AsyncRAT__08e41de61e3905ec8…",
    "User": "WIN10\\bach",
    "IntegrityLevel": "High",
    "Hashes": "MD5=7191F2FDD2E2DCB928E0FF6A1CD1E622,SHA256=08E41DE61E3905EC85A0054…",
    "ParentProcessId": "9568",
    "ParentImage": "C:\\Users\\bach\\AppData\\Local\\Programs\\Python\\Python311\\python.exe"
  }
}

[SAU CHUẨN HÓA — 17 trường]
─────────────────────────────────────────────────────────
{
  "_source": "sysmon",
  "_eid": 1,
  "_category": "process_create",
  "_pid": 8856,
  "_timestamp": 1781237035804,
  "image": "C:\\Users\\bach\\Downloads\\malware-samples\\AsyncRAT__08e41de61e3905ec8…",
  "cmdline": "C:\\Users\\bach\\Downloads\\malware-samples\\AsyncRAT__08e41de61e3905ec8…",
  "parent_image": "C:\\Users\\bach\\AppData\\Local\\Programs\\Python\\Python311\\python.exe",
  "parent_cmdline": "\"C:\\Users\\bach\\AppData\\Local\\Programs\\Python\\Python311\\python.exe\" …",
  "parent_pid": 9568,
  "user": "WIN10\\bach",
  "integrity": "High",
  "cwd": "C:\\Users\\bach\\Downloads\\sandbox\\",
  "original_filename": "ejixwomms.exe",
  "md5": "7191f2fdd2e2dcb928e0ff6a1cd1e622",
  "sha256": "08e41de61e3905ec85a0054033b69a51e9bd4ee9e0e565bf57c82e27821ff468",
  "imphash": "f34d5f2d4577ed6d9ceec516c1f5a744"
}
```

Hình 4.8: Sự kiện Sysmon EID 1 trước và sau chuẩn hóa.

Bộ chuẩn hóa SilkETW chỉ giữ lại sự kiện `LoadVerbose` và `AssemblyLoad` phản ánh hoạt động nạp assembly .NET, đồng thời loại bỏ sự kiện từ tám tiến trình hạ tầng phát triển, công cụ thu thập và trình thông dịch shell để tránh nhiễu (Phụ lục B.4, Bảng B.5), rồi trích xuất namespace và phương thức được nạp. Namespace là căn cứ để bước ánh xạ MITRE (Mục 4.3.5) nhận diện kỹ thuật nạp mã phản chiếu trong bộ nhớ (T1620) — lớp hành vi mà Sysmon ở mức hệ điều hành không quan sát được; bảy từ khóa namespace phản ánh nạp và thực thi mã động dùng để khoanh vùng các sự kiện .NET đáng chú ý được liệt kê tại Phụ lục B.4 (Bảng B.6). Hình 4.9 cho thấy kết quả chuẩn hóa tương ứng cho một sự kiện SilkETW `LoadVerbose`.



Hình 4.9: Sự kiện SilkETW `Method/LoadVerbose` trước và sau chuẩn hóa.

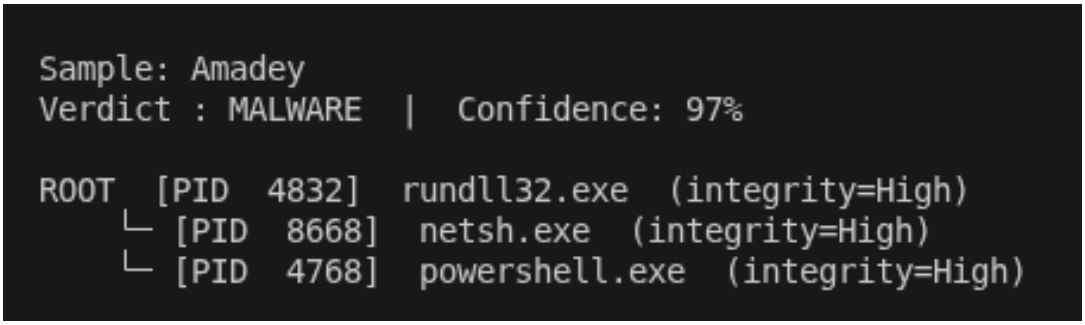
Sau khi qua hai bộ chuẩn hóa, sự kiện được hợp nhất và sắp xếp theo thứ tự thời gian, làm đầu vào cho bước xây dựng hành vi.

4.3.5 Xây dựng chuỗi hành vi, IOC và ánh xạ MITRE ATT&CK

Tập sự kiện đã chuẩn hóa được xử lý thành ba thành phần biểu diễn hành vi — cây tiến trình, chuỗi hành vi và tập IOC — rồi từ đó ánh xạ sang danh sách kỹ thuật MITRE ATT&CK.

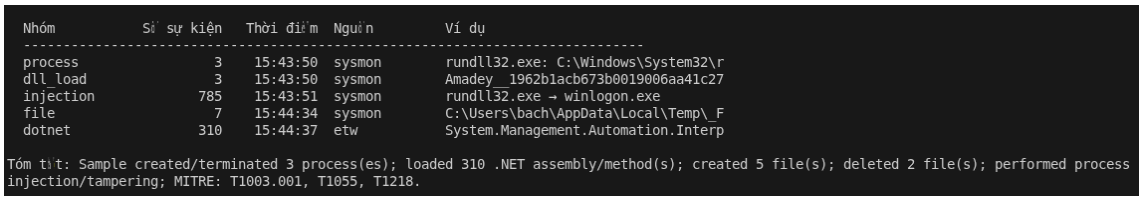
Cây tiến trình. Cây được xây dựng từ sự kiện tạo tiến trình (EID 1); mỗi nút lưu PID, tên tệp thực thi, dòng lệnh, PID cha và dấu thời gian. 27 tiến trình hệ thống thường trực được lọc theo bốn nhóm nhiễu nền (Phụ lục B.4, Bảng B.7), nhưng chỉ ẩn khỏi cây khi toàn bộ nhánh bên dưới cũng là nhiễu — ngược lại nút cha được

giữ để bảo toàn ngữ cảnh cha–con. Hình 4.10 minh họa cây tiến trình của một mẫu phân tích sau khi lọc nhiễu.



Hình 4.10: Ví dụ cây tiến trình sau khi lọc nhiễu của một mẫu phân tích.

Chuỗi hành vi. 24 loại sự kiện chuẩn hóa được gom vào 11 nhóm hành vi theo bảng ánh xạ cố định tại Phụ lục B.6 (Bảng B.9). Mỗi nhóm ghi lại thời điểm xuất hiện đầu tiên, tổng số sự kiện, và tối đa ba sự kiện tiêu biểu được kết xuất thành chuỗi dễ đọc (tên tiến trình kèm dòng lệnh, địa chỉ đích, hoặc cặp nguồn–đích cho sự kiện tiêm mã) để minh họa cụ thể mà không liệt kê toàn bộ. Toàn bộ chuỗi được sắp xếp theo thời gian, phản ánh diễn biến hoạt động của mẫu. Hình 4.11 là đầu ra chuỗi hành vi của một mẫu cụ thể.



Hình 4.11: Ví dụ đầu ra chuỗi hành vi của một mẫu phân tích.

Tập IOC. IOC được trích xuất trong cùng vòng duyệt sự kiện, thu vào mười danh mục: địa chỉ IP đích, tên miền, URL, tệp được tạo, tệp bị xóa, khóa registry, câu lệnh thực thi, named pipe, PID tiến trình bị tiêm mã và giá trị băm SHA-256. Địa chỉ IP chỉ ghi nhận khi là IP công khai; tên miền loại trừ dịch vụ hệ thống phổ biến và được kiểm tra dấu hiệu DGA theo entropy Shannon; URL được trích xuất bằng biểu thức chính quy từ nội dung sự kiện. Sau khi xây dựng xong chuỗi và IOC, hệ thống tổng hợp một đoạn tóm tắt định tính về hoạt động của mẫu trong phiên quan sát.

Ánh xạ MITRE ATT&CK. Hệ thống ánh xạ ATT&CK theo 23 kỹ thuật xác định trước; khi điều kiện thỏa mãn, kỹ thuật tương ứng được ghi nhận kèm nguồn tín hiệu (Sysmon, SilkETW hoặc cả hai) và mức tin cậy, đưa vào báo cáo để người phân tích cân nhắc độ tin của từng dấu hiệu. Các luật phủ ba nhóm chính: tiêm mã và leo thang đặc quyền, cố thủ qua registry và WMI, phát hiện kênh C2 và mạng. Bốn kỹ thuật (T1071.001, T1568.002, T1566.001 và T1070.007) được đánh mức

tin cậy trung bình do tín hiệu kích hoạt có thể xuất hiện cả trong hoạt động hợp pháp; các kỹ thuật còn lại được đánh mức cao. Toàn bộ 23 kỹ thuật được liệt kê tại Phụ lục B.2 (Bảng B.2). Hình 4.12 biểu diễn ví dụ kết quả ánh xạ của một phiên phân tích thực tế.

```
Kỹ thuật      Tên kỹ thuật
-------------------------------------------------------------
T1003.001     OS Credential Dumping: LSASS Memory
T1055         Process Injection
T1218         System Binary Proxy Execution (LOLBin)
```

Hình 4.12: Ví dụ kết quả ánh xạ MITRE ATT&CK của một mẫu phân tích.

4.4 Triển khai pipeline tích hợp

Theo thiết kế tại Mục 3.5, pipeline tích hợp phối hợp hai giai đoạn phân tích tĩnh và động để đưa ra kết luận cuối cùng và xuất báo cáo. Triển khai tập trung vào ba khía cạnh: cơ chế thực thi song song, quy tắc kết hợp kết quả và cấu trúc báo cáo đầu ra.

4.4.1 Luồng xử lý song song

Theo thiết kế tại Mục 3.5.1, khi nhận tệp PE đầu vào, pipeline khởi động đồng thời hai luồng qua `ThreadPoolExecutor(max_workers=2)`. Luồng phân tích tĩnh (Stage 1) trích xuất vector đặc trưng từ cấu trúc PE, rút gọn xuống 384 chiều theo bộ chỉ số đã lưu và chạy mô hình LightGBM, trả về nhãn phân loại cùng xác suất malware. Toàn bộ bước này chỉ đọc cấu trúc của tệp trên host và không thực thi mẫu, nên hoàn tất trong vài giây và độc lập hoàn toàn với máy ảo; nhờ đó kết quả tĩnh luôn có sẵn, kể cả với mẫu từ chối chạy hoặc cố phát hiện sandbox để không bộc lộ hành vi.

Ngược lại, luồng phân tích động phải thực thi mẫu thật trong máy ảo cách ly và quan sát đến hết cửa sổ thời gian định trước, nên chậm hơn và phụ thuộc vào guest; bù lại, nó bộc lộ được các hành vi lúc chạy — tạo tiến trình, tiêm mã, nạp assembly .NET trong bộ nhớ — vốn không hiện ra khi chỉ phân tích cấu trúc tệp. Luồng này lần lượt qua ba nhóm việc: chuẩn bị môi trường (Stage 2–4: khôi phục snapshot, xác nhận Agent, tải mẫu), thực thi và thu thập (Stage 5–8: ghi mốc Sysmon baseline, chạy mẫu, khởi động bộ thu thập, chờ hết cửa sổ quan sát và kéo tệp về host), và xử lý (Stage 9–11: chuẩn hóa sự kiện, xây dựng hành vi và IOC, đóng gói bằng chứng). Hai luồng không phụ thuộc nhau; cả hai hoàn tất mới chuyển sang Stage 12 tổng hợp kết quả.

Mỗi bước được bọc riêng trong khối xử lý ngoại lệ; khi lỗi xảy ra, báo cáo vẫn được xuất với kết quả tĩnh đã có, đáp ứng yêu cầu NF4. Hình 4.13 ghi lại log thực

thi của pipeline trong một phiên phân tích thực tế.

```
============================================================
Sample : Amadey__1962b1acb673...exe
SHA256 : 1962b1acb673b0019006aa41c27fd05b25c978625a0e4851632c1cd93f2840f4
============================================================
[Stage 1] Static analysis...
[Stage 1] label=malware prob=0.733
[Stage 2] Restoring VM snapshot 'Clean_Base'...
[Stage 2] VM ready
[Stage 3] Waiting for agent...
[Stage 4] Uploading sample...
[Stage 4] → C:\Users\bach\Downloads\malware-samples\Amadey__...exe
[Stage 5] Recording Sysmon baseline...
[Stage 5] baseline RecordId=1356870
[Stage 6] Executing sample...
[Stage 6] pid=-1
[Stage 7] Starting collector (duration=90s pid=-1 baseline=1356870)...
[Stage 7] run_id=20260614_141551
[Stage 8] Collecting events...
Sysmon poll ended — 1114 lines
[Stage 8] → /tmp/sandbox_run_8fpsqwag
[Stage 9] Loading events...
[Stage 9] sysmon=236 etw=18
[Stage 10] Building behavioral chain...
[Stage 10] ttps=3 ips=0 domains=0 groups=6
[Stage 11] Packaging dynamic evidence...
[Stage 11] ttps=3 proc_tree=3
[Stage 12] verdict=malware conf=85% ml=malware(0.733) ttps=3 injection=True
verdict=malware conf=85% | report=results/pipeline_runs/1962b1... | 150.6s
```

Hình 4.13: Log thực thi pipeline tích hợp trong một phiên phân tích.

4.4.2 Kết hợp kết quả và xác định kết luận

Sau khi cả hai luồng hoàn tất, hệ thống tổng hợp kết quả theo năm quy tắc ưu tiên được liệt kê trong Bảng 4.6. Tín hiệu tiêm mã được xác định từ hai nguồn: danh sách PID tiến trình bị tiêm trong IOC, hoặc sự xuất hiện của kỹ thuật T1055 (Process Injection) hay T1003.001 (LSASS Memory) trong tập MITRE.

Trong toàn bộ tín hiệu thu được ở giai đoạn động, chỉ tiêm mã được dùng làm căn cứ điều chỉnh kết luận, còn các hành vi khác (tạo tệp, ghi registry, kết nối mạng, ánh xạ MITRE) chỉ đóng vai trò bằng chứng diễn giải trong báo cáo. Lý do là tiêm mã có độ chính xác cao nhất: hệ thống chỉ tính là tiêm mã khi một tiến trình mở tiến trình khác với quyền *ghi* vào bộ nhớ (kèm thao tác cấp phát vùng nhớ hoặc tạo luồng), nên loại được các công cụ hợp pháp vốn chỉ *đọc* thông tin tiến trình; nhờ vậy tỉ lệ dương tính giả thấp (Mục 5.3.4). Ngược lại, tạo tệp, ghi khóa registry hay

kết nối mạng xuất hiện phổ biến cả ở phần mềm hợp pháp như bộ cài đặt, nên nếu dùng để đổi nhãn sẽ gây nhiều báo động giả. Cơ chế kết hợp vì vậy bất đối xứng: tín hiệu động chỉ có thể nâng mức nghi ngờ, không hạ một kết luận `malware` của giai đoạn tĩnh xuống `benign`.

Bảng 4.6: Quy tắc kết hợp kết quả và tính độ tin cậy tổng hợp.

Quy tắc	Kết quả ML	Tín hiệu tiêm mã	Kết luận	Độ tin cậy
1	Xác suất malware $\geq 5\%$	Có	`malware`	$\max(\text{ml_prob}, 0{,}85)$
2	Xác suất malware $< 5\%$	Có	`suspicious`	$0{,}65$
3	ML dự đoán malware	Không	`malware`	ml_prob
4	ML dự đoán benign	Không	`benign`	$1 - \text{ml_prob}$
5	Không có kết quả ML	—	`unknown`	0

Điều kiện ML ở hai cặp quy tắc được diễn đạt khác nhau vì chúng so với hai ngưỡng khác nhau. Khi phát hiện tiêm mã (quy tắc 1 và 2), hệ thống không dùng nhãn phân loại thường mà so trực tiếp xác suất malware thô với ngưỡng hạ thấp 5%: tín hiệu hành vi đã đủ mạnh nên chỉ cần xác suất tĩnh vượt 5% là kết luận `malware`. Khi không có tiêm mã (quy tắc 3 và 4), hệ thống dùng nhãn ML thường — vốn đã được quyết định theo ngưỡng triển khai $\tau = 0{,}33$ ngay trong bộ phân loại (Mục 5.2.3); khi đó "ML dự đoán malware" nghĩa là xác suất vượt 0,33. Ngưỡng override 5% thấp hơn τ chính là để thu hồi những mẫu bị giai đoạn tĩnh chấm dưới ngưỡng nhưng bộc lộ tiêm mã khi thực thi. Quy tắc 1 còn đặt sàn độ tin cậy 85% theo cùng lý do, ngay cả khi mô hình tĩnh cho xác suất thấp. Quy tắc 3 kèm thêm điều kiện: nếu phân tích động chạy thành công nhưng không thu được tín hiệu MITRE nào và không có kết nối IP ra ngoài, hệ thống ghi cảnh báo nghi ngờ mẫu lẩn tránh sandbox.

4.4.3 Xuất báo cáo đầu ra

Sau khi xác định kết luận, pipeline ghi kết quả ra tệp JSON trong thư mục `results/pipeline_runs/`. Tên tệp có dạng `{sha256[:12]}_{timestamp}.json` để dễ tra cứu theo mẫu và theo thời gian. Báo cáo tổ chức theo năm nhóm trường như thiết kế tại Mục 3.5.2; tên trường thực tế của từng nhóm được liệt kê tại Phụ lục B.7 (Bảng B.10). Hình 4.14 là ví dụ báo cáo JSON của một phiên phân tích mẫu mã độc thực tế.

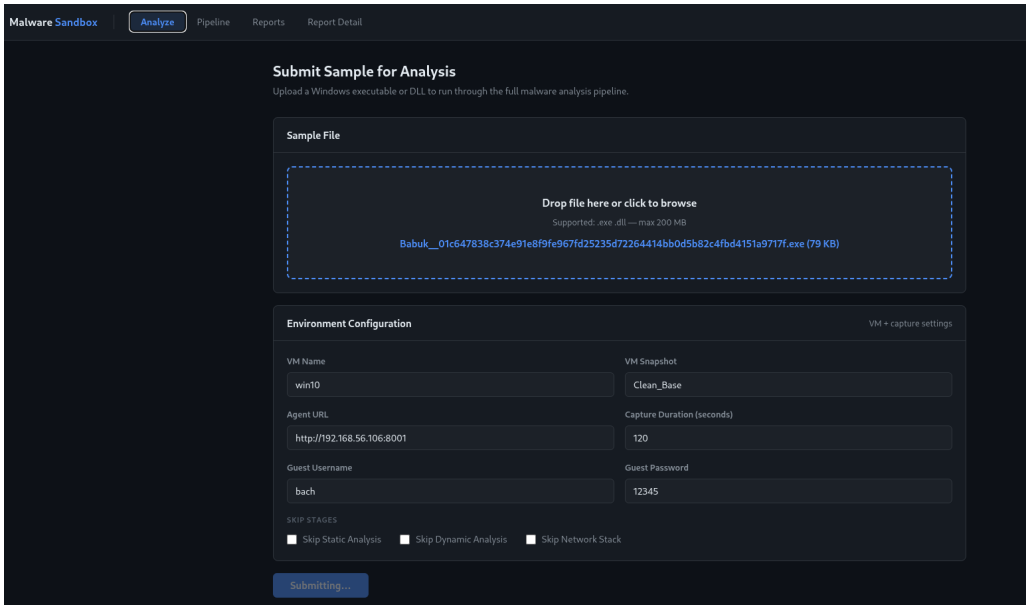
```
{
  "verdict": "malware",
  "confidence": 0.85,
  "ml_label": "malware",
  "ml_prob": 0.7327,
  "mitre_techniques": [
    "T1003.001",
    "T1055",
    "T1218"
  ],
  "narrative": "Sample created/terminated 6 process(es); loaded 18 .NET assembly/methoc
  "metadata": {
    "sha256": "1962b1acb673b0019006aa41c27fd05b25c978625a0e4851632c1cd93f2840f4",
    "file_size": 1282048,
    "duration_s": 150.6,
    "stages": [
      "static",
      "network",
      "vm_restore",
      "agent_ready",
      "upload",
      "baseline_recorded",
      "sample_executed",
      "collector_started",
      "events_collected",
      "events_loaded",
      "chain_done",
      "evidence_done",
      "verdict"
    ],
    "event_count": 254,
    "ttp_count": 3
  },
  "ioc": {
    "files_created": "...",
    "files_deleted": "...",
    "registry_keys": "...",
    "commands": "...",
    "hashes": "..."
  }
}
```


Hình 4.14: Ví dụ báo cáo JSON đầu ra của một phiên phân tích mẫu mã độc.

4.5 Giao diện web demo

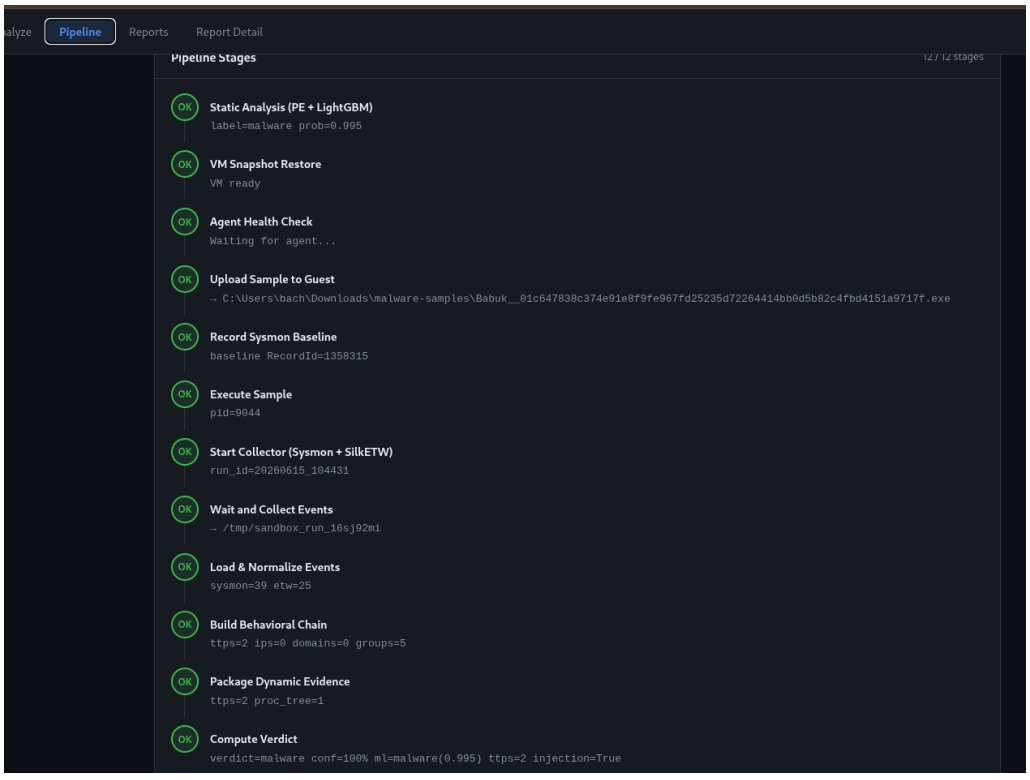
Server Flask lắng nghe tại cổng `5001`, cung cấp tám endpoint REST và phát sự kiện tiến trình phân tích qua SSE; mỗi yêu cầu phân tích được xử lý bởi một luồng nền độc lập kèm hàng đợi sự kiện riêng. Danh sách endpoint và loại sự kiện SSE được trình bày tại Phụ lục B.3 (Bảng B.3, Bảng B.4). Giao diện phía client được tổ chức thành một tệp CSS và bốn tệp JavaScript, phân theo chức năng: điều hướng, xử lý SSE, danh sách báo cáo và kết xuất chi tiết. Giao diện được tổ chức thành bốn trang chính, lần lượt trình bày dưới đây.

Trang Analyze (Hình 4.15). Người dùng tải mẫu `.exe`/`.dll` qua vùng kéo-thả rồi tinh chỉnh các thông số môi trường: máy ảo và snapshot, địa chỉ Agent, thời lượng quan sát (mặc định 120 giây) và thông tin đăng nhập guest. Ba tùy chọn bật/tắt cho phép bỏ qua phân tích tĩnh, động hoặc tầng mạng để chạy riêng từng phần. Nhấn “Run Analysis” để khởi chạy.



Hình 4.15: Trang Analyze: vùng kéo-thả và biểu mẫu cấu hình môi trường phân tích.

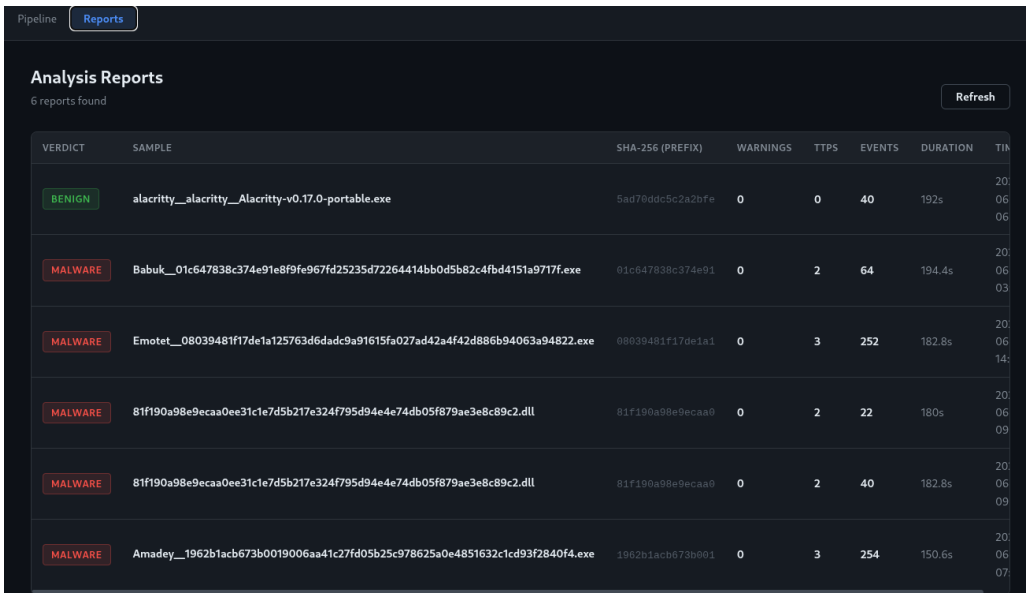
Trang Pipeline (Hình 4.16). Giao diện cập nhật 12 giai đoạn qua luồng SSE và xuất báo cáo tự động khi phân tích hoàn tất; nút “Stop Pipeline” cho phép dừng phiên giữa chừng.



Hình 4.16: Trang Pipeline: 12 giai đoạn phân tích.

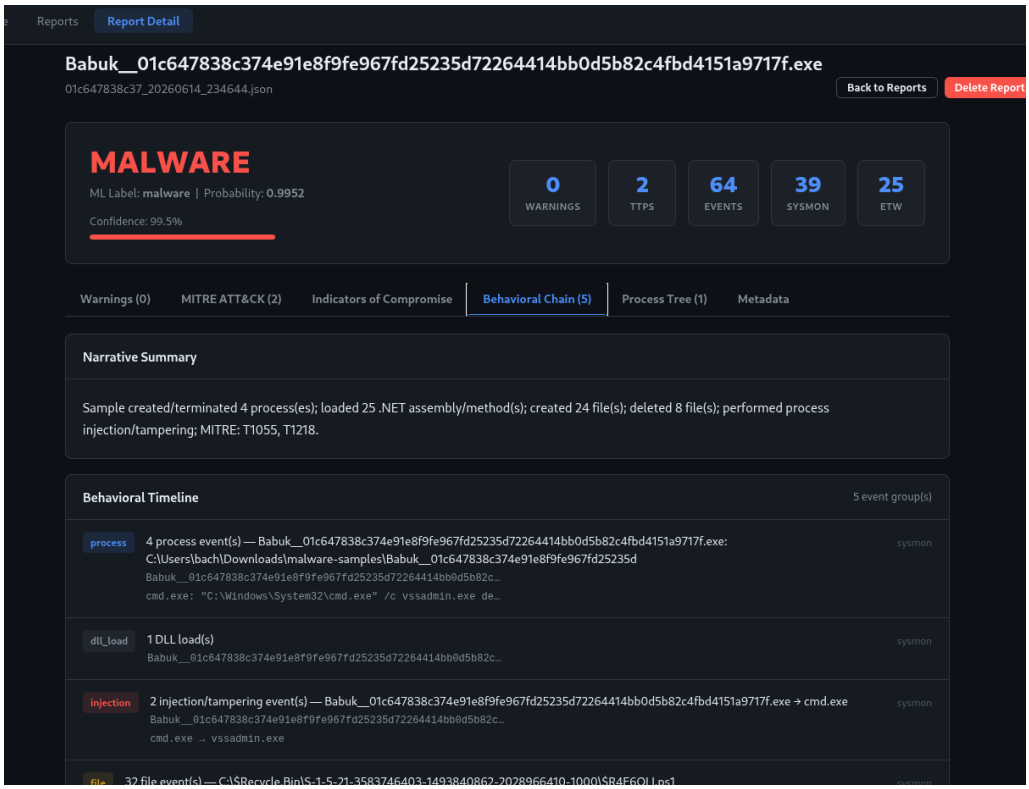
Trang Reports (Hình 4.17). Bảng liệt kê các phiên phân tích đã lưu, sắp xếp theo thời gian giảm dần; mỗi hàng hiển thị verdict, tên mẫu, tiền tố SHA-256, số cảnh báo/lỗi, số kỹ thuật MITRE, số sự kiện, thời lượng và thời điểm phân tích. Nhấn

vào một hàng để chuyển sang trang báo cáo chi tiết.



Hình 4.17: Trang Reports: bảng tổng hợp kết quả các phiên phân tích đã lưu.

Trang Report Detail (Hình 4.18). Trang trình bày kết quả phân tích chi tiết của một phiên, gồm phần kết luận (verdict, nhãn ML, độ tin cậy) và các tab dữ liệu: Warnings, MITRE ATT&CK, Indicators of Compromise, Behavioral Chain và Metadata; tab Process Tree chỉ xuất hiện khi phiên thu được dữ liệu cây tiến trình.



Hình 4.18: Trang Report Detail: báo cáo chi tiết 1 mẫu phân tích.

Kết quả triển khai là một pipeline vận hành tự động hoàn toàn từ khâu nhận tệp PE đến khâu xuất báo cáo JSON: giai đoạn tĩnh chạy trên host song song với giai đoạn động trong máy ảo, hai luồng sự kiện Sysmon và ETW được chuẩn hóa rồi hợp nhất, và kết quả được hiển thị qua giao diện web. Hiệu năng và độ chính xác của hệ thống được đánh giá ở Chương 5.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này đánh giá hệ thống trên hai giai đoạn: giai đoạn tĩnh đo hiệu năng phân loại và phân tích sâu mô hình, giai đoạn động kiểm chứng khả năng thu thập bằng chứng hành vi.

5.1 Thiết lập thực nghiệm

Quá trình đánh giá được tổ chức thành hai phần tương ứng hai giai đoạn của hệ thống. Phần tĩnh sử dụng ba tập dữ liệu độc lập, thống kê trong Bảng 5.1.

Bảng 5.1: Ba tập dữ liệu dùng trong đánh giá giai đoạn phân tích tĩnh.

Tập dữ liệu	Số mẫu	Mục đích
Holdout nội bộ	40.328	So sánh ba thuật toán trong cùng điều kiện (tách phân tầng 20% tập huấn luyện, seed 42)
Mẫu kiểm thử riêng	1.189	Đánh giá theo bảy nhóm họ trên mẫu độc lập, không trùng tập huấn luyện (675 mã độc từ MalwareBazaar [28], 514 lành tính)
EMBER 2024 [29]	119.992	Đánh giá tổng quát hóa trên benchmark công khai, khác nguồn thu thập và ngoài phân phối (gần cân bằng)

Các độ đo gồm Accuracy, Precision, Recall, F1-Score, AUC-ROC và AUC-PR. Do bài toán nhạy cảm với cả hai loại sai sót, hai chỉ số FPR và FNR được theo dõi riêng. Nhãn 1 (mã độc) được quy ước là lớp dương cần phát hiện, nhãn 0 là lành tính.

Phần động được đánh giá trên 305 mẫu. Tập mã độc gồm 205 mẫu thu từ MalwareBazaar [28], phân bố trên bảy nhóm hành vi (dropper, ransomware, trojan, RAT, worm, cryptominer, spyware) thuộc 36 họ; tập đối chứng gồm 100 ứng dụng mã nguồn mở hợp pháp. Mỗi mẫu được thực thi trong cửa sổ quan sát 120 giây với cấu hình phần cứng tại Mục 4.1.

5.2 Đánh giá giai đoạn phân tích tĩnh

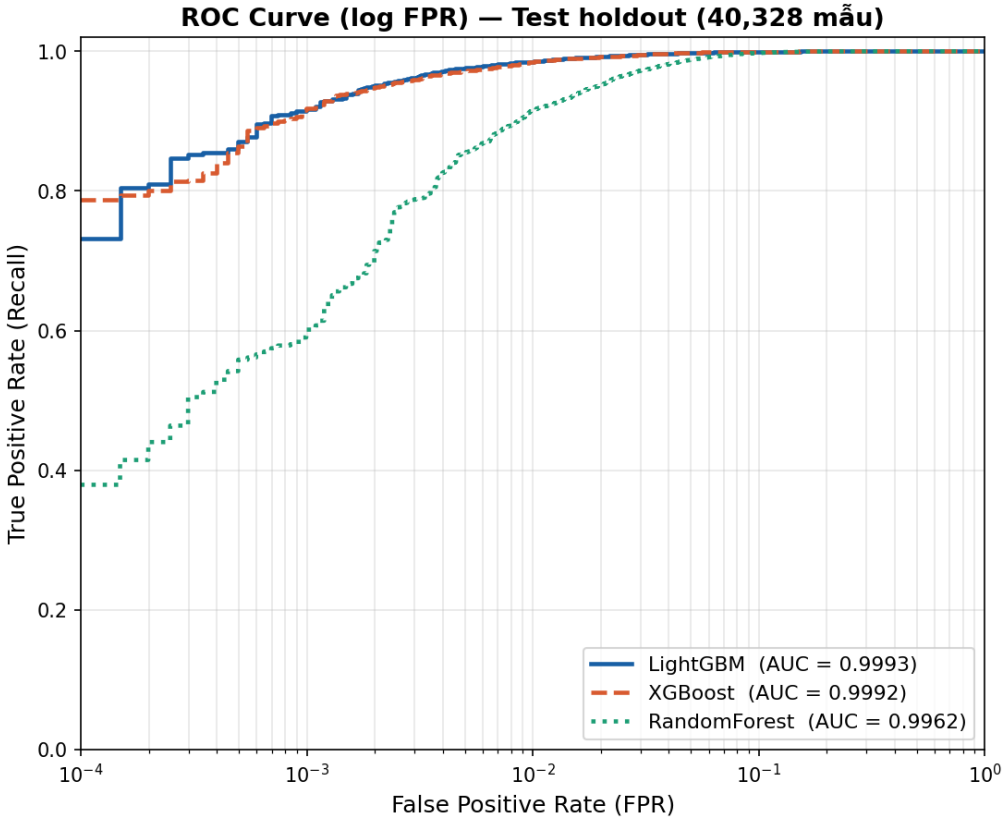
5.2.1 So sánh ba thuật toán

Ba thuật toán Random Forest, XGBoost và LightGBM được huấn luyện trên toàn bộ 2.381 đặc trưng với cùng tập huấn luyện, sau đó đánh giá trên tập holdout 40.328 mẫu (Bảng 5.2).

Bảng 5.2: So sánh ba thuật toán trên tập holdout 40.328 mẫu (toàn bộ 2.381 đặc trưng).

Thuật toán	Accuracy	F1	AUC-ROC	FPR	FNR
Random Forest	0,9704	0,9709	0,9962	0,0429	0,0163
XGBoost	0,9875	0,9875	0,9992	**0,0144**	0,0107
LightGBM	**0,9876**	**0,9876**	**0,9993**	0,0155	**0,0094**

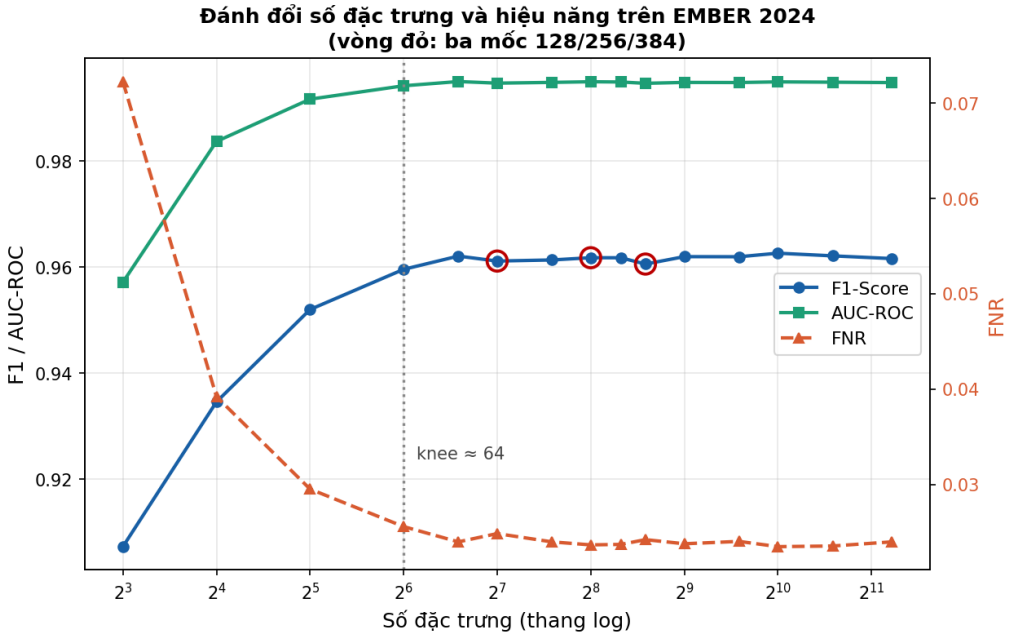
Hai mô hình boosting đạt kết quả cao hơn Random Forest trên mọi chỉ số, với AUC-ROC lần lượt là 0,9993 và 0,9992 so với 0,9962. Giữa hai mô hình boosting, LightGBM đạt F1 và AUC-ROC cao hơn, đồng thời có FNR thấp nhất là 0,0094. Chỉ số FNR được ưu tiên vì bỏ sót mã độc gây hậu quả nặng hơn báo động giả. Đường cong ROC (Hình 5.1) cho thấy hai mô hình boosting sát nhau ở vùng FPR thấp và nằm trên Random Forest. LightGBM được chọn cho các bước tiếp theo.



Hình 5.1: Đường cong ROC của ba thuật toán trên tập holdout, trục FPR theo thang log.

5.2.2 Lựa chọn tập con đặc trưng

Chỉ số Importance Gain của LightGBM được dùng để xếp hạng 2.381 đặc trưng. Để định vị số chiều cần thiết, hiệu năng trên EMBER 2024 được khảo sát nhanh qua mười lăm mốc từ 8 đến 2.381 đặc trưng, mỗi mốc huấn luyện một lần (Hình 5.2). Đường cong bão hòa từ khoảng 64 đặc trưng: chỉ 96 chiều đã đạt trên 99,9% F1 của mô hình đầy đủ, sau đó khác biệt giữa các mốc nằm trong nhiễu thống kê ($\pm$0,003). Cả ba ngưỡng 128, 256 và 384 đều nằm trên vùng bão hòa này.



Hình 5.2: Đánh đổi số đặc trưng và hiệu năng trên EMBER 2024 qua mười lăm mốc; vòng đỏ đánh dấu ba ngưỡng 128/256/384, đường chấm dọc là điểm gãy (knee) tại 64 đặc trưng.

Ba ngưỡng được huấn luyện lại đầy đủ bằng 5-fold cross-validation và so sánh với mô hình 2.381 chiều (Bảng 5.3, 119.992 mẫu EMBER 2024).

Bảng 5.3: So sánh bốn cấu hình đặc trưng trên tập EMBER 2024 (119.992 mẫu).

Cấu hình	Số chiều	Accuracy	F1	AUC-ROC	FNR
Full	2.381	0,9613	0,9618	0,9948	0,0241
Top-128	128	0,9608	0,9614	0,9947	0,0247
Top-256	256	0,9603	0,9609	0,9946	0,0249
Top-384	384	0,9610	0,9615	0,9948	0,0241

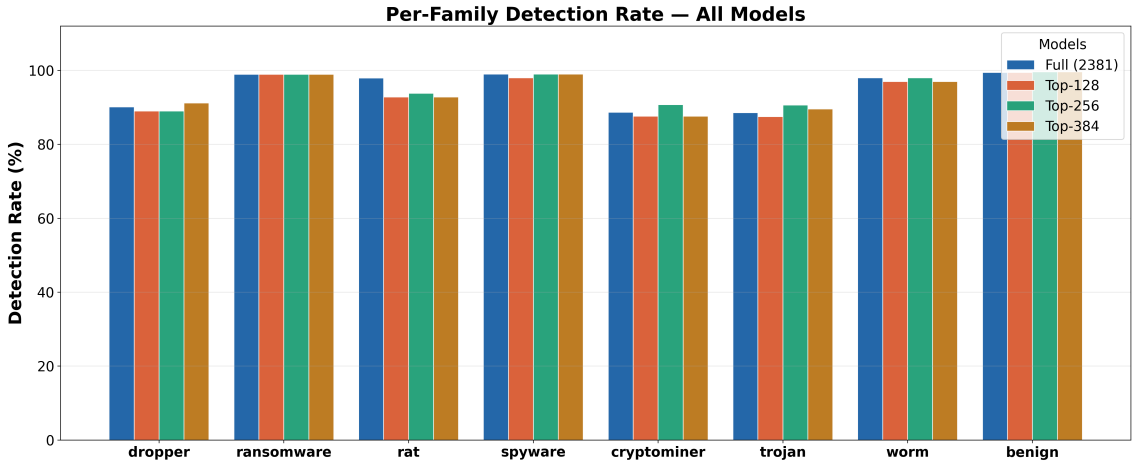
Bốn cấu hình cho hiệu năng tương đương: chênh lệch F1 giữa Top-384 và Full chỉ 0,0003, AUC-ROC bằng nhau ở 0,9948 dù số chiều giảm hơn sáu lần. So với holdout nội bộ, F1 giảm từ 0,9876 xuống 0,9615, phản ánh độ khó khi tổng quát hóa sang dữ liệu khác nguồn, ngoài phân phối. Việc giảm chiều không làm mô hình nhẹ hay nhanh hơn, vì kích thước và độ trễ phụ thuộc số cây chứ không phải số chiều; lợi ích là tập đặc trưng gọn và dễ diễn giải hơn. Trong ba cấu hình rút gọn, Top-384 đạt F1 và AUC-ROC cao nhất, đồng thời FNR thấp nhất 0,0241 bằng đúng mô hình đầy đủ; do bỏ sót mã độc là sai sót cần ưu tiên giảm, Top-384 được chọn triển khai.

Tỉ lệ phát hiện theo nhóm của mô hình Top-384 trên tập kiểm thử riêng được trình bày trong Bảng 5.4 và Hình 5.3.

Bảng 5.4: Tỉ lệ phát hiện theo nhóm của mô hình Top-384 (tập kiểm thử riêng).

Nhóm	Tỉ lệ	Nhóm	Tỉ lệ
Lành tính (benign)	99,6%	RAT	92,8%
Ransomware	98,9%	Trojan	89,6%
Worm	97,0%	Cryptominer	87,6%
Spyware	99,0%	Dropper	91,2%

Mô hình đạt tỉ lệ phát hiện quanh 99% trên nhóm lành tính, ransomware và spyware, nhưng thấp hơn ở cryptominer (87,6%) và trojan (89,6%). Hai nhóm này thường tái sử dụng cấu trúc PE gần với phần mềm hợp pháp, làm giảm khả năng phân biệt của đặc trưng tĩnh và là lý do cần bổ sung giai đoạn phân tích động.



Hình 5.3: Tỉ lệ phát hiện theo nhóm của bốn cấu hình đặc trưng.

Kết quả này phù hợp với các công trình trên dòng đặc trưng EMBER. Anderson và Roth đạt AUC-ROC 0,9991 trên EMBER-2018 [17]; Choudhary và cộng sự đạt 97,52% độ chính xác khi rút gọn còn 384 chiều [18] — cả hai đánh giá trên dữ liệu cùng thời kỳ huấn luyện. Mô hình Top-384 của đồ án đạt AUC-ROC 0,9948 và 96,10% độ chính xác trên EMBER 2024, bộ dữ liệu mới hơn và khó hơn do nằm ngoài phân phối. Mức chênh nhỏ trên tập khó hơn cho thấy mô hình tổng quát hóa tốt.

5.2.3 Tối ưu ngưỡng phân loại

Ba mục dưới đây phân tích sâu mô hình Top-384 đã chọn triển khai, nhằm phản ánh đúng hành vi của mô hình vận hành.

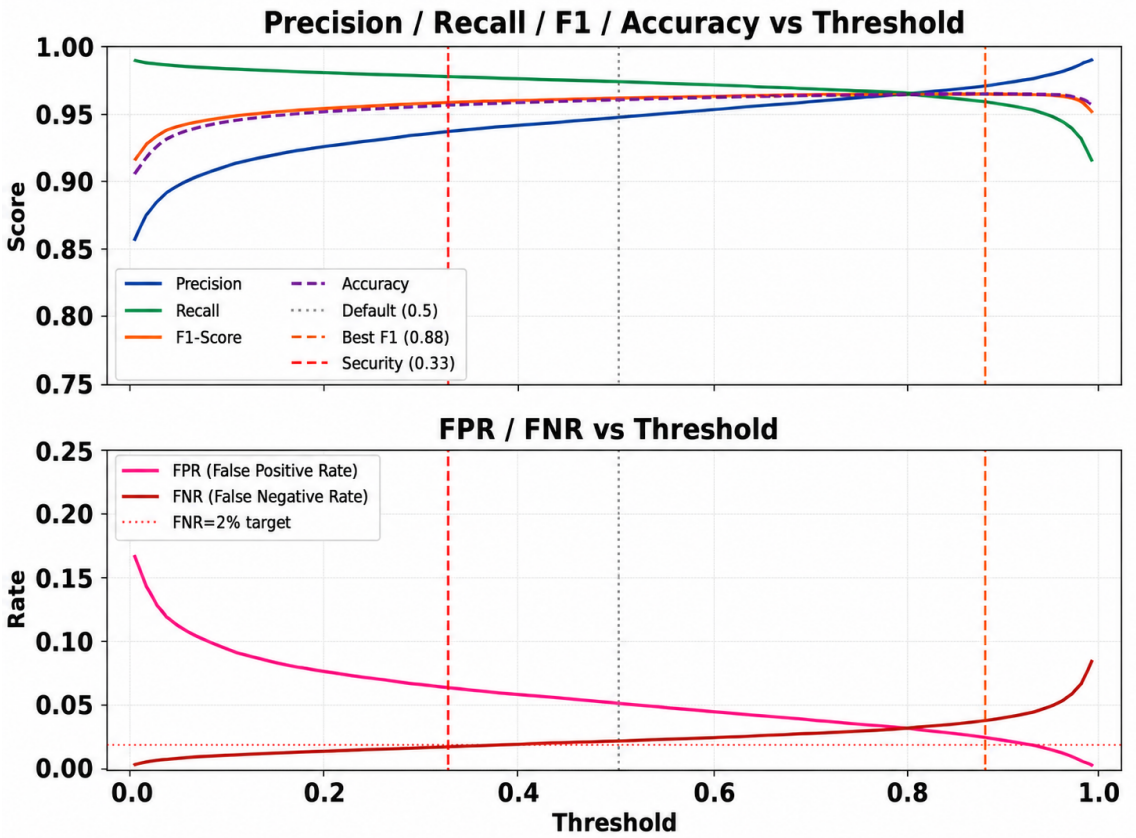
Ngưỡng quyết định τ điều tiết đánh đổi giữa FPR và FNR. Xác suất dự đoán được quét trên 99 ngưỡng cách đều từ 0,01 đến 0,99, tại mỗi ngưỡng tính FPR, FNR và F1. Từ đường quét này chọn ba mốc tiêu biểu để so sánh: ngưỡng mặc định 0,50 theo quy ước thông dụng; ngưỡng tối ưu F1 là điểm cho F1 cao nhất (0,88); và ngưỡng ưu tiên an toàn là điểm có FPR thấp nhất trong số các ngưỡng

giữ FNR $\leq 2\%$ (0,33). Bảng 5.5 và Hình 5.4 so sánh ba mốc này trên tập EMBER 2024.

Bảng 5.5: So sánh ba mốc ngưỡng phân loại trên tập EMBER 2024.

Ngưỡng	τ	Accuracy	F1	FPR	FNR
Mặc định	0,50	0,9610	0,9615	0,0540	0,0241
Tối ưu F1	0,88	0,9660	0,9658	0,0276	0,0404
Ưu tiên an toàn	0,33	0,9573	0,9582	0,0655	0,0199

Ngưỡng 0,88 cho F1 cao nhất nhưng FNR tăng lên 0,0404, tức bỏ sót nhiều mã độc hơn. Ngưỡng 0,33 đưa FNR xuống mức thấp nhất là 0,0199, đổi lại FPR tăng lên 0,0655. Hệ thống chọn $\tau = 0{,}33$ theo định hướng ưu tiên an toàn, do bỏ sót mã độc gây hậu quả nặng hơn báo động giả. Phần báo động giả tăng thêm được giai đoạn động sàng lọc qua nhãn `suspicious` (Mục 5.3.3).

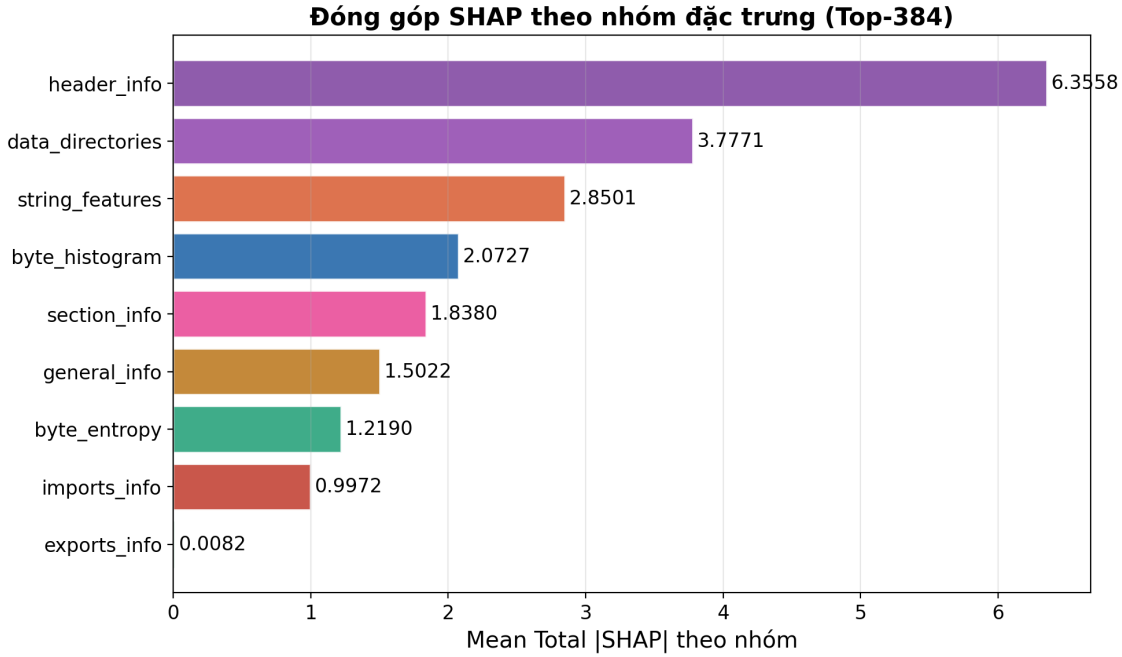


Hình 5.4: Biến thiên FPR và FNR theo ngưỡng phân loại.

5.2.4 Phân tích đặc trưng

Giá trị SHAP được tính trên mẫu con ngẫu nhiên 2.000 phần tử (seed 42) rồi tổng hợp theo chín nhóm đặc trưng EMBER, nhằm xác định nhóm ảnh hưởng nhiều nhất đến quyết định của mô hình Top-384 (Hình 5.5). Do `TreeExplainer` có chi phí lớn trên từng mẫu, việc lấy mẫu con là cần thiết; với mục tiêu đo mức đóng góp tổng thể (trung bình trị tuyệt đối SHAP theo nhóm), 2.000 mẫu đã đủ cho ước

lượng ổn định mà không phải chạy trên toàn tập.



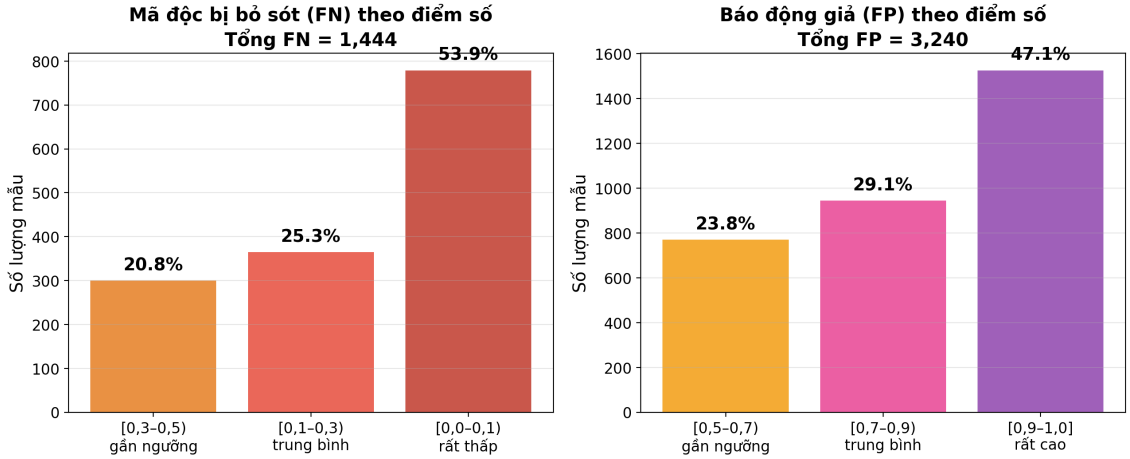
Hình 5.5: Tổng giá trị SHAP tuyệt đối theo nhóm đặc trưng.

Hai nhóm *header_info* và *data_directories* đóng góp lớn nhất, với tổng giá trị SHAP lần lượt là 6,36 và 3,78. Kết quả cho thấy mô hình dựa chủ yếu vào các trường cấu trúc trong PE header và bảng data directories, vốn phản ánh cách tệp được liên kết và đóng gói. Các nhóm *string_features*, *byte_histogram* và *section_info* đóng vai trò bổ trợ, còn *exports_info* gần như không đóng góp do phần lớn tệp không có bảng export. Việc đóng góp tập trung ở một số nhóm giải thích vì sao tập rút gọn 384 chiều duy trì được hiệu năng.

5.2.5 Phân tích lỗi

Phân tích lỗi được thực hiện tại ngưỡng mặc định 0,5, dùng làm mốc trung lập để khảo sát cấu trúc lỗi (không phải ngưỡng triển khai 0,33). Tại mốc này, trên tập EMBER 2024 mô hình Top-384 sinh 1.444 trường hợp bỏ sót (FN) và 3.240 báo động giả (FP); các mức 0,1, 0,3 hay 0,9 nêu dưới đây là khoảng điểm dự đoán, không phải ngưỡng phân loại. Phân bố hai loại lỗi theo khoảng điểm được trình bày trong Hình 5.6. Trong nhóm bỏ sót, 54% có điểm dưới 0,1, tức mô hình gán điểm rất thấp cho mẫu mã độc. Các mẫu này có cấu trúc PE gần với phần mềm lành tính, đặc trưng của kỹ thuật đóng gói và làm rối mã đã nêu tại Mục 2.2.2, nên không thể khắc phục bằng điều chỉnh ngưỡng. Phần 46% còn lại có điểm cao hơn, gần ngưỡng 0,5; trong đó các mẫu thuộc khoảng $[0{,}3; 0{,}5)$ được thu hồi khi hạ ngưỡng về 0,33, giúp phát hiện thêm 249 mã độc so với ngưỡng 0,5 — đây chính là cơ sở định lượng cho lựa chọn $\tau = 0{,}33$ ở Mục 5.2.3. Trong nhóm báo động giả, 47% có điểm trên 0,9 và rơi vào phần mềm hợp pháp có cấu trúc gần với mã độc, phần còn

lại có điểm gần ngưỡng 0,5. Cả hai loại lỗi đều xuất phát từ giới hạn của phân tích tĩnh và là cơ sở cho vai trò bổ sung của giai đoạn động.



Hình 5.6: Phân bố các trường hợp phân loại sai theo khoảng điểm số (FN bên trái, FP bên phải).

5.2.6 Hiệu chuẩn độ tin cậy

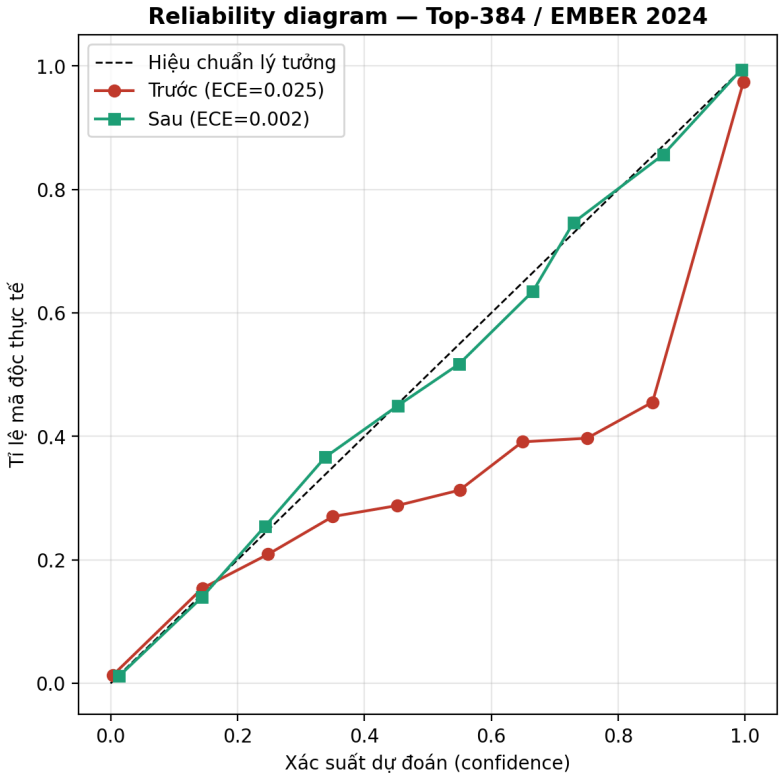
Độ chính xác phân loại chỉ phản ánh nhãn dự đoán đúng hay sai, không bảo đảm con số xác suất xuất ra phản ánh đúng tần suất thực tế. Để con số độ tin cậy hiển thị có ý nghĩa, xác suất thô của mô hình Top-384 được đối chiếu với tỉ lệ mã độc thực tế theo từng khoảng điểm trên tập EMBER 2024 (Bảng 5.6). Trong mỗi khoảng điểm, cột "điểm trung bình" là xác suất trung bình mà mô hình gán cho các mẫu thuộc khoảng đó, còn cột "tỉ lệ mã độc thực tế" là phần mẫu thực sự là mã độc theo nhãn. Mô hình được hiệu chuẩn tốt khi hai con số này xấp xỉ nhau; chênh lệch lớn nghĩa là xác suất xuất ra chưa phản ánh đúng rủi ro thật.

Bảng 5.6: Đối chiếu điểm dự đoán với tỉ lệ thực tế (trích các khoảng tiêu biểu, thống kê trên nửa tập kiểm EMBER 2024 dùng đánh giá hiệu chuẩn, khoảng 60.000 mẫu). Vùng điểm cực thấp và cực cao khớp tốt, vùng giữa tự tin thái quá.

Khoảng điểm	Số mẫu	Điểm trung bình	Tỉ lệ mã độc thực tế
0,0–0,1	27.474	0,004	0,013
0,3–0,4	322	0,350	0,270
0,7–0,8	340	0,751	0,397
0,8–0,9	505	0,854	0,455
0,9–1,0	29.515	0,997	0,975

Khoảng 95% mẫu trong nửa tập kiểm rơi vào hai vùng cực (điểm dưới 0,1 hoặc trên 0,9), nơi con số khớp tốt với thực tế. Tuy nhiên ở vùng lưỡng lự, mô hình tự tin thái quá rõ rệt: nhóm điểm 0,8–0,9 có điểm trung bình 0,854 nhưng thực tế chỉ 45,5% là mã độc. Xác suất được hiệu chuẩn bằng Isotonic Regression (học trên một nửa tập, kiểm trên nửa còn lại); sai số hiệu chuẩn kỳ vọng (ECE) giảm từ 0,0249

xuống 0,0022 (Hình 5.7). Do phép hiệu chuẩn đơn điệu, thứ tự xếp hạng các mẫu không đổi nên nhãn phân loại và các độ đo ở những mục trước được giữ nguyên; chỉ con số độ tin cậy hiển thị trở nên đáng tin.



Hình 5.7: Reliability diagram của mô hình Top-384 trước và sau hiệu chuẩn trên tập EMBER 2024.

5.3 Đánh giá giai đoạn phân tích động

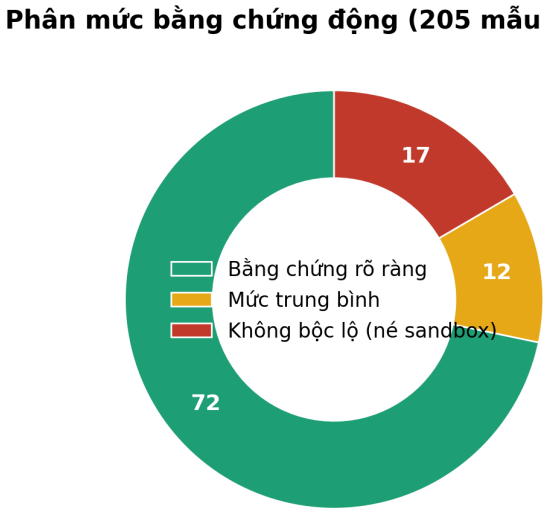
Mục tiêu đánh giá ở giai đoạn động khác với giai đoạn tĩnh: thay vì đo lại độ chính xác phân loại, phần này kiểm chứng khả năng thu thập bằng chứng hành vi của mẫu khi thực thi. Tập đánh giá gồm 205 mẫu mã độc trải trên bảy nhóm hành vi (36 họ) và 100 ứng dụng hợp pháp. Hai tập đóng vai trò khác nhau: 205 mẫu mã độc dùng đo độ phủ bằng chứng và khả năng phát hiện hành vi (Mục 5.3.1, 5.3.4), còn 100 ứng dụng hợp pháp dùng đo tỉ lệ dương tính giả (Mục 5.3.3, 5.3.4). Cả 305 phiên đều hoàn tất và sinh báo cáo, mỗi phiên ghi nhận trung bình 162 sự kiện chuẩn hóa từ Sysmon và SilkETW, gồm sự kiện tiến trình, tệp, registry, mạng và nạp assembly.

5.3.1 Độ phủ bằng chứng hành vi

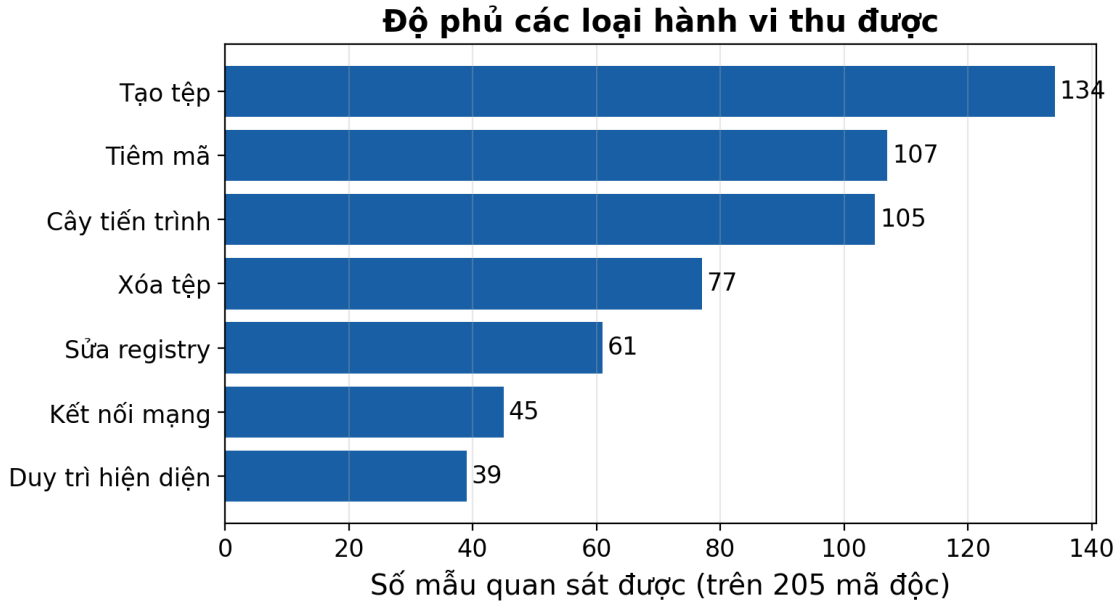
Theo lượng bằng chứng thu được, các mẫu mã độc được chia thành ba mức theo tín hiệu quan sát được (Hình 5.8). Mức rõ ràng (147 mẫu, chiếm 72%) gồm các mẫu bộc lộ ít nhất một tín hiệu có ý nghĩa: kỹ thuật ATT&CK, kết nối mạng, thao tác registry, tạo từ ba tệp trở lên, hoặc xóa tệp. Mức trung bình (24 mẫu) không đạt

mức trên nhưng vẫn sinh tiến trình con hoặc tạo một–hai tệp. Mức không bộc lộ (34 mẫu) không để lại tín hiệu nào, chủ yếu thuộc các họ có cơ chế né sandbox, phù hợp với phạm vi giới hạn tại Chương 1.

Xét độ phủ trên bảy loại hành vi quan sát được — tạo tệp, tiêm mã, cây tiến trình, xóa tệp, sửa registry, kết nối mạng và duy trì hiện diện (Hình 5.9) — thì tạo tệp, tiêm mã và cây tiến trình phổ biến nhất, lần lượt xuất hiện ở 134, 107 và 105 mẫu. Kết nối mạng và duy trì hiện diện ít gặp hơn do phụ thuộc vào máy chủ điều khiển hoặc điều kiện kích hoạt.



Hình 5.8: Phân mức bằng chứng động trên 205 mẫu mã độc.



Hình 5.9: Độ phủ các loại hành vi thu được.

5.3.2 Bằng chứng hành vi qua các mẫu đại diện

Bằng chứng động của một số mẫu mã độc tiêu biểu được trình bày trong Bảng 5.7. Mỗi dòng liệt kê chỉ số xâm phạm và kỹ thuật ATT&CK trích xuất trực tiếp từ phiên thực thi.

Bảng 5.7: Bằng chứng hành vi của các mẫu mã độc đại diện.

Mẫu	Bằng chứng động	ATT&CK
Phobos (ransomware)	Mã hóa tệp đuôi `.id[...].[email].eight`; xóa hàng loạt tệp và bản sao lưu; ghi khóa Run; tiêm mã 9 tiến trình	T1055, T1070, T1218, T1547.001
STOP (ransomware)	Kết nối C2 `api.2ip.ua`, `asvb.top`; giả mạo tiến trình (process hollowing); tự sao chép vào `AppData`; ghi khóa Run `SysHelper`	T1003.001, T1055, T1055.012, T1547.001
Babuk (ransomware)	Tắt tường lửa (`netsh advfirewall`); ghi tệp `ransom_message.txt`; tiêm mã 11 tiến trình	T1055, T1218, T1547.001
Sality (virus lây nhiễm)	Vô hiệu hóa tường lửa; tiêm mã vào khoảng 140 tiến trình; cài dịch vụ tự khởi động; truy cập LSASS	T1003.001, T1055, T1543.003
NetWire (RAT)	Lạm dụng `RegSvcs.exe` (.NET) để thực thi; thả tệp `.exe` và `.vbs`; ghi khóa Run	T1055, T1218, T1547.001

Các bằng chứng này tái hiện đúng đặc trưng của từng họ: ransomware (Phobos, STOP, Babuk) mã hóa tệp, xóa bản sao lưu hoặc tắt phòng thủ và cố thủ qua khóa Run; virus lây nhiễm (Sality) tiêm mã diện rộng vào hàng trăm tiến trình và cài dịch vụ tự khởi động; RAT (NetWire) lạm dụng tệp hệ thống .NET để thực thi rồi duy trì hiện diện. Kết quả cho thấy giai đoạn động tái hiện hành vi của mẫu ở mức có thể diễn giải và truy vết.

5.3.3 Đánh giá pipeline tích hợp

Kết luận cuối được tổng hợp từ cả hai giai đoạn. Trên 205 mẫu mã độc, hệ thống xác định đúng 204 mẫu (200 nhãn `malware`, 4 nhãn `suspicious`); mẫu bỏ sót duy nhất là một mẫu Babuk có xác suất tĩnh sát ngưỡng và không bộc lộ hành vi. Trên 100 ứng dụng hợp pháp, 83 mẫu được kết luận lành tính, 9 mẫu bị gắn nhãn `suspicious` và 8 mẫu bị quy thành `malware`. Phần lớn mẫu `malware` là các tiện ích mã hóa và gỡ lỗi bị giai đoạn tĩnh chấm điểm cao do cấu trúc gần với mã độc; các nhãn `suspicious` đến từ phần mềm hợp pháp có thao tác truy cập tiến

trình. Tỉ lệ dương tính giả theo kỹ thuật được trình bày tại Mục 5.3.4.

Một số mẫu thuộc các họ có cơ chế né tránh không bộc lộ hành vi trong sandbox nhưng vẫn được kết luận mã độc nhờ giai đoạn tĩnh, kèm cảnh báo nghi ngờ né tránh. Trường hợp này thể hiện vai trò bổ sung giữa hai giai đoạn: khi một giai đoạn không thu được tín hiệu, giai đoạn còn lại vẫn giữ được kết luận.

Quy mô đánh giá động là 305 mẫu, giới hạn bởi chi phí thực thi trong sandbox (trung bình năm phút mỗi mẫu). Độ phủ bảy nhóm hành vi cùng tỉ lệ 72% mẫu cho bằng chứng cụ thể, kết hợp với F1 $= 0{,}9615$ của giai đoạn tĩnh trên EMBER 2024, cho thấy thiết kế tĩnh–động đáp ứng mục tiêu đề ra tại Chương 1.

5.3.4 Phân tích sâu kết quả tích hợp

Mục này phân tích ba khía cạnh của bước hợp nhất: độ bền của ngưỡng quyết định, độ chính xác của ánh xạ hành vi và vai trò bù trừ giữa hai giai đoạn.

Độ bền của ngưỡng hợp nhất

Khi phát hiện tiêm mã, hệ thống hạ ngưỡng xác suất tĩnh xuống 0,05 để xác nhận kết luận (Quy tắc 1 và 2, Mục 4.4.2). Câu hỏi đặt ra là con số 0,05 này có phải lựa chọn may rủi, dễ làm đổi kết quả nếu chọn khác hay không. Để kiểm, ngưỡng được thay lần lượt bằng nhiều giá trị từ 0,01 đến 0,50 rồi quan sát kết luận trên toàn bộ 305 mẫu (Bảng 5.8): cột Recall là tỉ lệ mã độc bị bắt đúng, ba cột Benign cho biết trong 100 phần mềm sạch có bao nhiêu bị xếp thành `malware`, `suspicious` hay vẫn sạch.

Bảng 5.8: Độ nhạy của kết luận theo ngưỡng override xác suất tĩnh khi phát hiện tiêm mã (205 mã độc + 100 lành tính).

Ngưỡng override	Recall	Benign→mal	Benign→susp	Benign sạch
0,01	0,995	9	8	83
0,05	0,995	8	9	83
0,10	0,995	8	9	83
0,20	0,995	8	9	83
0,33	0,995	7	10	83
0,50	0,995	7	10	83

Recall trên mã độc giữ nguyên 0,995 với mọi ngưỡng: không mẫu nào rơi vào vùng khiến ngưỡng này thay đổi kết luận phát hiện, nên giá trị 0,05 là lựa chọn bền. Hai giá trị độ tin cậy 0,85 và 0,65 trong Bảng 4.6 là quy ước hiển thị độ tin cậy tổng hợp, không phải ngưỡng phân loại.

Độ chính xác của ánh xạ hành vi

Ánh xạ MITRE được đánh giá trên hai mặt. Mặt recall (Bảng 5.9) xét tín hiệu tiêm mã theo nhóm họ; mặt precision (Bảng 5.10) đếm số lần quy tắc kích hoạt sai

trên phần mềm hợp pháp. Recall phản ánh khả năng không bỏ sót — trong các mẫu đáng lẽ bộc lộ tiêm mã, hệ thống nhận diện được bao nhiêu phần; precision phản ánh độ tin của tín hiệu — trong các lần quy tắc kích hoạt, bao nhiêu lần là dương tính giả trên phần mềm hợp pháp. Để xét theo cơ chế tiêm mã, bảy nhóm hành vi ở Mục 5.1 được gộp về bốn nhóm họ: dropper, trojan và spyware nằm trong nhóm injector/RAT/loader/stealer, ba nhóm còn lại giữ nguyên.

Bảng 5.9: Tỉ lệ phát hiện hành vi tiêm mã theo nhóm họ; ground-truth lấy xấp xỉ ở mức taxonomy họ.

Nhóm họ (theo Bảng 2.1)	Số mẫu	Có tín hiệu tiêm mã
Injector / RAT / loader / stealer	120	66
Ransomware	29	19
Worm	29	11
Cryptominer	27	11

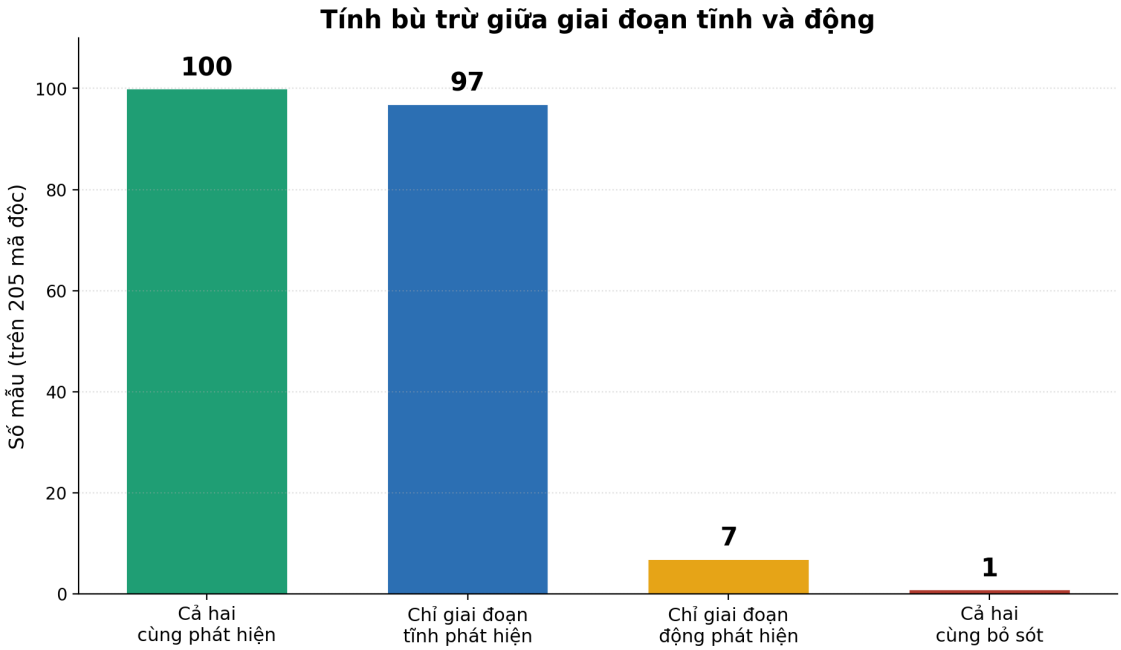
Bảng 5.10: Số lần các quy tắc MITRE cho kết quả dương tính giả trên 100 ứng dụng hợp pháp (mặt precision của ánh xạ hành vi).

Kỹ thuật	Mô tả	Số lần dương tính giả / 100 benign
T1055	Process Injection	7
T1003.001	LSASS Memory Access	5
T1218	Signed Binary Proxy	3

Tín hiệu tiêm mã tập trung ở nhóm injector/RAT (66/120) và ransomware (19/29), thấp hơn ở worm (11/29) phù hợp bản chất tự nhân bản thay vì tiêm mã. Ở mặt precision, tỉ lệ dương tính giả ở mức thấp trên 100 phần mềm hợp pháp (T1055: 7, T1003.001: 5, T1218: 3), tập trung ở các tiện ích có thao tác truy cập tiến trình như trình giám sát hệ thống và gỡ lỗi.

Tính bù trừ giữa hai giai đoạn

Phân rã 205 mẫu mã độc theo hai tín hiệu độc lập — giai đoạn tĩnh dự đoán `malware` và giai đoạn động phát hiện tiêm mã — cho thấy vai trò bù trừ rõ rệt (Hình 5.10). Mỗi mẫu rơi vào đúng một trong bốn trường hợp: cả hai giai đoạn cùng phát hiện, chỉ tĩnh phát hiện, chỉ động phát hiện, hoặc cả hai cùng bỏ sót. 100 mẫu được cả hai giai đoạn xác nhận; 97 mẫu chỉ giai đoạn tĩnh bắt được khi mẫu không bộc lộ tiêm mã trong sandbox; 7 mẫu ngược lại bị tĩnh chấm dưới ngưỡng nhưng được động thu hồi nhờ tín hiệu tiêm mã; chỉ 1 mẫu cả hai cùng bỏ sót. Nhờ cơ chế kết hợp, số mẫu bỏ sót cuối cùng chỉ còn 1/205, trong khi giai đoạn tĩnh dùng riêng bỏ sót 8 mẫu — phần lớn được giai đoạn động thu hồi. Kết quả định lượng vai trò bù trừ mà thiết kế tĩnh–động hướng tới.



Hình 5.10: Tính bù trừ giữa giai đoạn tĩnh và động trên 205 mẫu mã độc.

Kết quả thực nghiệm cho thấy mô hình tĩnh Top-384 đạt F1 bằng 0,9615 trên EMBER 2024, giai đoạn động thu được bằng chứng hành vi cụ thể ở 72% số mẫu, và khi hợp nhất hai giai đoạn hệ thống nhận diện đúng 204/205 mẫu mã độc — minh chứng cho vai trò bù trừ của thiết kế tĩnh–động. Những hạn chế còn lại và hướng phát triển được trình bày ở phần Kết luận.

KẾT LUẬN

Đồ án đã xây dựng một hệ thống hỗ trợ phát hiện và phân tích phần mềm độc hại trên nền tảng Windows, kết hợp phân tích tĩnh bằng học máy với phân tích động trong môi trường sandbox. Hệ thống vận hành tự động từ khâu nhận tệp PE đầu vào đến khâu xuất báo cáo, không yêu cầu can thiệp thủ công, đáp ứng các yêu cầu chức năng và phi chức năng đặt ra tại Chương 3. Điểm khác biệt chính so với các công cụ hiện có là khai thác ETW của .NET Runtime để quan sát hành vi nạp và sinh mã .NET trong bộ nhớ, đồng thời hợp nhất tín hiệu tĩnh và động để bù trừ cho nhau.

Giai đoạn tĩnh so sánh ba thuật toán Random Forest, XGBoost và LightGBM trên cùng tập dữ liệu; LightGBM đạt AUC-ROC cao nhất là 0,9993 và được chọn triển khai. Khảo sát số chiều cho thấy hiệu năng phân loại bão hòa từ khoảng 64 đặc trưng, tức tín hiệu tập trung ở một số ít đặc trưng; nhờ đó mô hình rút gọn còn 384 chiều vẫn giữ F1 ở mức 0,9615 trên tập EMBER 2024. Ngưỡng phân loại được đặt ở 0,33 theo định hướng ưu tiên giảm bỏ sót mã độc.

Giai đoạn động thu thập sự kiện từ Sysmon và SilkETW, sau đó chuẩn hóa, dựng cây tiến trình, xây dựng chuỗi hành vi, trích xuất chỉ số xâm phạm và ánh xạ sang khung MITRE ATT&CK. Trên 205 mẫu mã độc thuộc bảy nhóm hành vi (36 họ), 72% mẫu sinh bằng chứng hành vi cụ thể và hệ thống nhận diện đúng 204 mẫu. Kết quả của cả hai giai đoạn được tổng hợp thành một báo cáo JSON thống nhất và hiển thị qua giao diện web.

Quy mô đánh giá động giới hạn ở 305 mẫu (205 mã độc và 100 lành tính) do chi phí thực thi trong sandbox, nên kết quả mang tính minh chứng hơn là thống kê trên tập lớn. Khoảng 17% mẫu mã độc (34 trong 205) không bộc lộ hành vi trong cửa sổ quan sát do sử dụng kỹ thuật né tránh môi trường ảo hóa, vốn nằm ngoài phạm vi xử lý của đồ án.

Giai đoạn tĩnh vẫn sinh báo động giả trên một số phần mềm hợp pháp được đóng gói nặng, do cấu trúc tệp gần với mã độc. Cơ chế kết hợp kết quả hai giai đoạn dựa trên tập quy tắc cố định, chưa học tự động từ dữ liệu nên còn trường hợp quy tắc tiêm mã kích hoạt sai. Phạm vi đồ án cũng giới hạn ở tệp PE trên Windows, chưa bao gồm tài liệu chứa macro hay tệp kịch bản.

Hướng phát triển trước mắt là nâng cao khả năng quan sát đối với mẫu né tránh sandbox, thông qua giả lập tương tác người dùng, kéo dài thời gian quan sát và che giấu dấu hiệu môi trường ảo hóa. Cơ chế tổng hợp kết quả có thể được thay bằng

một mô hình học máy kết hợp đặc trưng tĩnh và động, giảm phụ thuộc vào tập quy tắc thủ công.

Về dữ liệu, quy mô đánh giá động có thể mở rộng bằng cách tự động hóa việc thực thi song song trên nhiều máy ảo. Về phạm vi, hệ thống có thể được mở rộng để phân tích thêm tài liệu Office chứa macro và tệp kịch bản, đồng thời bổ sung phân tích bộ nhớ và lưu lượng mạng nhằm làm giàu bằng chứng hành vi.

TÀI LIỆU THAM KHẢO

[1] AV-TEST Institute, *Malware statistics and trends*, Truy cập tháng 5 năm 2026, 2026. [Online]. Available: https://portal.av-atlas.org/malware/statistics

[2] IBM Security and Ponemon Institute, "Cost of a data breach report 2025," IBM, Tech. Rep., 2025. [Online]. Available: https://www.ibm.com/reports/data-breach

[3] Cybersecurity Ventures, *Cybercrime to cost the world $10.5 trillion annually by 2025*, Truy cập tháng 5 năm 2026, 2025. [Online]. Available: https://cybersecurityventures.com/cybercrime-damage-costs-10-trillion-by-2025/

[4] Verizon, *2026 data breach investigations report*, Truy cập tháng 5 năm 2026, 2026. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[5] Mandiant, *M-trends 2025: Special report*, Truy cập tháng 5 năm 2026, 2025. [Online]. Available: https://www.mandiant.com/m-trends

[6] CBS News, *WannaCry ransomware attacks: Losses could reach $4 billion*, Truy cập tháng 6 năm 2026, 2017. [Online]. Available: https://www.cbsnews.com/news/wannacry-ransomware-attacks-wannacry-virus-losses/

[7] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *Journal of Network and Computer Applications*, vol. 153, p. 102 526, 2020.

[8] SonicWall, *2025 sonicwall cyber threat report*, Truy cập tháng 5 năm 2026, 2025. [Online]. Available: https://www.sonicwall.com/threat-report/

[9] MITRE Corporation, *ATT&CK: Adversarial tactics, techniques, and common knowledge*, Truy cập tháng 5 năm 2026, 2018. [Online]. Available: https://attack.mitre.org/

[10] Y. Ye, T. Li, D. Adjeroh, and S. S. Iyengar, "A survey on malware detection using data mining techniques," *ACM Computing Surveys*, vol. 51, no. 3, pp. 1–40, 2017.

[11] H. Oz, A. Aris, A. Levi, and A. S. Uluagac, "A survey on ransomware: Evolution, taxonomy, and defense solutions," *CoRR*, vol. abs/2102.06249, 2021.

arXiv: 2102.06249. [Online]. Available: https://arxiv.org/abs/2102.06249

[12] B. Farinholt et al., "To catch a ratter: Monitoring the behavior of amateur darkcomet rat operators in the wild," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 770–787. DOI: 10.1109/SP.2017.48

[13] M. Rezaeirad, B. Farinholt, H. Dharmdasani, P. Pearce, K. Levchenko, and D. McCoy, "Schrödinger's RAT: Profiling the stakeholders in the remote access trojan ecosystem," in *Proceedings of the 27th USENIX Security Symposium*, 2018. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/rezaeirad

[14] A. I. Rashid et al., "Detecting remote access trojan (RAT) attacks based on different LAN analysis methods," *Engineering, Technology & Applied Science Research*, vol. 14, no. 5, pp. 17 294–17 301, 2024. DOI: 10.48084/etasr.8422

[15] C. C. Zou, D. Towsley, and W. Gong, "On the performance of internet worm scanning strategies," *Performance Evaluation*, vol. 63, no. 7, pp. 700–723, 2006. DOI: 10.1016/j.peva.2005.07.032 [Online]. Available: https://www.cs.ucf.edu/~czou/research/wormStrategy-elsevier.pdf

[16] P. Porras, H. Saïdi, and V. Yegneswaran, "A foray into Conficker's logic and rendezvous points," in *Proceedings of the 2nd USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET '09)*, Boston, MA: USENIX Association, 2009. [Online]. Available: https://www.usenix.org/conference/leet-09/foray-confickers-logic-and-rendezvous-points

[17] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," *arXiv preprint arXiv:1804.04637*, 2018. [Online]. Available: https://arxiv.org/abs/1804.04637

[18] A. Choudhary, S. Pawar, and Y. Haribhakta, *Efficient malware detection with optimized learning on high-dimensional features*, Accepted at ISAC3 2025, 2025. arXiv: 2506.17309 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2506.17309

[19] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324

[20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowl-*

edge Discovery and Data Mining, ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785

[21] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[22] Microsoft Sysinternals, *Sysmon – system monitor*, Truy cập tháng 5 năm 2026, 2023. [Online]. Available: https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon

[23] J. Desimone, *Hunting for in-memory .net attacks*, Elastic Security Labs, Truy cập tháng 6 năm 2026, 2022. [Online]. Available: https://www.elastic.co/security-labs/hunting-memory-net-attacks

[24] Microsoft, *About event tracing*, https://learn.microsoft.com/en-us/windows/win32/etw/about-event-tracing, Truy cập: 2026-05-25, 2024.

[25] R. Mathieu, *SilkETW & SilkService: Flexible C# wrappers for ETW*, Truy cập tháng 5 năm 2026, 2019. [Online]. Available: https://github.com/mandiant/SilkETW

[26] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, ACM, 2009, pp. 1113–1120. DOI: 10.1145/1553374.1553516

[27] "VirusShare : Malware repository." Truy cập: 2026-03-25. [Online]. Available: https://virusshare.com

[28] abuse.ch, *MalwareBazaar: A project to share malware samples with the infosec community*, Truy cập tháng 5 năm 2026, 2020. [Online]. Available: https://bazaar.abuse.ch/

[29] R. J. Joyce et al., "EMBER2024 – a benchmark dataset for holistic evaluation of malware classifiers," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025.

PHỤ LỤC

A. NGUỒN DỮ LIỆU

A.1 Thống kê mẫu lành tính theo nguồn

Bảng A.1 liệt kê 13 nguồn thu thập mẫu lành tính, gồm bảy phiên bản Windows khác nhau từ ISO gốc của Microsoft và sáu nguồn phần mềm hợp pháp bên ngoài.

Bảng A.1: Thống kê mẫu lành tính (benign) theo nguồn.

STT	Nguồn	Số mẫu
1	Windows 10 LTSC 2021 (en-US x64)	18.009
2	Windows 11 Consumer 25H2 (en-US x64)	21.540
3	Windows Server 2022 (en-US x64)	15.801
4	Windows Server 2016 (en-US x64)	12.168
5	Windows Server 2019 (en-US x64)	12.875
6	Windows 8.1 Update 1 (en-US x64)	10.856
7	Windows 7 SP1 Ultimate (en-US x64)	8.821
8	Windows 10 22H2 (en-US x64)	5.275
9	Chocolatey packages (batch 1)	1.813
10	Chocolatey packages (batch 2)	1.091
11	Sysinternals Suite 2025-03	405
12	Nirsoft utilities collection	505
13	GitHub open-source releases	843
	Tổng	**110.002**

A.2 Thống kê mẫu mã độc theo nguồn

Bảng A.2 liệt kê bốn tập VirusShare và tập MalwareBazaar được thu thập làm mẫu mã độc.

Bảng A.2: Thống kê mẫu mã độc theo nguồn.

STT	Nguồn	Số mẫu
1	VirusShare_00499	31.042
2	VirusShare_00495	32.891
3	VirusShare_00483	38.536
4	VirusShare_00480	28.603
5	MalwareBazaar (nhiều họ mã độc)	1.404
	Tổng	**132.476**

B. QUY TẮC ÁNH XẠ MITRE ATT&CK VÀ THAM CHIẾU KỸ THUẬT

B.1 Sysmon Event ID được theo dõi

Bảng B.1 liệt kê toàn bộ 23 Event ID Sysmon được ghi nhận trong phiên phân tích cùng nhóm hành vi tương ứng.

Bảng B.1: Các Sysmon Event ID được theo dõi và nhóm hành vi tương ứng.

Event ID	Nhóm hành vi	Mô tả
1	Tiến trình	Tạo tiến trình
2	Tệp	Thay đổi thời gian tạo tệp
3	Mạng	Kết nối mạng
5	Tiến trình	Kết thúc tiến trình
6	Khác	Nạp driver
7	DLL	Nạp thư viện động
8	Tiêm mã	Tạo luồng từ xa (remote thread)
9	Khác	Đọc thô đĩa cứng
10	Tiêm mã	Truy cập tiến trình khác
11	Tệp	Tạo tệp
12, 13, 14	Registry	Tạo/xóa khóa, ghi giá trị, đổi tên khóa registry
15	Tệp	Tạo luồng dữ liệu thay thế (ADS)
17, 18	Named pipe	Tạo / kết nối named pipe
19, 20, 21	WMI	Đăng ký WMI filter, consumer, binding
22	DNS	Truy vấn DNS
23, 26	Tệp	Xóa tệp
25	Tiêm mã	Giả mạo tiến trình

B.2 Các luật ánh xạ MITRE ATT&CK đã triển khai

Bảng B.2 liệt kê toàn bộ 23 kỹ thuật ánh xạ MITRE ATT&CK được triển khai trong bộ phân tích hành vi. Bốn kỹ thuật T1071.001, T1566.001, T1070.007 và T1568.002 được đánh mức tin cậy trung bình do tín hiệu kích hoạt có thể xuất hiện trong hoạt động bình thường; các kỹ thuật còn lại được đánh mức cao.

Bảng B.2: Các luật ánh xạ MITRE ATT&CK được triển khai trong bộ phân tích hành vi.

Kỹ thuật	Tên	Tín hiệu kích hoạt
T1003.001	LSASS Memory	Process access nhắm vào `lsass.exe`
T1053.005	Scheduled Task	Ghi vào khóa `ScheduledTask`
T1055	Process Injection	Remote thread (EID 8) hoặc process access (EID 10)
T1055.012	Process Hollowing	Giả mạo tiến trình (EID 25)
T1070	Indicator Removal	Xóa tệp log, shadow, backup hoặc .evt
T1070.007	Clear Persistence	Xóa khóa registry
T1071.001	App. Layer Protocol	Kết nối ra IP công khai ngoài mạng nội bộ
T1071.003	Named Pipe C2	Named pipe khớp từ khóa C2 đã biết
T1090.003	Multi-hop Proxy	Truy vấn DNS tên miền `.onion`
T1218	Signed Binary Proxy Exec.	Tiến trình là công cụ LOLBin
T1490	Inhibit System Recovery	Thao tác tệp shadow, VSS hoặc backup
T1543.003	Windows Service	Ghi vào khóa `Services`
T1546.003	WMI Event Subscription	Đăng ký WMI filter, consumer, binding
T1546.010	AppInit DLLs	Ghi vào khóa `AppInit_DLLs`
T1546.012	IFEO Injection	Ghi vào `Image File Execution`
T1547.001	Registry Run Keys	Khóa `CurrentVersion\Run` hoặc Startup
T1547.004	Winlogon Helper DLL	Ghi vào khóa `Winlogon`
T1547.005	Security Support Provider	Ghi vào khóa `SecurityProviders`
T1547.008	LSASS Driver	Ghi vào `Lsa\Notification`
T1566.001	Spearphishing Attachment	Tiến trình cha là ứng dụng Office hoặc Acrobat
T1568.002	Domain Generation Alg.	Entropy Shannon nhãn phụ $> 3{,}5$ bit và độ dài ≥ 12 ký tự
T1571	Non-Standard Port	Kết nối đến cổng C2 (4444, 1337, 8888, 9999, 31337, 8080, 8443)
T1620	Reflective Code Loading	Assembly .NET nạp qua `System.Reflection`

B.3 Endpoint và sự kiện SSE của server web demo

Bảng B.3 liệt kê tám endpoint REST của server; Bảng B.4 mô tả chín loại sự kiện trong luồng SSE.

Bảng B.3: Toàn bộ endpoint của server web demo.

Endpoint	Phương thức	Chức năng
`/`	GET	Phục vụ trang giao diện (`index.html`)
`/api/analyze`	POST	Nhận tệp mẫu và tham số; tạo `job_id`, lưu vào `uploads/`, khởi chạy pipeline trong luồng nền
`/api/jobs/<id>/stream`	GET	Luồng SSE; phát sự kiện pipeline theo thời gian thực
`/api/jobs/<id>/stop`	POST	Đặt cờ dừng, đẩy `stopped`+`eof` vào hàng đợi, tắt VM qua `VMManager`
`/api/jobs/<id>`	GET	Trả về trạng thái và kết quả cuối của một job
`/api/reports`	GET	Liệt kê tệp JSON trong `results/pipeline_runs/`, sắp xếp theo thời gian giảm dần
`/api/report/<filename>`	GET	Đọc và trả về nội dung một báo cáo JSON
`/api/report/<filename>`	DELETE	Xóa tệp báo cáo JSON khỏi thư mục `results/pipeline_runs/`

Bảng B.4: Chín loại sự kiện SSE trong luồng `/api/jobs/<id>/stream`.

Loại sự kiện	Nội dung
`ping`	Giữ kết nối; client bỏ qua
`started`	Pipeline bắt đầu; kèm tên tệp mẫu
`stage_start`	Giai đoạn bắt đầu; kèm số thứ tự `n` (1–12) và tên
`stage_done`	Giai đoạn hoàn thành; kèm số thứ tự `n`
`log`	Dòng nhật ký; kèm cấp độ (INFO/WARNING/ERROR) và nội dung
`done`	Pipeline hoàn tất; kèm toàn bộ báo cáo JSON
`stopped`	Pipeline bị người dùng dừng
`error`	Pipeline gặp lỗi; kèm thông báo lỗi
`eof`	Kết thúc luồng; client đóng `EventSource`

B.4 Danh sách lọc nhiễu trong giai đoạn động

Mục này tổng hợp các danh sách lọc nhiễu dùng ở giai đoạn động. Bảng B.5 liệt kê tám tiến trình thuộc `_DOTNET_NOISE_PROCS` bị loại khỏi đầu ra SilkETW;

Bảng B.6 liệt kê bảy từ khóa namespace thuộc `_SUSPICIOUS_NS` dùng để đánh dấu nghi ngờ; Bảng B.7 liệt kê 27 tiến trình hệ thống thường trực bị lọc khỏi cây tiến trình (`_TREE_NOISE`).

Bảng B.5: Tám tiến trình trong `_DOTNET_NOISE_PROCS` bị loại khỏi đầu ra bộ chuẩn hóa SilkETW.

Tiến trình	Lý do loại trừ
`msbuild`, `devenv`, `dotnet`, `vstest`, `testhost`	Công cụ phát triển .NET, không phải đối tượng phân tích
`silketw`	Tránh tự tham chiếu của công cụ thu thập
`powershell`, `pwsh`	Sysmon EID 1 đã ghi đầy đủ khi mẫu gọi shell

Bảng B.6: Bảy từ khóa trong `_SUSPICIOUS_NS` dùng đánh dấu sự kiện SilkETW nghi ngờ.

Từ khóa namespace	Hành vi phản ánh
`dynamicClass`	Tạo kiểu .NET động tại thời điểm chạy
`Invoke`	Gọi phương thức qua Reflection
`Emit`	Tự sinh mã IL lúc chạy (IL Emit)
`Reflection`	Truy cập metadata và bộ nhớ qua introspection
`RunspaceFactory`	Tạo môi trường PowerShell trong bộ nhớ
`AmsiUtils`	Vô hiệu hóa Antimalware Scan Interface
`System.Net`	Kết nối mạng từ bên trong .NET Runtime

27 tiến trình hệ thống thường trực sau đây bị lọc khỏi cây tiến trình (hằng `_TREE_NOISE`), chia theo bốn nhóm; tiến trình chỉ bị ẩn khi toàn bộ nhánh con cháu của nó cũng là nhiễu. Tên đã bỏ hậu tố `.exe`.

Bảng B.7: 27 tiến trình hệ thống bị lọc khỏi cây tiến trình (hằng `_TREE_NOISE`).

Nhóm	Tiến trình
Lõi và phiên Windows	`csrss`, `smss`, `wininit`, `winlogon`, `services`, `lsm`, `lsass`, `svchost`, `dllhost`, `conhost`, `taskhostw`, `runtimebroker`, `sihost`, `ctfmon`, `fontdrvhost`, `dwm`
Dịch vụ hệ thống và WMI	`spoolsv`, `msdtc`, `audiodg`, `wmiprvse`, `wsmprovhost`
Tìm kiếm và bảo mật Windows	`searchindexer`, `searchprotocolhost`, `securityhealthservice`, `securityhealthsystray`
Tiến trình VirtualBox	`vboxservice`, `vboxtray`

B.5 Endpoint của Agent HTTP trong máy ảo

Bảng B.8 liệt kê toàn bộ 11 endpoint của Agent HTTP chạy trong máy ảo Windows.

Bảng B.8: Toàn bộ endpoint của Agent HTTP trong máy ảo.

Endpoint	Phương thức	Chức năng
`/health`	GET	Kiểm tra Agent đã khởi động; trả về trạng thái và thời gian hoạt động
`/execute`	POST	Khởi chạy tệp mẫu dưới dạng tiến trình mới; trả về PID
`/shell`	POST	Thực thi lệnh shell tùy ý, đồng bộ hoặc nền (`async=true`)
`/upload`	POST	Ghi nội dung văn bản vào đường dẫn tùy chỉnh trên guest
`/read`	GET	Đọc nội dung tệp từ hệ thống tệp guest; Bridge dùng để poll log Sysmon
`/process_tree/{pid}`	GET	Liệt kê toàn bộ PID con và cháu của tiến trình gốc
`/process/{pid}`	GET	Kiểm tra trạng thái và thời gian chạy của một PID cụ thể
`/status`	GET	Trạng thái tổng thể tiến trình hiện tại; tự dừng nếu vượt timeout
`/stop`	POST	Dừng tiến trình đang chạy theo PID
`/logs`	GET	Trả về nhật ký nội bộ của Agent (sự kiện execute, stop, . . .)
`/screenshot`	GET	Chụp màn hình guest, trả về ảnh dạng Base64 PNG

B.6 Ánh xạ sự kiện chuẩn hóa vào nhóm hành vi

Bảng B.9 liệt kê 11 nhóm hành vi và các loại sự kiện chuẩn hóa tương ứng dùng trong mô-đun xây dựng chuỗi hành vi.

Bảng B.9: Ánh xạ 24 loại sự kiện chuẩn hóa vào 11 nhóm hành vi.

Nhóm hành vi	Loại sự kiện
Tiến trình	Tạo tiến trình, kết thúc tiến trình
Tiêm mã	Truy cập tiến trình khác, tạo luồng từ xa, giả mạo tiến trình
Nạp thư viện động	Nạp DLL
Tệp	Tạo tệp, xóa tệp, xóa tệp có ghi nhận, thay đổi thời gian tệp, tạo ADS
Registry	Tạo khóa, ghi khóa, xóa khóa
Mạng	Kết nối mạng
DNS	Truy vấn DNS
Named pipe	Tạo pipe, kết nối pipe
WMI	Đăng ký WMI filter, consumer, binding
.NET Runtime	Nạp assembly .NET
Khác	Nạp driver, đọc đĩa thô

B.7 Cấu trúc báo cáo JSON đầu ra của pipeline

Bảng B.10 liệt kê tên trường thực tế của từng nhóm trong báo cáo JSON theo thiết kế tại Mục 3.5.2.

Bảng B.10: Cấu trúc báo cáo JSON đầu ra của pipeline.

Nhóm	Trường JSON
Kết quả phân loại	`verdict`, `confidence`, `ml_label`, `ml_prob`, `static_result` (xác suất ML thô và thông tin PE cơ bản)
Thông tin hành vi	`behavioral_chain`, `process_tree`, `narrative`, `mitre_techniques`, `mitre_details` (mỗi kỹ thuật kèm tên, nguồn và mức tin cậy)
Chỉ số xâm phạm	`ioc` (10 danh mục: ips, domains, urls, files_created, files_deleted, registry_keys, commands, named_pipes, injected_pids, hashes)
Thông tin phiên	`metadata`: sha256, file_size, started_at, ended_at, duration_s, stages, event_count, ttp_count, analyzed_at
Cảnh báo và lỗi	`warnings`, `errors`